

ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN MÔN HỌC

Đề tài:

**CHATAPPZALO - HỆ THỐNG NHẮN TIN ĐA NỀN TẢNG
TÍCH HỢP GEMINI AI**

Lớp: DHKTPM18ATT

GVHD: Tôn Long Phước

Nhóm: 10

MSSV	Tên thành viên	Vai trò
20105921	Trần Phúc Thịnh	Nhóm trưởng
21022271	Phạm Tấn Đạt	Thành viên
20069151	Dương Trọng Tiến	Thành viên

TP. Hồ Chí Minh, 02/06/2026

LỜI NÓI ĐẦU

Trong quá trình học tập và thực hiện đồ án, nhóm nhận thấy một ứng dụng nhắn tin hiện đại không chỉ cần gửi và nhận nội dung nhanh. Người dùng còn cần tài khoản an toàn, xác thực rõ ràng, danh sách bạn bè, nhóm trò chuyện, gửi file, trạng thái online và giao diện chạy được trên nhiều thiết bị. Vì vậy, nhóm chọn xây dựng ChatAppZALO - hệ thống nhắn tin đa nền tảng tích hợp Gemini AI.

Đề tài được thực hiện theo hướng gần với một sản phẩm thực tế. Backend được xây dựng bằng Spring Boot, dữ liệu quan hệ lưu trong MySQL, dữ liệu tin nhắn và dữ liệu bình chọn, reaction và trạng thái hội thoại lưu bằng MongoDB. Giao diện web sử dụng React/Vite, ứng dụng di động sử dụng React Native/Expo. Các luồng realtime được triển khai thông qua WebSocket/STOMP để người dùng có thể nhận tin nhắn gần như tức thời.

Điểm nhóm muốn nhấn mạnh trong đề tài là tính đầy đủ của hệ thống. Ngoài các chức năng đăng ký, đăng nhập, chat cá nhân và chat nhóm, hệ thống còn có OTP qua email, cập nhật hồ sơ, avatar/cover, kết bạn, chặn người dùng, gửi file, thu hồi tin nhắn, reaction, ghim tin, đổi nền hội thoại, đặt biệt danh, bình chọn nhóm, gọi audio/video demo và Gemini AI hỗ trợ gợi ý nội dung.

LỜI CẢM ƠN

Nhóm xin gửi lời cảm ơn đến giảng viên hướng dẫn đã hỗ trợ, góp ý và định hướng trong quá trình thực hiện đồ án. Những góp ý về cách phân tích yêu cầu, trình bày báo cáo và tổ chức mã nguồn đã giúp nhóm hoàn thiện sản phẩm tốt hơn.

Nhóm cũng xin cảm ơn các bạn trong lớp đã hỗ trợ kiểm thử, góp ý giao diện và phản hồi các lỗi phát sinh trong quá trình chạy thử. Nhờ quá trình kiểm thử thực tế, nhóm có thêm cơ sở để chỉnh sửa các chức năng như đăng nhập, chat realtime, gửi file và tạo nhóm.

Do thời gian thực hiện có hạn, hệ thống vẫn còn một số điểm cần cải thiện. Nhóm mong nhận được thêm nhận xét để có thể tiếp tục phát triển đề tài trong các phiên bản sau.

MỤC LỤC

<u>Lời nói đầu</u>	2	
<u>Lời cảm ơn</u>	3	
<u>Danh mục hình ảnh</u>	5	
<u>Danh mục bảng biểu</u>	6	
<u>Chương 1: Tổng quan đề tài</u>	7	
<u>Chương 2: Cơ sở lý thuyết và công nghệ sử dụng</u>	10	
<u>Chương 3: Phân tích và thiết kế hệ thống</u>	16	
<u>3.6 Đặc tả use case chi tiết</u>	18	
<u>3.7 Thiết kế dữ liệu</u>	32	
<u>Chương 4: Hiện thực hệ thống</u>	34	
<u>4.8 Hiện thực giao diện Web</u>	39	
<u>4.10 Phiếu mô tả chi tiết các màn hình Web</u>	45	
<u>4.11 Phiếu mô tả chi tiết các màn hình Mobile</u>	63	
<u>Chương 5: Kiểm thử hệ thống</u>	76	
<u>Chương 6: Kết luận và hướng phát triển</u>	78	
<u>Tài liệu tham khảo</u>	80	
<u>Phụ lục A: Danh sách vị trí cần chèn ảnh</u>	81	
<u>Phụ lục C: Checklist hoàn thiện trước khi nộp</u>	82	

DANH MỤC HÌNH ẢNH

Hình 1. Mô hình kiến trúc tổng quan ChatApp

Hình 2. Hệ sinh thái công nghệ sử dụng

Hình 3. Luồng xử lý request backend

Hình 4. Luồng gửi tin nhắn realtime

Hình 5. Mô hình dữ liệu chính

Hình 6. Luồng upload file/ảnh

Hình 7. Quy trình AI Gemini

Hình 8. Giao diện web minh họa

Hình 9. Giao diện mobile minh họa

Hình 10. Các ảnh giao diện hệ thống

DANH MỤC BẢNG BIỂU

- Bảng 1. Mục tiêu đề tài
- Bảng 2. Phạm vi đề tài
- Bảng 3. Yêu cầu chức năng
- Bảng 4. Yêu cầu phi chức năng
- Bảng 5. Tác nhân hệ thống
- Bảng 6. Danh sách use case
- Bảng 7. Đặc tả use case
- Bảng 8. Danh sách API backend
- Bảng 9. Cấu trúc project
- Bảng 10. Bảng kiểm thử chức năng
- Bảng 11. Hạn chế và hướng phát triển

CHƯƠNG 1: TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Hiện nay, nhu cầu trao đổi thông tin trực tuyến ngày càng phổ biến trong học tập, làm việc nhóm và sinh hoạt cá nhân. Các nhóm sinh viên cần một kênh liên lạc nhanh để trao đổi bài tập, gửi file, tạo nhóm thảo luận và theo dõi thông tin mới. Một ứng dụng chat nếu chỉ có chức năng gửi tin nhắn văn bản sẽ chưa đủ đáp ứng tình huống sử dụng thực tế.

Bên cạnh tốc độ gửi nhận, người dùng cũng quan tâm đến sự an toàn của tài khoản và nội dung trao đổi. Các chức năng như xác thực OTP, quên mật khẩu, chặn người dùng, thu hồi tin nhắn và gợi ý nội dung và trợ lý AI giúp hệ thống hoàn chỉnh hơn. Vì vậy, đề tài ChatApp được lựa chọn để nhóm luyện tập xây dựng một hệ thống có nhiều thành phần và gần với môi trường phát triển phần mềm thực tế.

Đề tài còn giúp nhóm vận dụng kiến thức về backend, frontend, mobile, cơ sở dữ liệu và realtime communication. Thay vì chỉ tập trung vào một giao diện đơn lẻ, nhóm phải xử lý nhiều vấn đề như phân quyền API, lưu trữ dữ liệu, đồng bộ trạng thái giữa các client và thiết kế trải nghiệm người dùng.

1.2 Mục tiêu đề tài

Mục tiêu chính của đề tài là xây dựng một hệ thống nhắn tin đa nền tảng, có thể sử dụng trên web và mobile. Người dùng có thể đăng ký tài khoản, xác thực OTP, đăng nhập, cập nhật hồ sơ, tìm kiếm bạn bè, nhắn tin cá nhân, tạo nhóm, gửi file và sử dụng một số tính năng hỗ trợ an toàn nội dung.

Về kỹ thuật, đề tài hướng đến việc áp dụng các công nghệ phù hợp với mô hình ứng dụng web hiện đại: Spring Boot cho backend, MySQL/MongoDB cho dữ liệu, WebSocket/STOMP cho realtime, React cho web client và React Native/Expo cho mobile app. Cách tổ chức này giúp nhóm hiểu được cách các thành phần giao tiếp với nhau trong một hệ thống hoàn chỉnh.

Về sản phẩm, nhóm mong muốn giao diện dễ thao tác, luồng chức năng rõ ràng và có thể trình diễn được các nghiệp vụ chính. Báo cáo cũng được thiết kế để sau này nhóm có thể bổ sung ảnh chụp giao diện thật và các sơ đồ draw.io vào đúng vị trí.

Nhóm mục tiêu	Nội dung thực hiện
Chức năng	Đăng ký, OTP, đăng nhập, hồ sơ, bạn bè, block, chat 1-1, chat nhóm, upload file, thu hồi/xóa/reaction/ghim tin, đổi nền/biệt danh hội thoại, bình chọn nhóm, gọi audio/video demo và AI gợi ý tin nhắn.
Kỹ thuật	Spring Boot, Spring Security, JWT, MySQL, MongoDB, WebSocket/STOMP, React, Vite, React Native, Expo, Spring Mail, S3Service và Gemini API.

Giao diện	Có web client và mobile client; các màn hình chính được trình bày trong chương hiện thực để nhóm chụp ảnh bổ sung.
-----------	--

1.3 Đối tượng sử dụng

Đối tượng sử dụng chính của hệ thống là sinh viên, nhóm học tập, nhóm đồ án hoặc người dùng cá nhân có nhu cầu trao đổi nhanh. Người dùng có thể tạo danh sách bạn bè, tạo nhóm trò chuyện theo môn học hoặc theo dự án và chia sẻ tài liệu trong cuộc hội thoại.

Đối với người quản trị nhóm, hệ thống hỗ trợ các thao tác quản lý thành viên như thêm thành viên, xóa thành viên, đổi vai trò hoặc đổi thông tin nhóm. Những thao tác này giúp mô hình nhóm chat sát với nhu cầu làm việc nhóm thực tế.

Đối với người dùng cần hỗ trợ soạn nội dung, Gemini AI đóng vai trò trợ lý gợi ý cách viết lại tin nhắn hoặc trả lời trong hộp AI Chatbox. Project hiện tại dùng lớp AIService gọi Gemini API, không có module kiểm tra nội dung độc lập. Vì vậy báo cáo chỉ mô tả AI ở phạm vi gợi ý nội dung và trợ lý hội thoại.

1.4 Phạm vi đề tài

Phạm vi của đề tài bao gồm xây dựng backend, web client và mobile client. Backend cung cấp các API và websocket endpoint cho những chức năng chính. Web client dùng để trình diễn giao diện desktop/browser, còn mobile client dùng để thể hiện khả năng hoạt động trên điện thoại.

Về cơ sở dữ liệu, hệ thống sử dụng MySQL cho các dữ liệu có quan hệ rõ như người dùng, OTP, bạn bè, block và lịch sử cuộc gọi; MongoDB được dùng cho message, group và dữ liệu bình chọn, reaction và trạng thái hội thoại vì các dữ liệu này có cấu trúc linh hoạt hơn. File upload được lưu qua backend và có thể mở rộng sang S3 khi triển khai cloud.

Ngoài phạm vi của đề tài là triển khai production quy mô lớn, tối ưu Kubernetes, hệ thống thanh toán, mã hóa đầu cuối hoàn chỉnh và các tính năng AI phức tạp như RAG chatbot. Những phần này được đưa vào hướng phát triển sau.

Nội dung	Có trong phạm vi	Chưa đưa vào phạm vi
Nền tảng	Web React/Vite và Mobile React Native/Expo	Desktop app native
Realtime	WebSocket/STOMP cho chat, online status và call signal demo	Hạ tầng realtime quy mô lớn, media server production
Dữ liệu	MySQL, MongoDB, local upload cho file chat, S3 cho avatar/cover	Data warehouse/BI

AI	Gemini assistant, gợi ý viết lại tin nhắn	RAG chatbot, training model riêng
Media call	Gọi audio/video demo 1-1 ở mức signaling/giao diện	Gọi nhóm hoàn chỉnh, SFU/media server
Triển khai	Chạy local/dev, có cấu trúc Docker backend	Production Kubernetes/CI-CD hoàn chỉnh

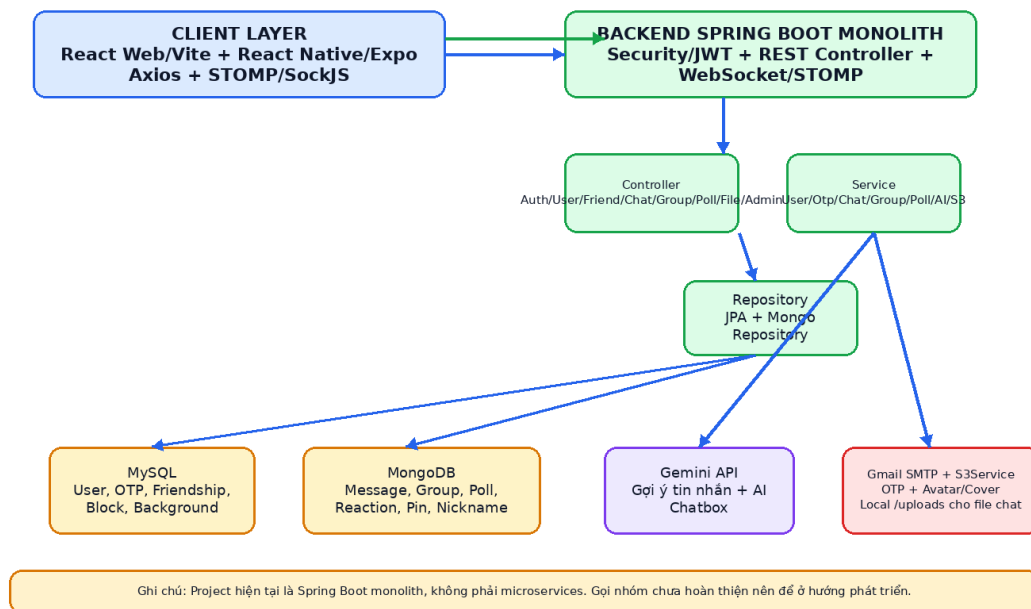
1.5 Ý nghĩa thực tiễn

Đề tài giúp nhóm hiểu rõ hơn cách một ứng dụng chat được tổ chức từ phía client đến server. Các chức năng như realtime, lưu trữ tin nhắn, quản lý bạn bè và upload file là những vấn đề thường gặp trong nhiều sản phẩm phần mềm hiện nay.

Việc có cả web và mobile giúp nhóm thấy được sự khác nhau trong cách tổ chức giao diện, lưu token và xử lý điều hướng. Backend phải được thiết kế đủ nhất quán để nhiều client cùng sử dụng mà không làm lặp lại logic nghiệp vụ ở từng nền tảng.

Gemini AI là phần bổ sung tạo điểm nhấn cho đề tài. Thay vì chỉ gửi tin nhắn, hệ thống có thêm gợi ý viết lại câu nhắn và hộp trợ lý AI để hỏi đáp theo context. Đây là hướng phù hợp với nhu cầu sử dụng AI hỗ trợ giao tiếp trong ứng dụng chat.

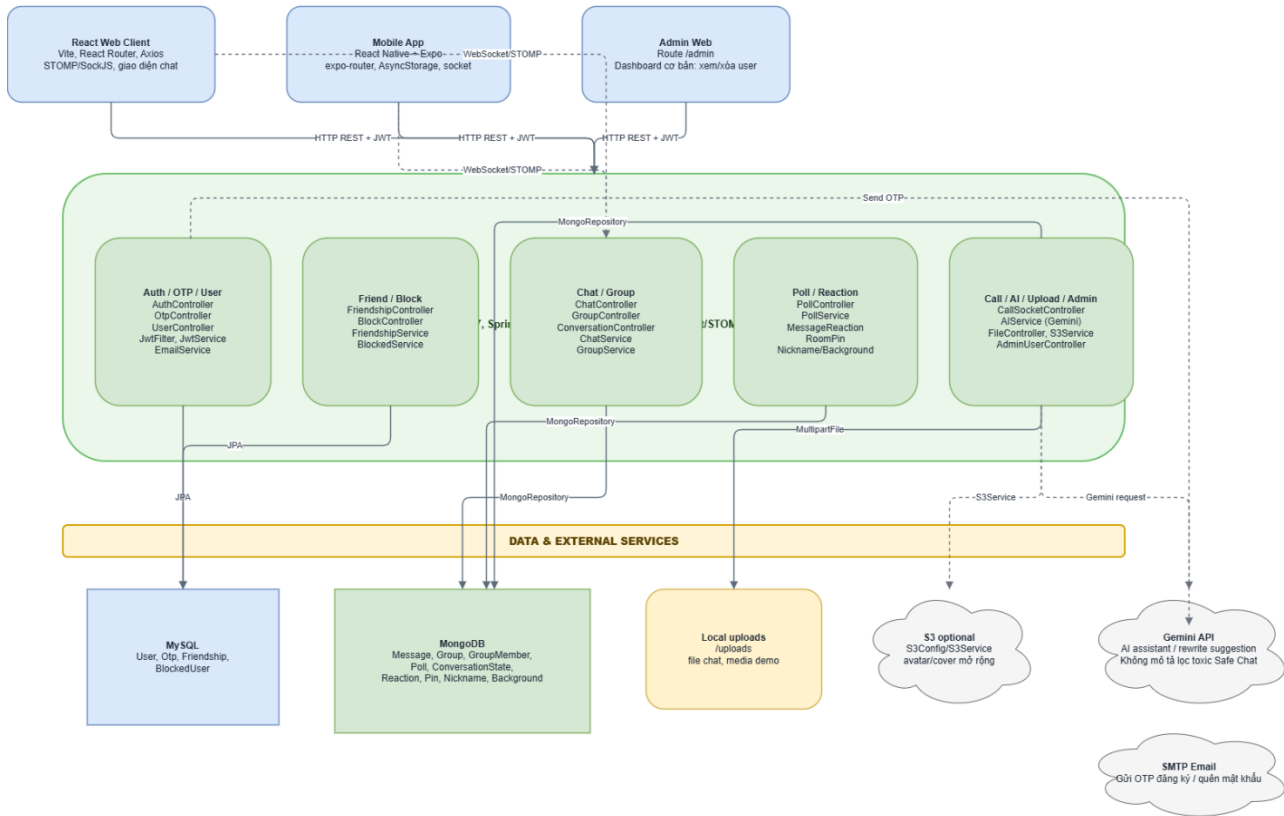
Kiến trúc tổng quan ChatAppZALO theo project hiện tại



Hình 1.1 Mô hình kiến trúc tổng quan của ChatApp

KIẾN TRÚC HỆ THỐNG CHATAPPZALO

Project thực tế: Spring Boot monolith + React Web + React Native/Expo + MySQL + MongoDB + WebSocket/STOMP + Gemini AI



CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG



Hình 2.1 Hệ sinh thái công nghệ sử dụng trong đề tài

2.1 Java 17

Java là ngôn ngữ chính dùng để xây dựng backend. Phiên bản Java 17 phù hợp với Spring Boot 3, hỗ trợ nhiều cải tiến về hiệu năng, cú pháp và khả năng bảo trì. Trong project, Java được dùng để định nghĩa entity, DTO, controller, service, repository và các lớp cấu hình hệ thống.



2.2 Spring Boot

Spring Boot giúp khởi tạo backend nhanh nhờ cơ chế auto-configuration, starter dependency và cách tổ chức dự án rõ ràng. Thay vì cấu hình thủ công nhiều thành phần, nhóm có thể tập trung vào nghiệp vụ như xác thực, quản lý người dùng, chat, nhóm và upload file.

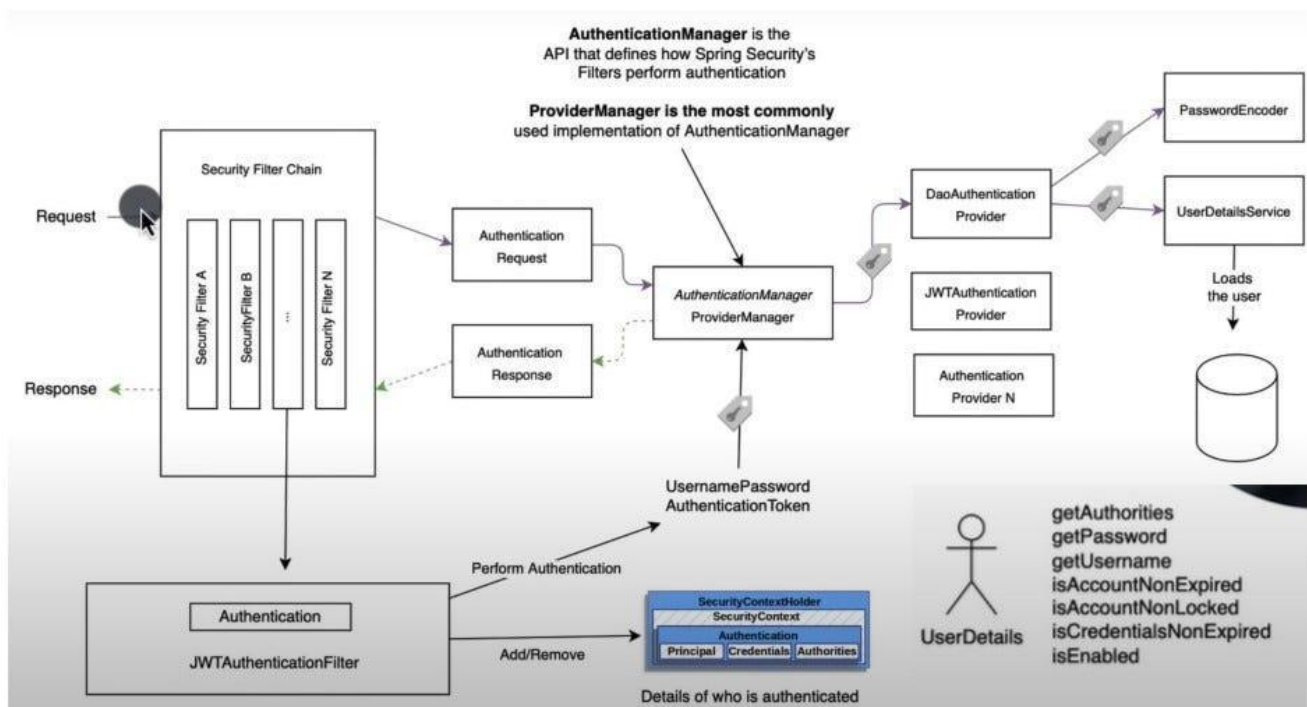


2.3 Spring MVC và REST API

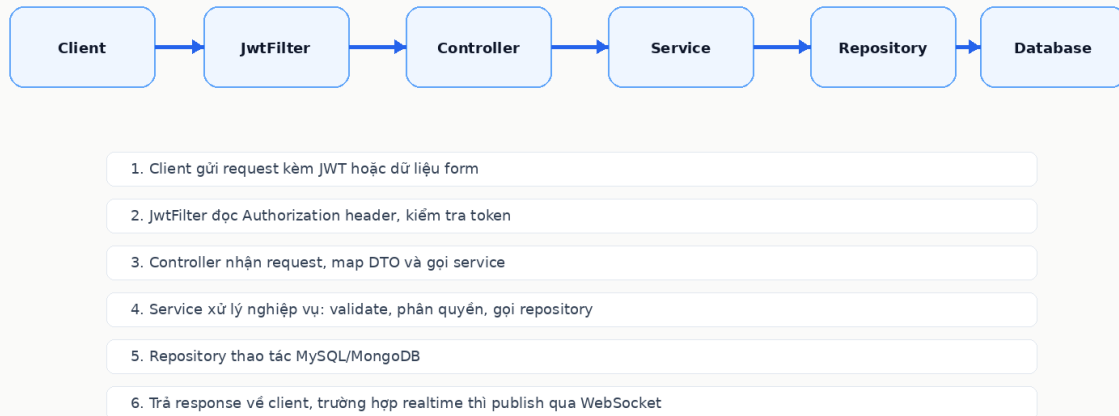
REST API là cách giao tiếp chính giữa client và server. Client gửi request HTTP đến các endpoint, backend xử lý dữ liệu và trả response JSON. Spring MVC hỗ trợ `@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping` và `@DeleteMapping` để xây dựng các API rõ ràng.

2.4 Spring Security

Spring Security được dùng để bảo vệ API. Các request cần đăng nhập sẽ đi qua bộ lọc JWT trước khi vào controller. Nếu token hợp lệ, backend xác định được user hiện tại; nếu token sai hoặc hết hạn, request bị từ chối.



Luồng xử lý request trong backend

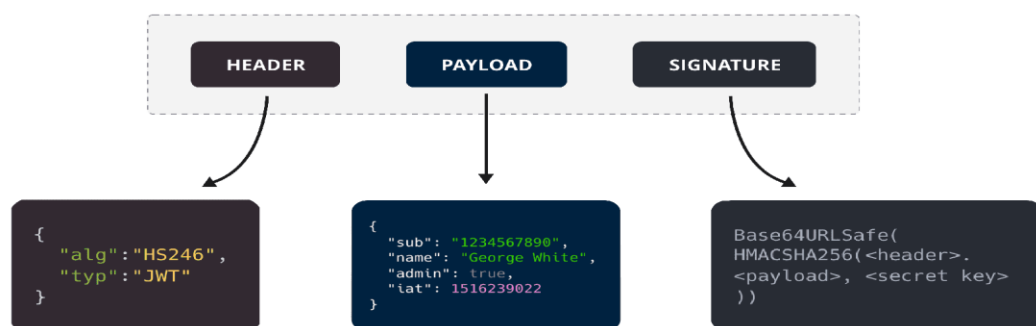


Hình 2.2 Luồng request qua JWT Filter, Controller, Service và Repository

2.5 JSON Web Token (JWT)

JWT là token chứa thông tin định danh người dùng và role. Sau khi đăng nhập thành công, backend sinh JWT và client lưu lại để gửi trong các request sau. Cách làm này giúp API không cần lưu session truyền thống trên server.

Structure of a JSON Web Token (JWT)



SuperTokens

2.6 Maven

Maven dùng để quản lý dependency và quá trình build backend. File pom.xml của project khai báo các thư viện như Spring Web, Spring Security, Spring Data JPA, MySQL connector, MongoDB, Mail, Validation, WebSocket, Lombok và JWT.

2.7 MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ. Trong hệ thống, MySQL phù hợp với các bảng có quan hệ rõ ràng như User, Friendship, BlockedUser, Otp và Call signal. Việc dùng khóa chính, khóa ngoại và ràng buộc dữ liệu giúp giảm lỗi logic.

2.8 JPA Repository

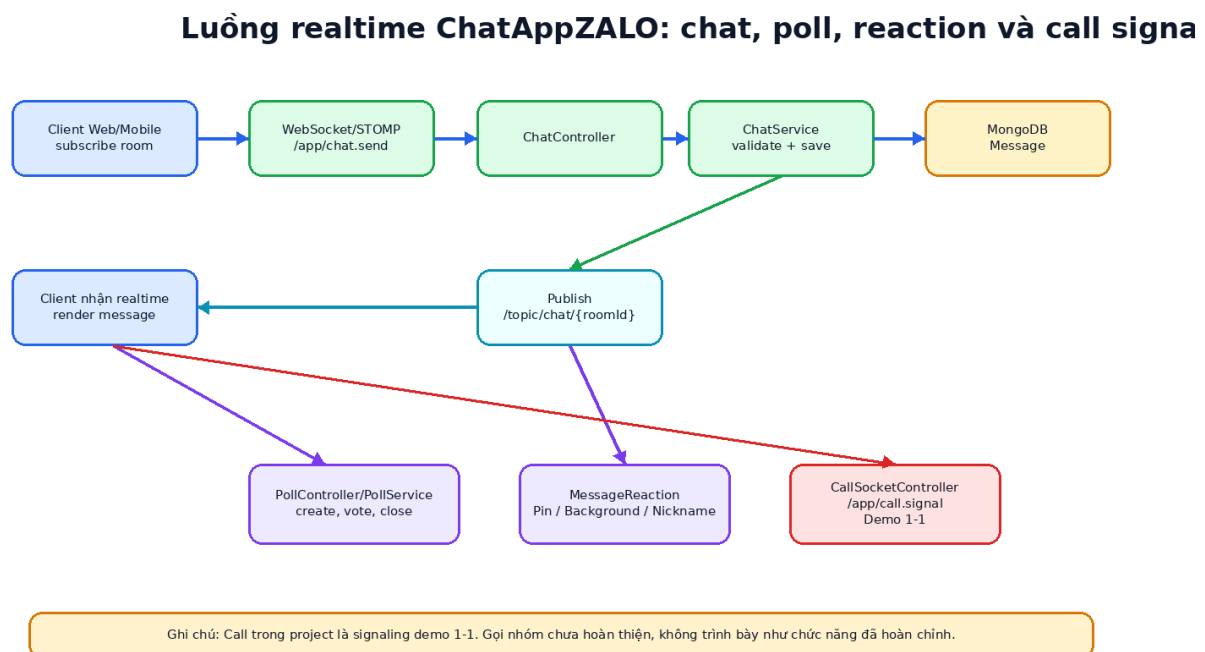
Spring Data JPA giúp thao tác với MySQL thông qua repository thay vì viết SQL thủ công cho mọi truy vấn. Các repository như UserRepository, FriendshipRepository, OtpRepository hỗ trợ tìm kiếm và lưu dữ liệu ngắn gọn.

2.9 MongoDB

MongoDB lưu dữ liệu dạng document. Đối với tin nhắn, nhóm và dữ liệu bình chọn, reaction và trạng thái hội thoại, cấu trúc dữ liệu có thể thay đổi theo loại message, trạng thái thu hồi, reaction hoặc seenBy. Vì vậy, MongoDB giúp lưu trữ linh hoạt hơn so với bảng quan hệ cứng.

2.10 WebSocket

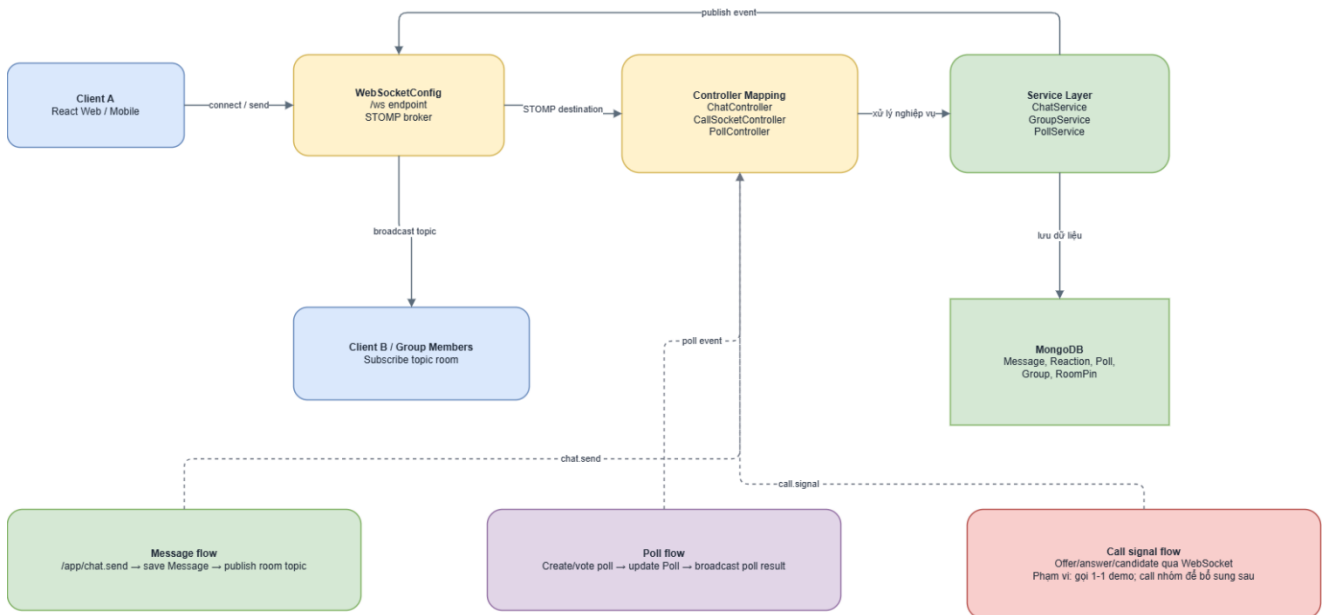
WebSocket cho phép thiết lập kết nối hai chiều giữa client và server. Khác với HTTP request-response thông thường, server có thể chủ động đẩy tin nhắn mới về client. Đây là nền tảng của chức năng chat realtime.



Hình 2.3 Luồng gửi tin nhắn realtime bằng WebSocket/STOMP

LUỒNG REALTIME: MESSAGE - CALL SIGNAL - POLL

WebSocket/STOMP cho tin nhắn, reaction, poll update và tín hiệu gọi 1-1 demo



GHI CHÚ CHÉN REPORT: Thay sơ đồ realtime cũ nếu có mô tả call nhóm hoàn chỉnh. Báo cáo chỉ nên ghi call nhóm chưa hoàn thiện/hướng phát triển.

2.11 STOMP và SockJS

STOMP giúp định nghĩa cách gửi/nhận message trên WebSocket theo dạng topic/destination. SockJS hỗ trợ fallback khi môi trường không hỗ trợ WebSocket thuần. Web client và mobile client đều có thể subscribe vào room để nhận tin nhắn.

2.12 React

React là thư viện xây dựng giao diện web theo hướng component. Trong project, các màn hình như LoginPage, RegisterPage, Chat, ChatBox, ChatInput, ChatList, FriendRequests, ProfileModal được tách thành các component để dễ bảo trì.

2.13 Vite

Vite là công cụ hỗ trợ phát triển frontend nhanh. Vite giúp chạy server dev, hot reload và build project React. Trong package.json của web client có các script như dev, build, lint và preview.

2.14 Axios và React Router

Axios được dùng để gọi API từ frontend đến backend. React Router quản lý điều hướng giữa các trang như login, register, forgot-password, chat và admin. ProtectedRoute kiểm tra token/role trước khi cho vào trang cần đăng nhập.

2.15 React Native

React Native cho phép xây dựng ứng dụng di động bằng JavaScript/TypeScript và React. Mobile app của project có các màn hình auth, chat, contacts, discovery, profile và edit profile.



2.16 Expo

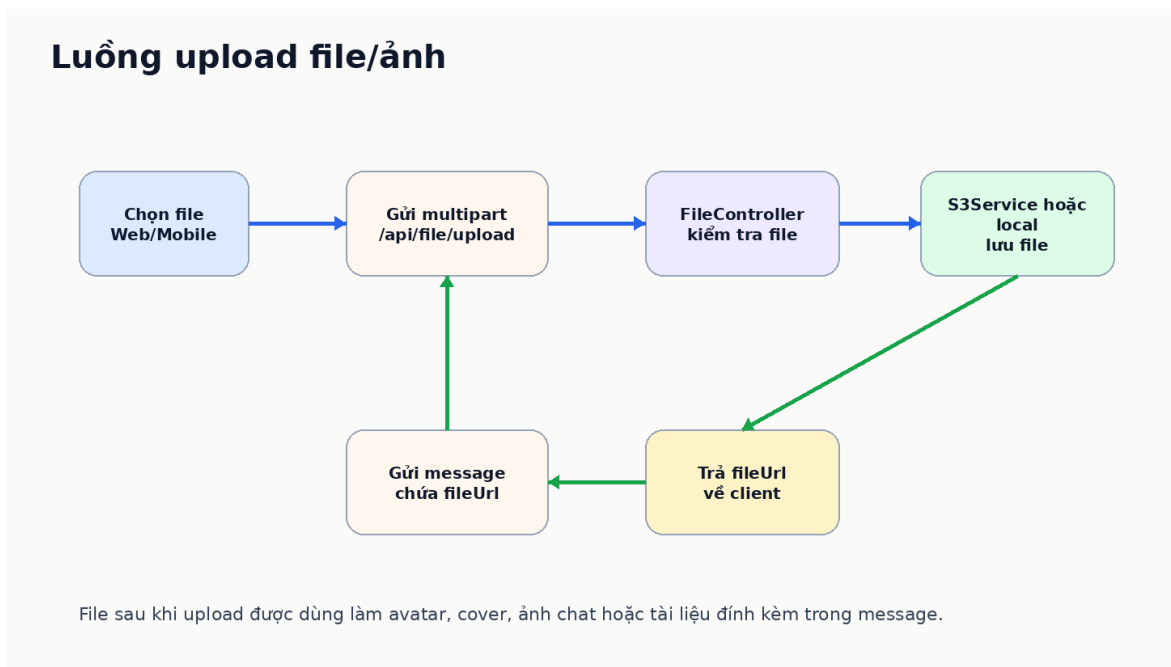
Expo giúp chạy, build và kiểm thử React Native nhanh hơn. Project mobile sử dụng expo-router để tổ chức route theo thư mục, có các tab chính và màn hình chat theo roomId.

2.17 Email OTP

OTP qua email được dùng trong đăng ký và quên mật khẩu. Backend sinh mã, lưu thời gian hết hạn và gửi email cho người dùng. Điều này giúp tăng độ tin cậy của tài khoản.

2.18 Upload file

Upload file cho phép người dùng gửi hình ảnh, tài liệu, avatar và ảnh bìa. Backend nhận MultipartFile, lưu vào storage và trả đường dẫn fileUrl để client hiển thị.

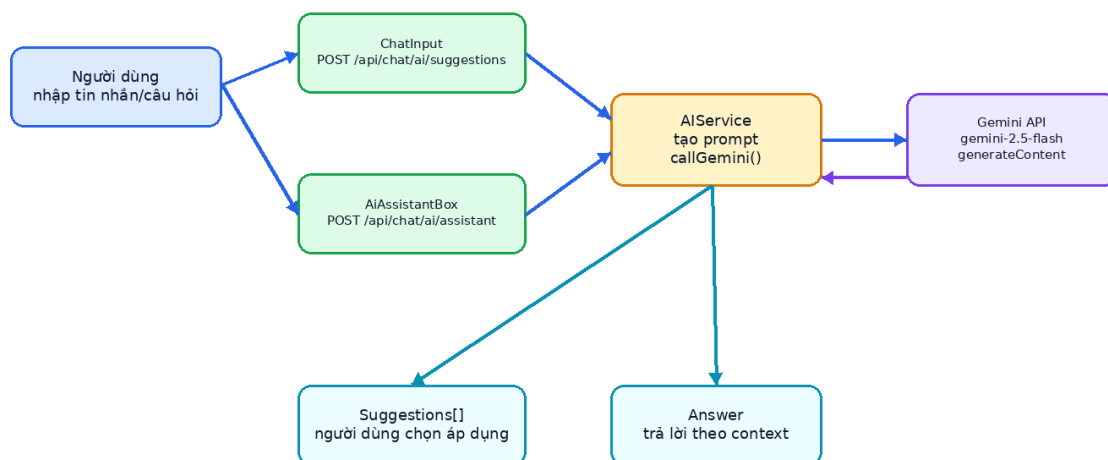


Hình 2.4 Luồng upload file/ảnh trong hệ thống

2.19 Gemini AI Assistant và gợi ý tin nhắn

Gemini AI trong đề tài được dùng cho hai chức năng chính: gợi ý viết lại tin nhắn và trợ lý AI trong khung chat. Backend có AIService gọi Gemini API; frontend có ChatInput để hiển thị gợi ý và AiAssistantBox để người dùng hỏi AI. Chức năng này hỗ trợ người dùng trước khi gửi tin, không tự động chặn nội dung và không thay thế quyết định của người dùng.

Quy trình Gemini AI Assistant

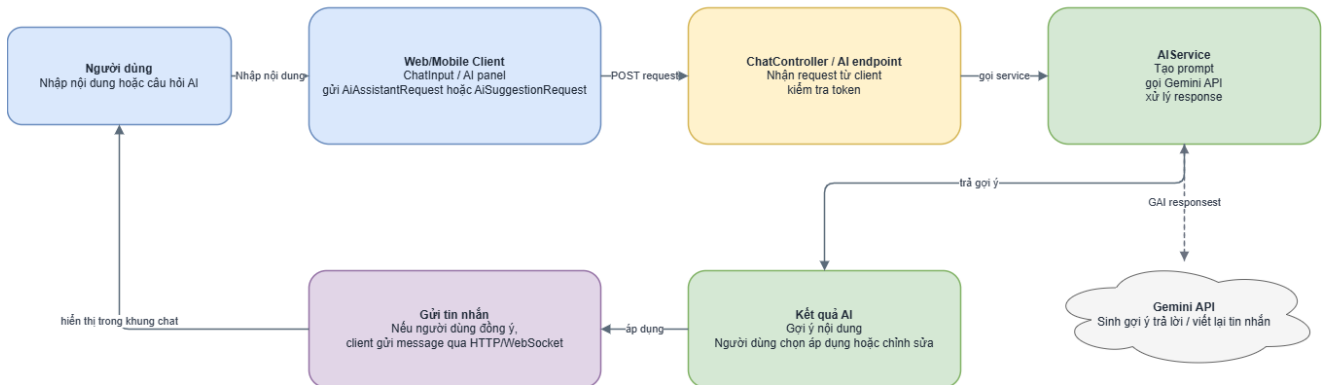


Ghi chú: Code hiện tại dùng AIService gọi Gemini API cho gợi ý viết lại tin nhắn và AI Chatbox. Không mô tả module kiểm tra nội dung độc lập.

Hình 2.5 Quy trình Gemini AI gợi ý tin nhắn và trợ lý hội thoại

LUỒNG GEMINI AI ASSISTANT / GỢI Ý TIN NHẮN

Đúng project hiện tại: AIService gọi Gemini API, không mở tá ToxicModerationService/ModerationLog.



2.20 Swagger/OpenAPI

OpenAPI/Swagger hỗ trợ xem và thử API trực tiếp trên trình duyệt. Trong project có OpenApiConfig và dependency springdoc-openapi-starter-webmvc-ui, thuận tiện cho việc kiểm tra endpoint backend.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Khảo sát yêu cầu hệ thống

Để xây dựng một ứng dụng chat hoàn chỉnh, hệ thống cần có các nhóm chức năng từ tài khoản, bạn bè, cuộc trò chuyện, nhóm, file, trạng thái online đến gợi ý nội dung và trợ lý AI. Các nhóm chức năng này liên hệ chặt chẽ với nhau nên cần phân tích rõ trước khi hiện thực.

Một yêu cầu quan trọng là backend phải phục vụ được cả web client và mobile client. Vì vậy, các API cần thống nhất dữ liệu trả về, cách xác thực và cách xử lý lỗi. Client không nên tự xử lý nghiệp vụ phức tạp mà chỉ gửi request và hiển thị kết quả từ server.

Đối với realtime, hệ thống cần đảm bảo người nhận có thể nhận tin nhắn mới mà không cần tải lại trang. WebSocket/STOMP được dùng để subscribe theo room, giúp tin nhắn mới được đẩy về client đang online.

3.2 Yêu cầu chức năng

Nhóm chức năng	Mô tả chi tiết
Tài khoản	Đăng ký, gửi OTP, xác thực OTP, đăng nhập, quên mật khẩu, đặt lại mật khẩu, đổi mật khẩu.
Hồ sơ	Xem thông tin cá nhân, cập nhật username, email/phone, giới tính, ngày sinh, avatar và cover image.
Bạn bè	Tìm kiếm người dùng, gửi lời mời, chấp nhận, từ chối, xem danh sách bạn bè và lời mời.
Chặn người dùng	Block/unblock, kiểm tra trạng thái chặn hai chiều, ngăn gửi tin nhắn khi bị chặn.
Chat cá nhân	Tạo room theo cặp user, gửi tin nhắn văn bản/file, xem lịch sử, xóa/khôi phục cuộc trò chuyện phía người dùng.
Chat nhóm	Tạo nhóm, xem nhóm của mình, thêm/xóa thành viên, đổi role, đổi tên nhóm, đổi avatar nhóm, rời nhóm, xóa nhóm.
Tin nhắn nâng cao	Thu hồi tin nhắn, xóa phía người dùng, reaction emoji, ghim/bỏ ghim tin nhắn và trạng thái tin nhắn.
Bình chọn trong nhóm	Tạo poll, thêm lựa chọn, bình chọn, đóng poll và hiển thị kết quả trong room.
Cá nhân hóa hội thoại	Đổi nền cuộc trò chuyện, đặt biệt danh cho thành viên trong hội thoại.
Upload	Upload file chat lưu local /uploads, upload avatar/cover qua S3Service.
Realtime	Gửi/nhận message qua WebSocket, call signal demo, danh sách user online.
AI Gemini	Gợi ý viết lại tin nhắn, trợ lý AI trả lời theo context hội thoại.
Admin	Xem danh sách user và xóa user trong trang quản trị.

3.3 Yêu cầu phi chức năng

Yêu cầu	Mô tả
Bảo mật	API cần xác thực bằng JWT, mật khẩu được mã hóa, endpoint quan trọng cần phân quyền.
Dễ mở rộng	Tách controller, service, repository, DTO và entity để dễ bổ sung chức năng.
Tính ổn định	Các lỗi phổ biến cần được xử lý qua GlobalExceptionHandler hoặc response rõ ràng.
Hiệu năng	Tin nhắn realtime cần trả về nhanh; dữ liệu lịch sử nên truy vấn theo room.
Khả dụng đa nền tảng	Web và mobile dùng chung backend, đảm bảo dữ liệu hiển thị nhất quán.
Trải nghiệm	Giao diện cần dễ thao tác, thông báo lỗi rõ ràng, hạn chế reload không cần thiết.
Bảo trì	Tên file, tên API và cấu trúc thư mục cần dễ hiểu để các thành viên tiếp tục phát triển.

3.4 Tác nhân hệ thống

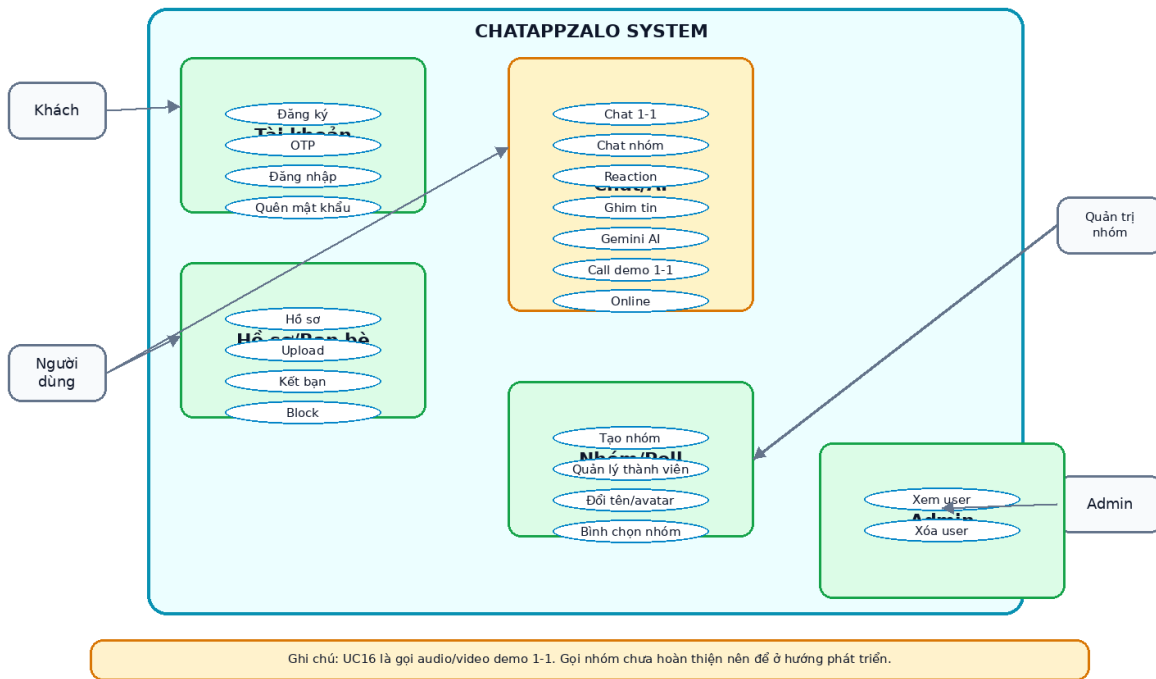
Tác nhân	Mô tả	Quyền/Chức năng chính
Khách	Người chưa đăng nhập	Đăng ký, gửi OTP, xác thực OTP, đăng nhập, quên mật khẩu.
Người dùng	Người đã đăng nhập và có token hợp lệ	Quản lý hồ sơ, kết bạn, chat cá nhân, chat nhóm, upload, bình chọn nhóm, AI Gemini và gọi demo.
Quản trị nhóm	Thành viên có role quản trị trong nhóm	Thêm/xóa thành viên, đổi role, đổi tên nhóm, đổi avatar, xóa nhóm.
Admin hệ thống	Tài khoản có role ADMIN	Xem danh sách user, xóa user.
Hệ thống	Backend và các service tự động	Xác thực JWT, gửi OTP, lưu dữ liệu, phát realtime, gợi ý nội dung và trợ lý AI.

3.5 Danh sách use case

Mã UC	Tên use case	Actor chính
UC01	Đăng ký tài khoản	Khách
UC02	Gửi và xác thực OTP	Khách

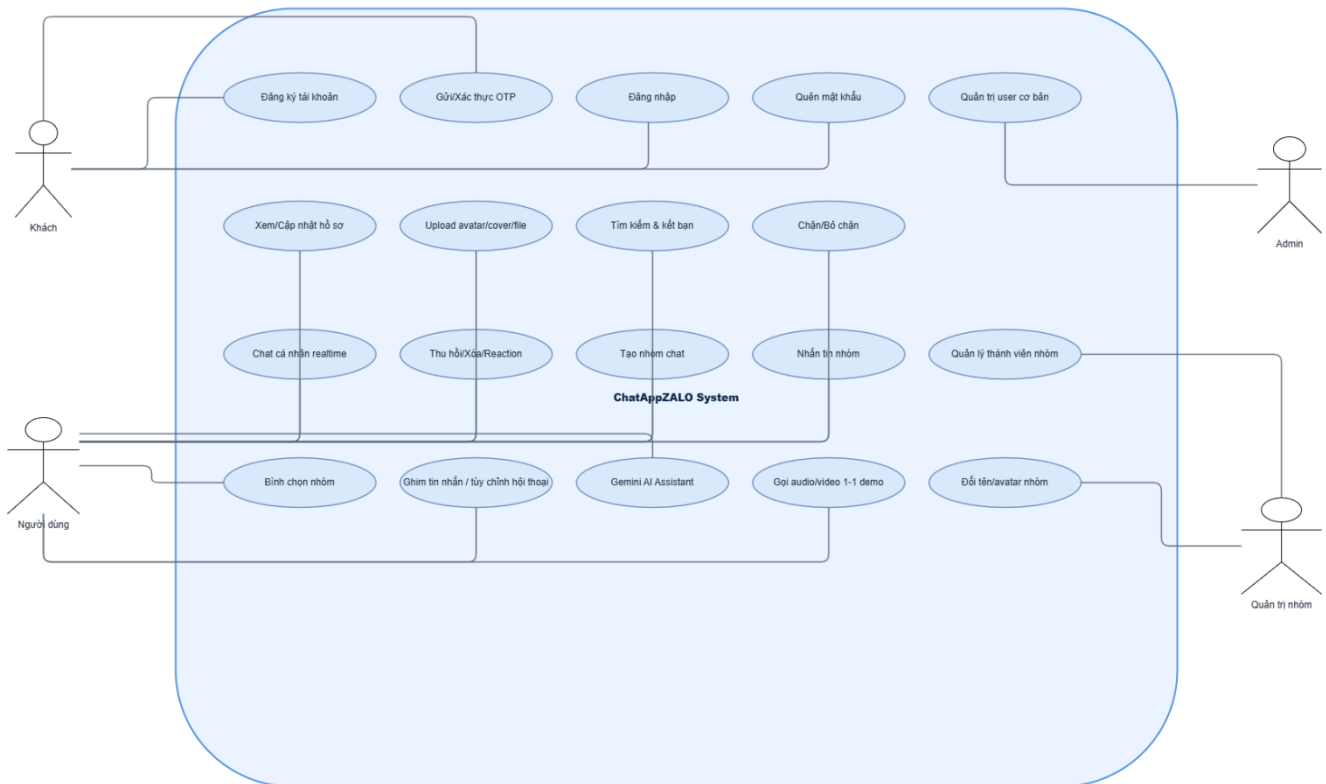
UC03	Đăng nhập hệ thống	Khách
UC04	Quên và đặt lại mật khẩu	Khách
UC05	Xem và cập nhật hồ sơ cá nhân	Người dùng
UC06	Upload avatar/cover/file	Người dùng
UC07	Tìm kiếm và kết bạn	Người dùng
UC08	Chấp nhận/từ chối lời mời kết bạn	Người dùng
UC09	Chặn và bỏ chặn người dùng	Người dùng
UC10	Nhắn tin cá nhân realtime	Người dùng
UC11	Thu hồi, xóa và reaction tin nhắn	Người dùng
UC12	Tạo nhóm chat	Người dùng
UC13	Quản lý thành viên nhóm	Quản trị nhóm
UC14	Nhắn tin nhóm	Người dùng
UC15	AI gợi ý tin nhắn và trợ lý Gemini	Người dùng
UC16	Gọi audio/video demo 1-1	Người dùng
UC17	Xem trạng thái online	Người dùng
UC18	Quản trị người dùng	Admin hệ thống
UC19	Tạo và quản lý bình chọn trong nhóm	Người dùng/Quản trị nhóm

Use Case tổng quát ChatAppZALO

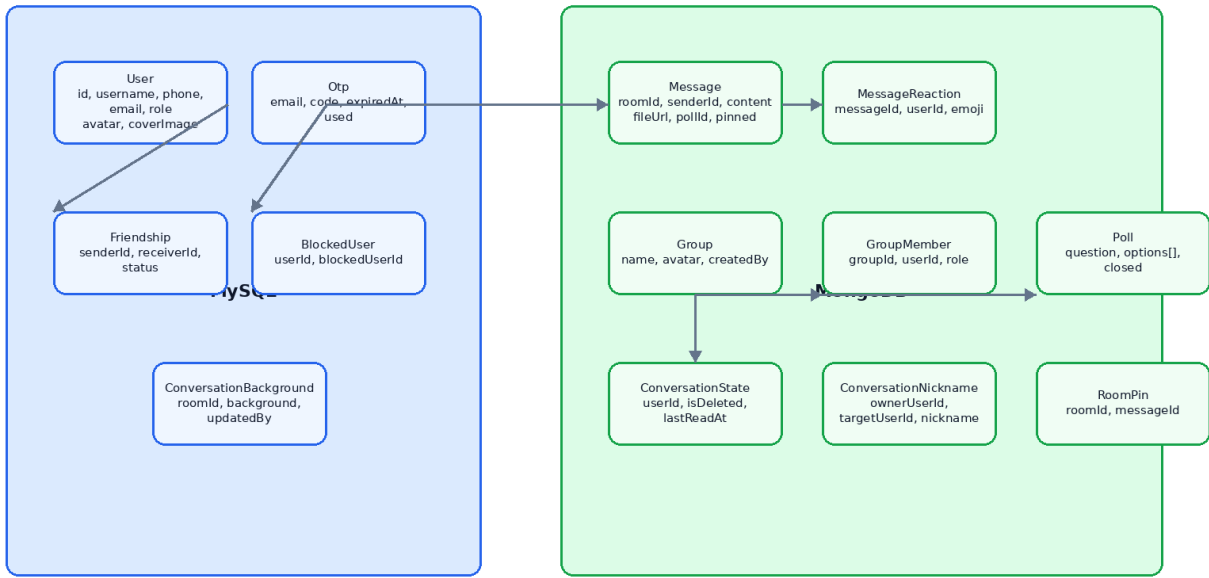


Hình 3.1 Use case tổng quát của ChatAppZALO

USE CASE TỔNG QUÁT CHATAPPZALO



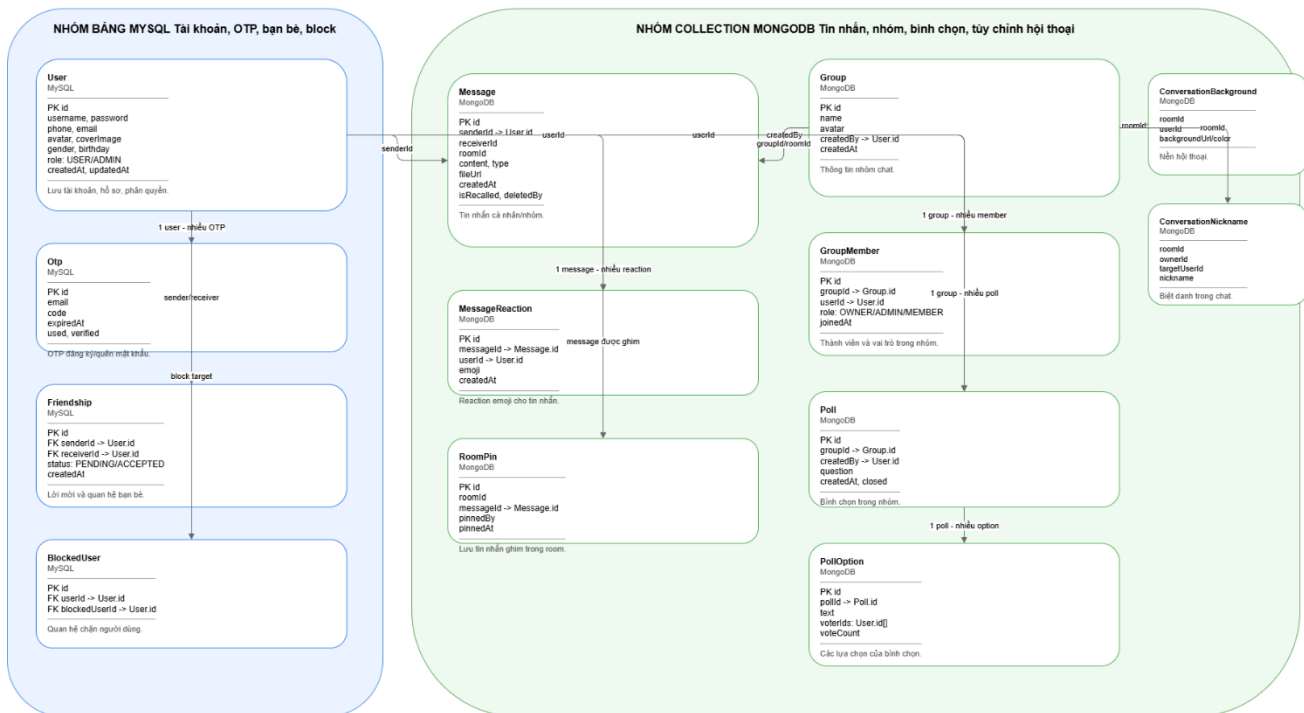
ERD dữ liệu chính ChatAppZALO



Ghi chú: Dữ liệu đúng với code hiện tại: Poll, MessageReaction, ConversationBackground, ConversationNickname và RoomPin đều cần thể hiện trong ERD.

Hình 3.2 Mô hình dữ liệu chính của ChatAppZALO

ERD THIẾT KẾ DỮ LIỆU - CHATAPPZALO



3.6 Đặc tả use case chi tiết

3.6.01 UC01: Đăng ký tài khoản

Mã UC	UC01
-------	------

Tên use case	Đăng ký tài khoản
Mô tả	Khách tạo tài khoản mới bằng thông tin đăng ký.
Actor chính	Khách
Tiền điều kiện	Khách chưa đăng nhập; email/phone chưa được dùng trong hệ thống.
Hậu điều kiện	User được tạo, mật khẩu mã hóa, OTP được gửi để xác thực.

ST T	Người dùng/Client	Hệ thống
1	Mở màn hình đăng ký.	Trả giao diện form đăng ký.
2	Nhập username, email/phone, mật khẩu và xác nhận mật khẩu.	Kiểm tra dữ liệu đầu vào.
3	Bấm nút Đăng ký.	AuthController nhận RegisterRequest.
4		UserService kiểm tra email/phone đã tồn tại hay chưa.
5		Mã hóa mật khẩu và lưu User.
6		OtpService sinh OTP, EmailService gửi OTP.
7	Nhận thông báo yêu cầu xác thực OTP.	Trả response đăng ký thành công/chờ xác thực.

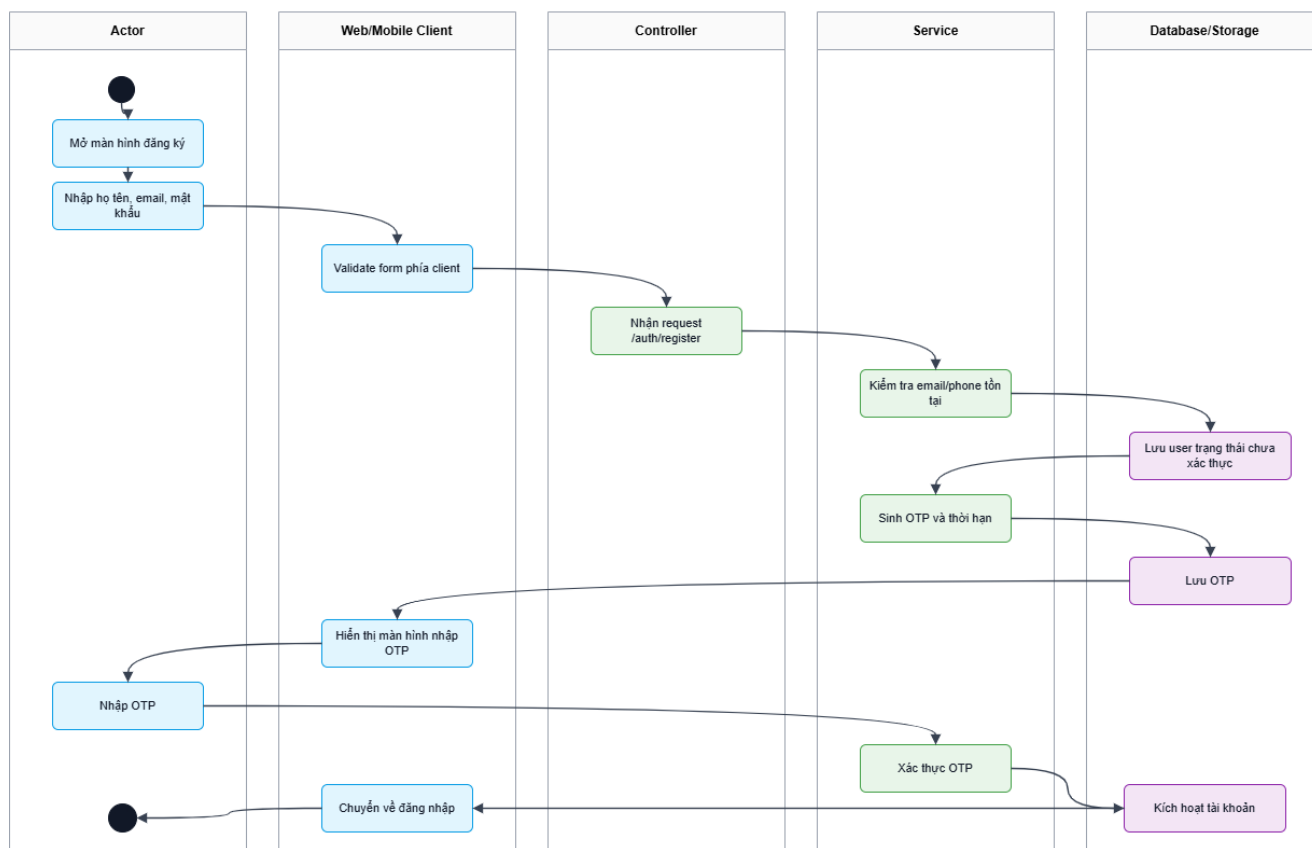
Mã	Luồng thay thế
A1	Email đã tồn tại nhưng chưa xác thực: hệ thống cho gửi lại OTP.
A2	Người dùng nhập thiếu trường: giao diện giữ dữ liệu đã nhập và hiển thị lỗi.

Mã	Luồng ngoại lệ
E1	Email/phone đã tồn tại: hệ thống trả lỗi và không tạo tài khoản.
E2	Mật khẩu không hợp lệ: hệ thống yêu cầu nhập lại.

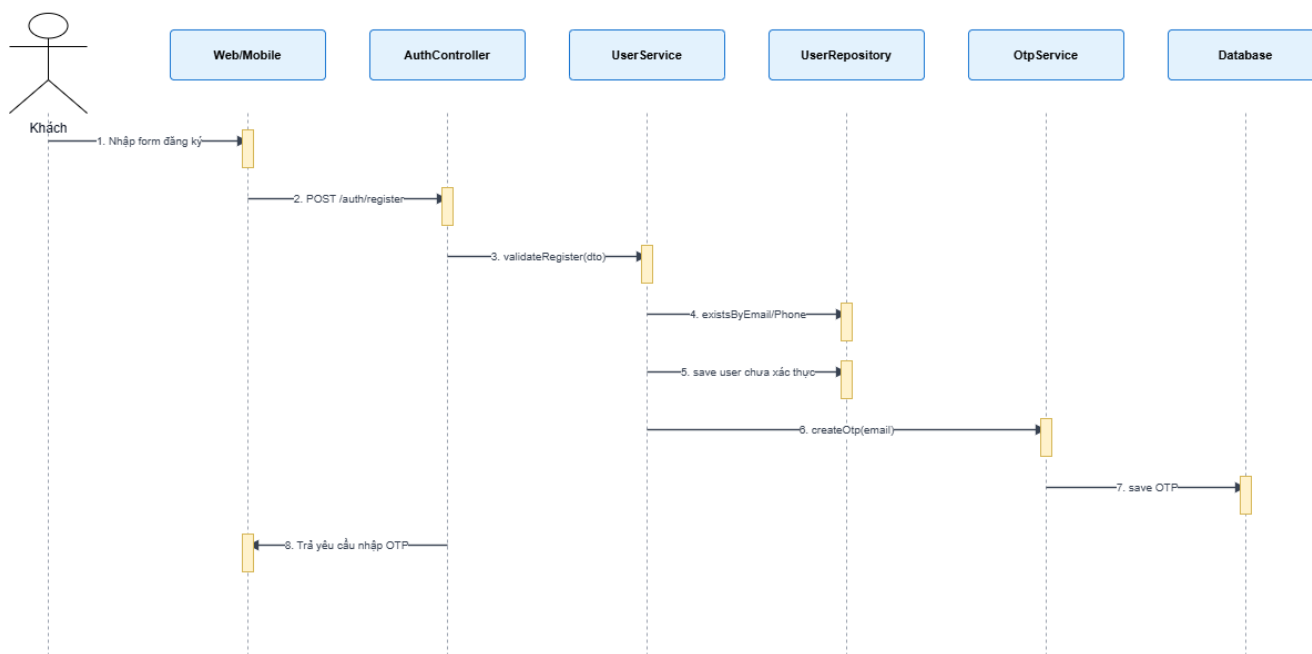
Ghi chú nghiệp vụ:

- Liên quan AuthController /register, UserService.register, OtpService.sendOtp.

Activity Diagram - UC01: Đăng ký tài khoản



Sequence Diagram - UC01: Đăng ký tài khoản



3.6.02 UC02: Gửi và xác thực OTP

Mã UC	UC02
Tên use case	Gửi và xác thực OTP
Mô tả	Khách nhận mã OTP qua email và nhập mã để xác thực tài khoản hoặc đặt lại mật khẩu.
Actor chính	Khách
Tiền điều kiện	OTP đã được sinh và còn thời hạn.
Hậu điều kiện	OTP được xác nhận, tài khoản hoặc yêu cầu reset password được chấp nhận.

ST T	Người dùng/Client	Hệ thống
1	Nhập email để gửi OTP hoặc nhập mã OTP đã nhận.	OtpController nhận request.
2	Bấm gửi/xác thực OTP.	OtpService tìm OTP mới nhất theo email.
3		Kiểm tra mã, thời hạn và trạng thái used/verified.
4		Cập nhật OTP đã dùng hoặc đã xác thực.
5	Nhận thông báo xác thực thành công.	Trả response thành công.

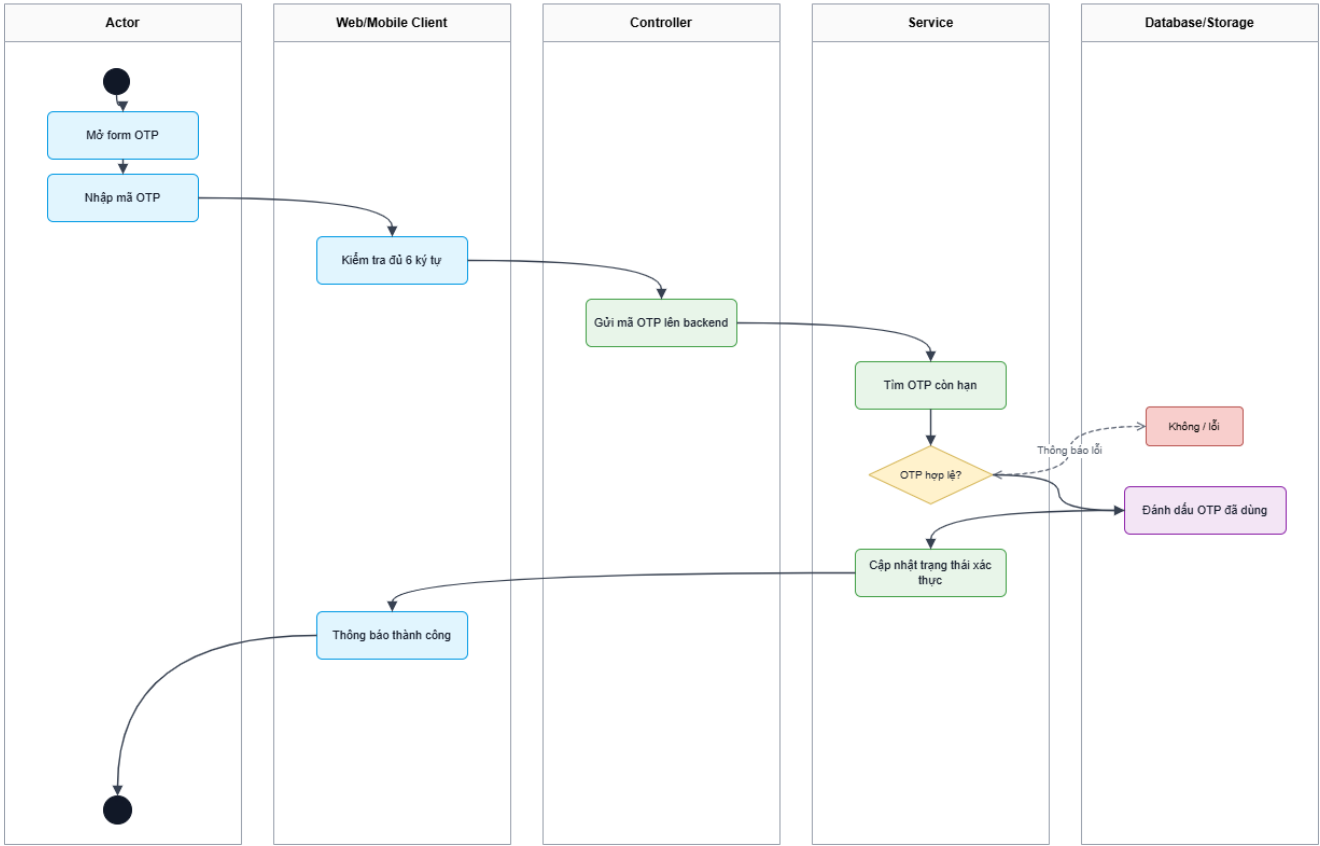
Mã	Luồng thay thế
A1	Người dùng yêu cầu gửi lại OTP: hệ thống sinh mã mới và gửi lại email.

Mã	Luồng ngoại lệ
E1	OTP sai: trả lỗi và cho nhập lại.
E2	OTP hết hạn: yêu cầu gửi mã mới.

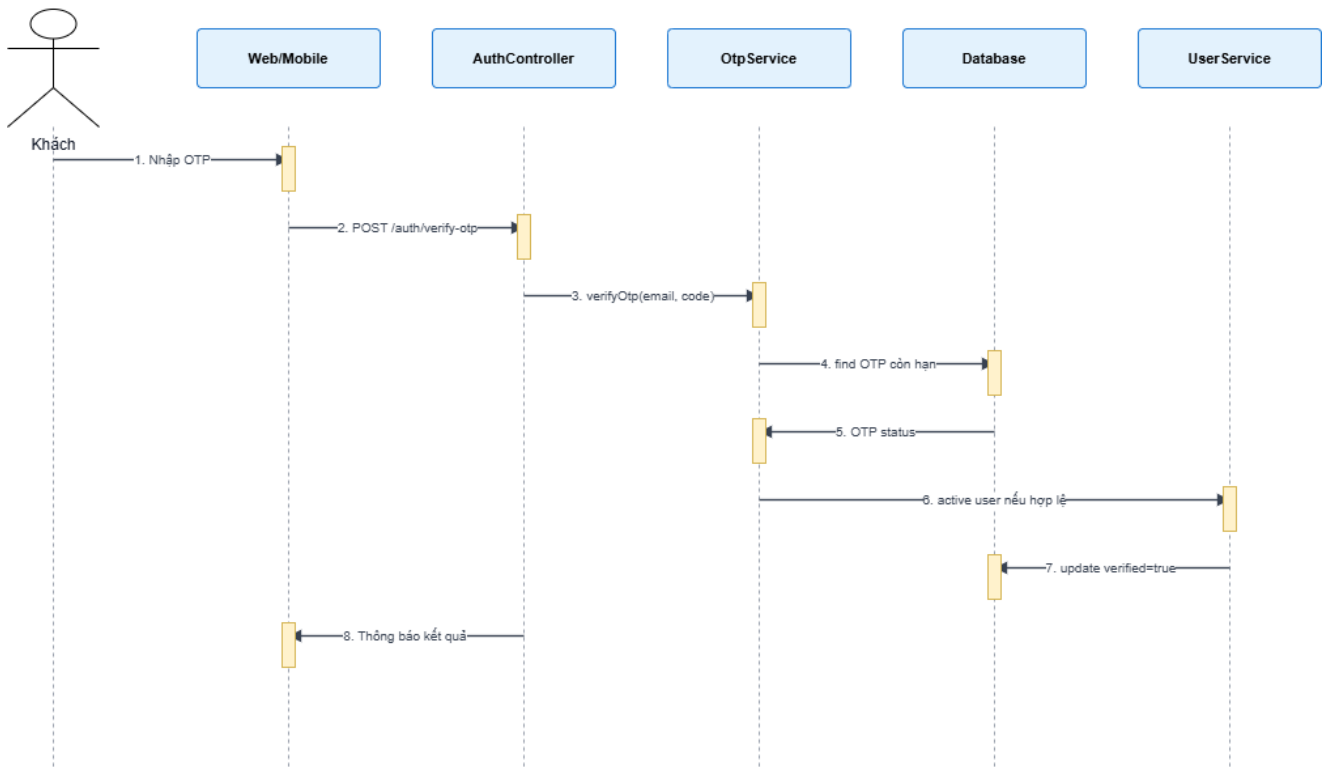
Ghi chú nghiệp vụ:

- Liên quan OtpController /send-otp và /verify-otp.

Activity Diagram - UC02: Gửi và xác thực OTP



Sequence Diagram - UC02: Gửi và xác thực OTP



3.6.03 UC03: Đăng nhập hệ thống

Mã UC	UC03
Tên use case	Đăng nhập hệ thống
Mô tả	Người dùng đăng nhập bằng email/phone và mật khẩu để nhận JWT.
Actor chính	Khách
Tiền điều kiện	Tài khoản đã tồn tại và đã xác thực.
Hậu điều kiện	JWT được tạo, client lưu token và chuyển vào trang chat.

ST T	Người dùng/Client	Hệ thống
1	Mở màn hình đăng nhập.	Trả form đăng nhập.
2	Nhập thông tin đăng nhập.	AuthController nhận LoginRequest.
3	Bấm Đăng nhập.	UserService tìm user theo email/phone.
4		So khớp mật khẩu đã mã hóa.
5		JwtService sinh token kèm subject và role.
6	Nhận token và thông tin user.	Trả LoginResponse.
7	Chuyển vào trang chat.	Client có thể bắt đầu kết nối WebSocket.

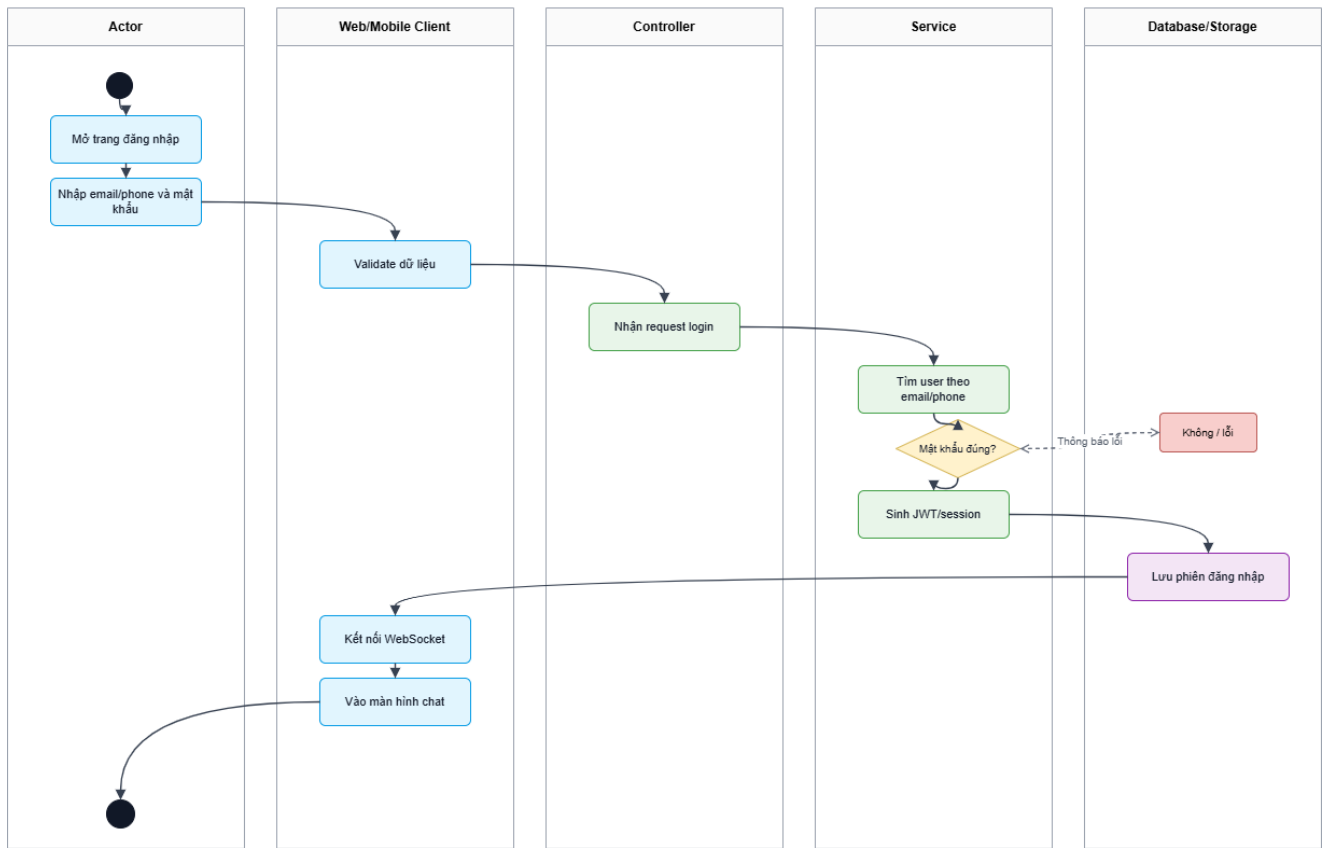
Mã	Luồng thay thế
A1	Nếu user có role ADMIN, client điều hướng sang trang admin.

Mã	Luồng ngoại lệ
E1	Sai mật khẩu: trả lỗi Unauthorized.
E2	Tài khoản không tồn tại: hiển thị lỗi đăng nhập.

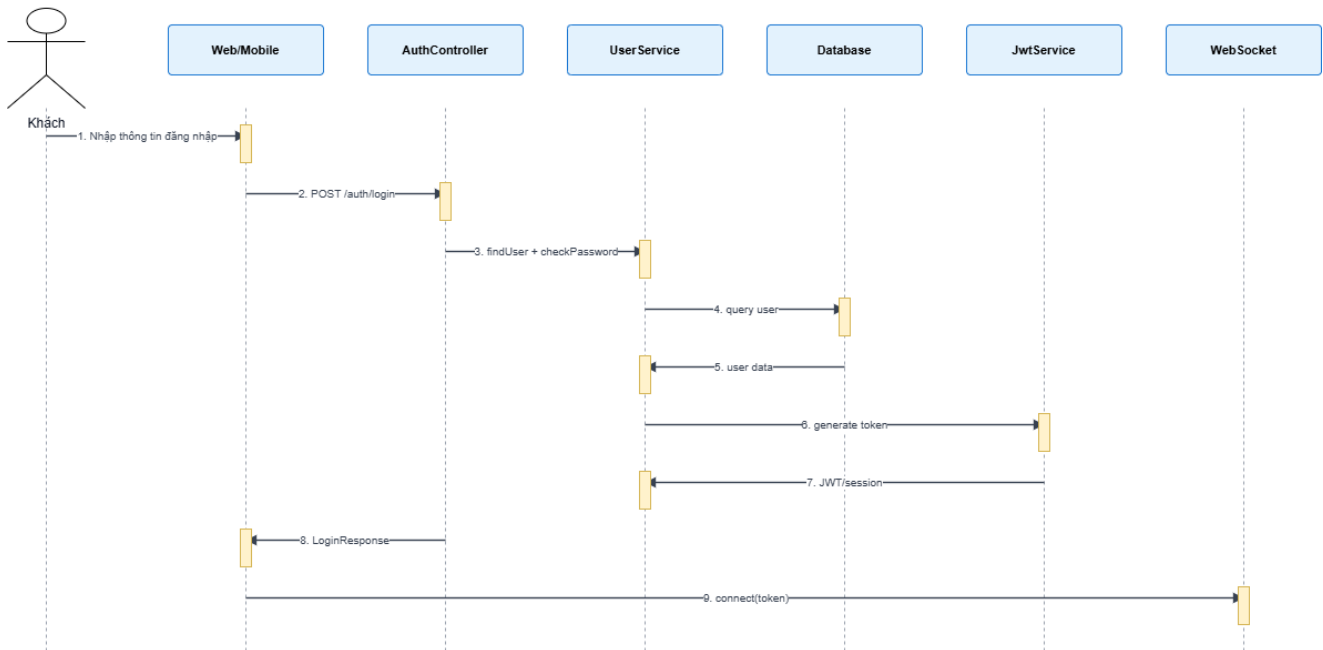
Ghi chú nghiệp vụ:

- Liên quan AuthController /login, JwtService.generateToken.

Activity Diagram - UC03: Đăng nhập hệ thống



Sequence Diagram - UC03: Đăng nhập hệ thống



3.6.04 UC04: Quên và đặt lại mật khẩu

Mã UC	UC04
Tên use case	Quên và đặt lại mật khẩu
Mô tả	Người dùng quên mật khẩu yêu cầu OTP và đặt mật khẩu mới.
Actor chính	Khách
Tiền điều kiện	Email tồn tại trong hệ thống.
Hậu điều kiện	Mật khẩu mới được lưu và người dùng có thể đăng nhập lại.

ST T	Người dùng/Client	Hệ thống
1	Mở trang quên mật khẩu.	Hiển thị form nhập email.
2	Nhập email và bấm gửi OTP.	AuthController/OtpService sinh OTP.
3	Nhập OTP và mật khẩu mới.	Backend kiểm tra OTP và validate password.
4	Bấm xác nhận.	Mã hóa mật khẩu mới và cập nhật User.
5	Nhận thông báo thành công.	Trả response và điều hướng về login.

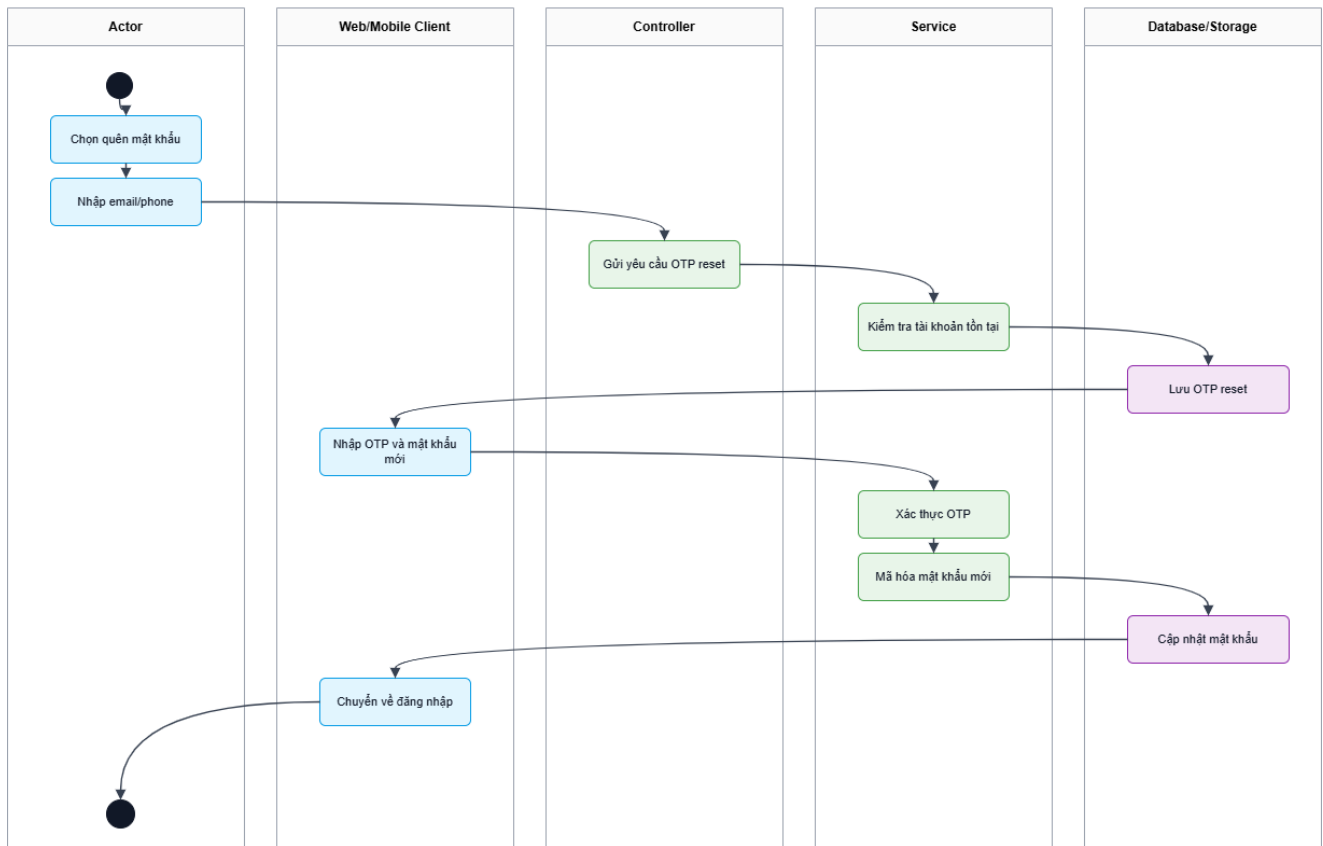
Mã	Luồng thay thế
A1	Người dùng chưa nhận email: gửi lại OTP.

Mã	Luồng ngoại lệ
E1	Email không tồn tại: báo lỗi.
E2	OTP sai/hết hạn: không cho đổi mật khẩu.

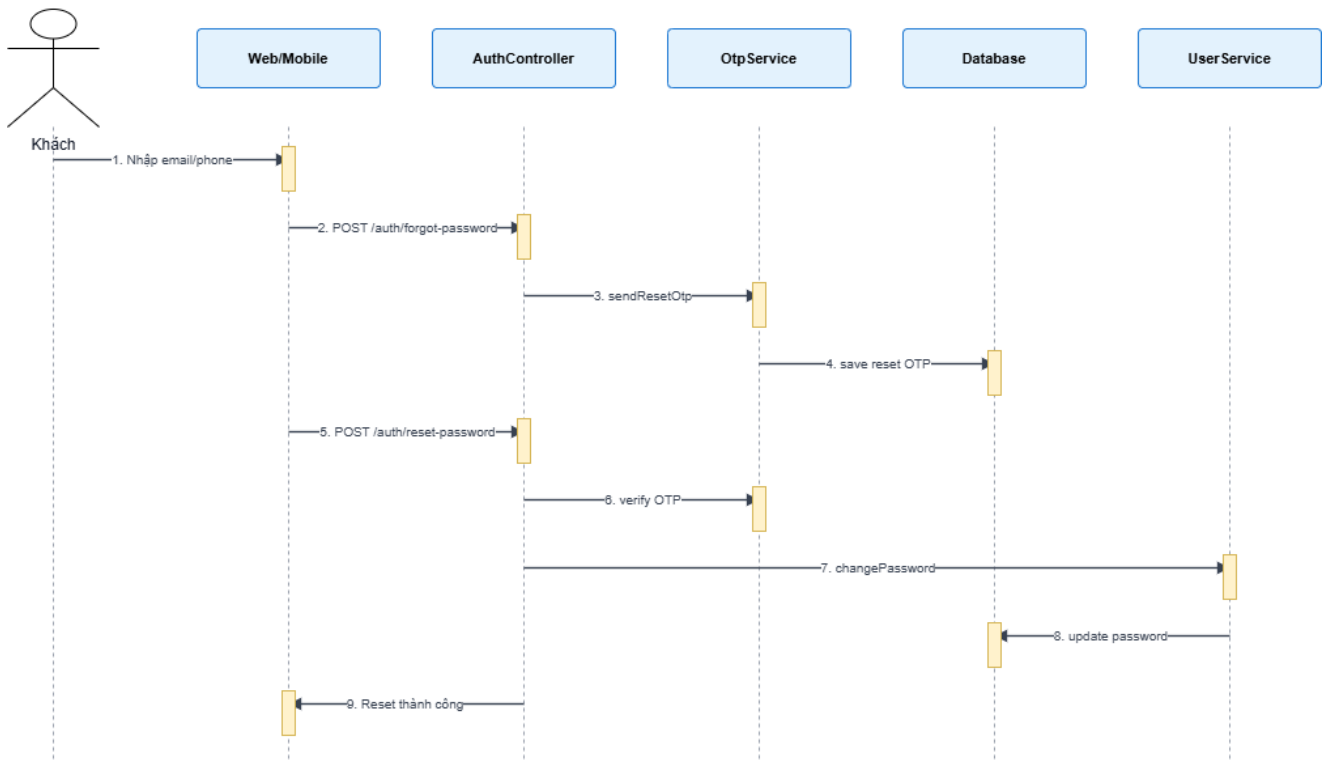
Ghi chú nghiệp vụ:

- Liên quan /forgot-password và /reset-password.

Activity Diagram - UC04: Quên và đặt lại mật khẩu



Sequence Diagram - UC04: Quên và đặt lại mật khẩu



3.6.05 UC05: Xem và cập nhật hồ sơ cá nhân

Mã UC	UC05
Tên use case	Xem và cập nhật hồ sơ cá nhân
Mô tả	Người dùng xem thông tin cá nhân và cập nhật thông tin hồ sơ.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập.
Hậu điều kiện	Thông tin mới được lưu và hiển thị lại trên web/mobile.

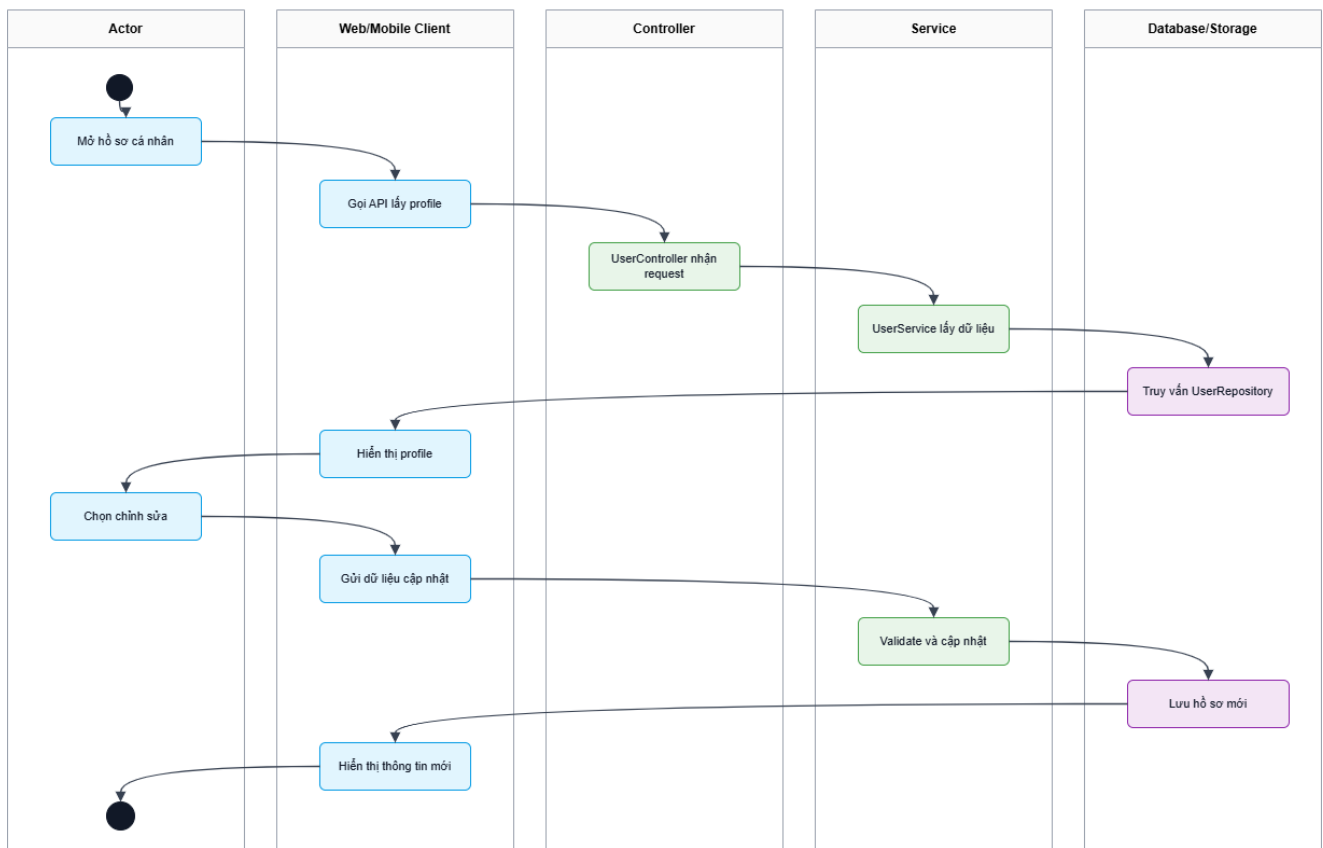
ST T	Người dùng/Client	Hệ thống
1	Mở profile.	UserController /me trả thông tin hiện tại.
2	Bấm chỉnh sửa hồ sơ.	Client mở modal/form chỉnh sửa.
3	Nhập thông tin mới.	Backend nhận UpdateUserRequest.
4	Bấm lưu.	UserService validate và cập nhật User.
5	Xem lại hồ sơ.	Trả UserResponse mới.

Mã	Luồng thay thế
A1	Người dùng chỉ cập nhật một số trường, hệ thống giữ nguyên trường còn lại.

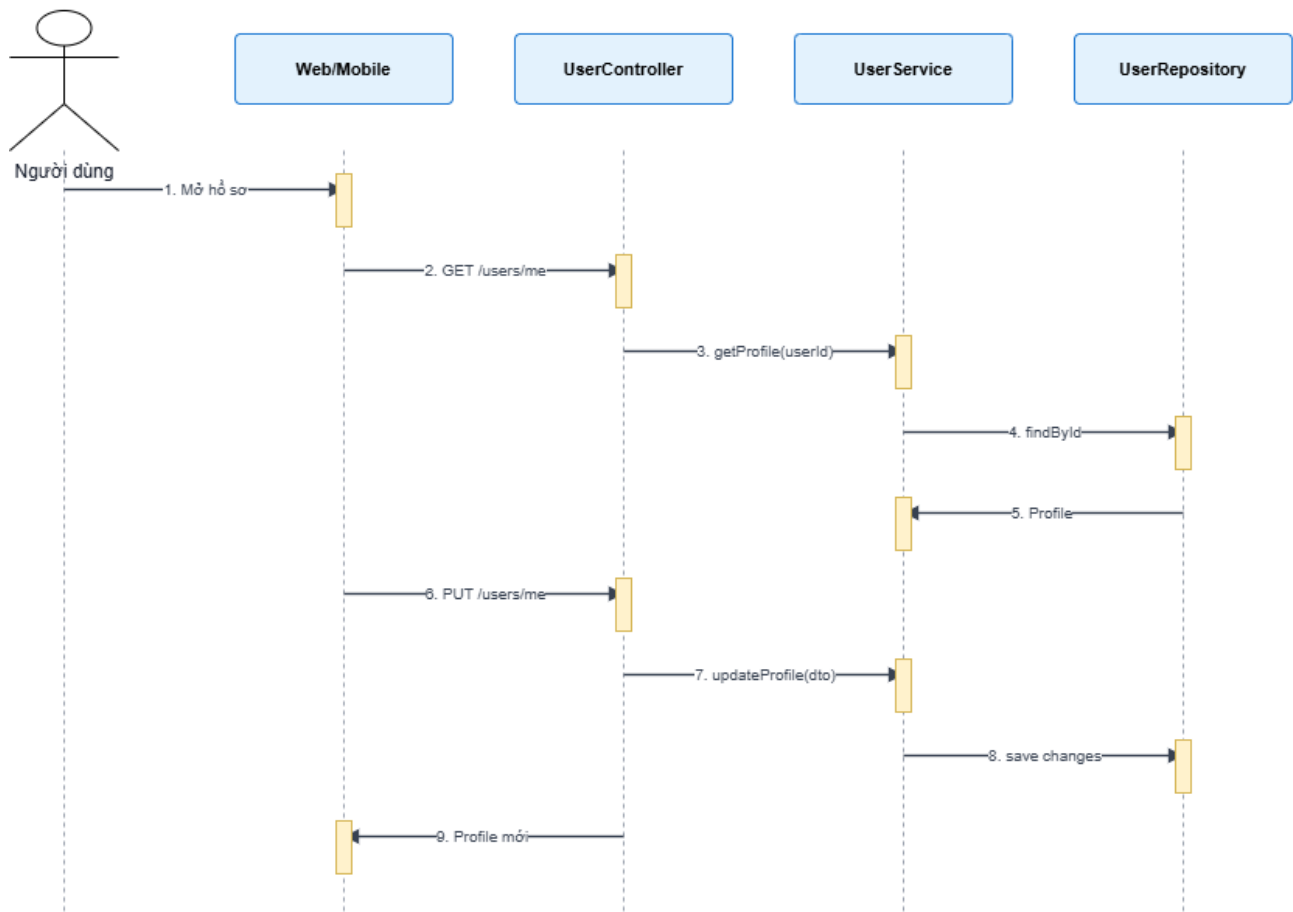
Mã	Luồng ngoại lệ
E1	Token hết hạn: yêu cầu đăng nhập lại.
E2	Dữ liệu không hợp lệ: trả thông báo lỗi.

Ghi chú nghiệp vụ:

Activity Diagram - UC05: Xem và cập nhật hồ sơ cá nhân



Sequence Diagram - UC05: Xem và cập nhật hồ sơ cá nhân



3.6.06 UC06: Upload avatar/cover/file

Mã UC	UC06
Tên use case	Upload avatar/cover/file
Mô tả	Người dùng upload avatar, ảnh bìa hoặc file đính kèm trong chat.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; file đúng định dạng và dung lượng cho phép.
Hậu điều kiện	File được lưu, đường dẫn fileUrl trả về cho client.

ST T	Người dùng/Client	Hệ thống
1	Chọn file từ thiết bị.	Client tạo multipart/form-data.
2	Bấm upload/gửi file.	FileController hoặc UserController nhận MultipartFile.
3		S3Service/local storage lưu file.
4		Backend tạo URL hoặc path truy cập file.
5	Client hiển thị ảnh/file.	Trả fileUrl về client.

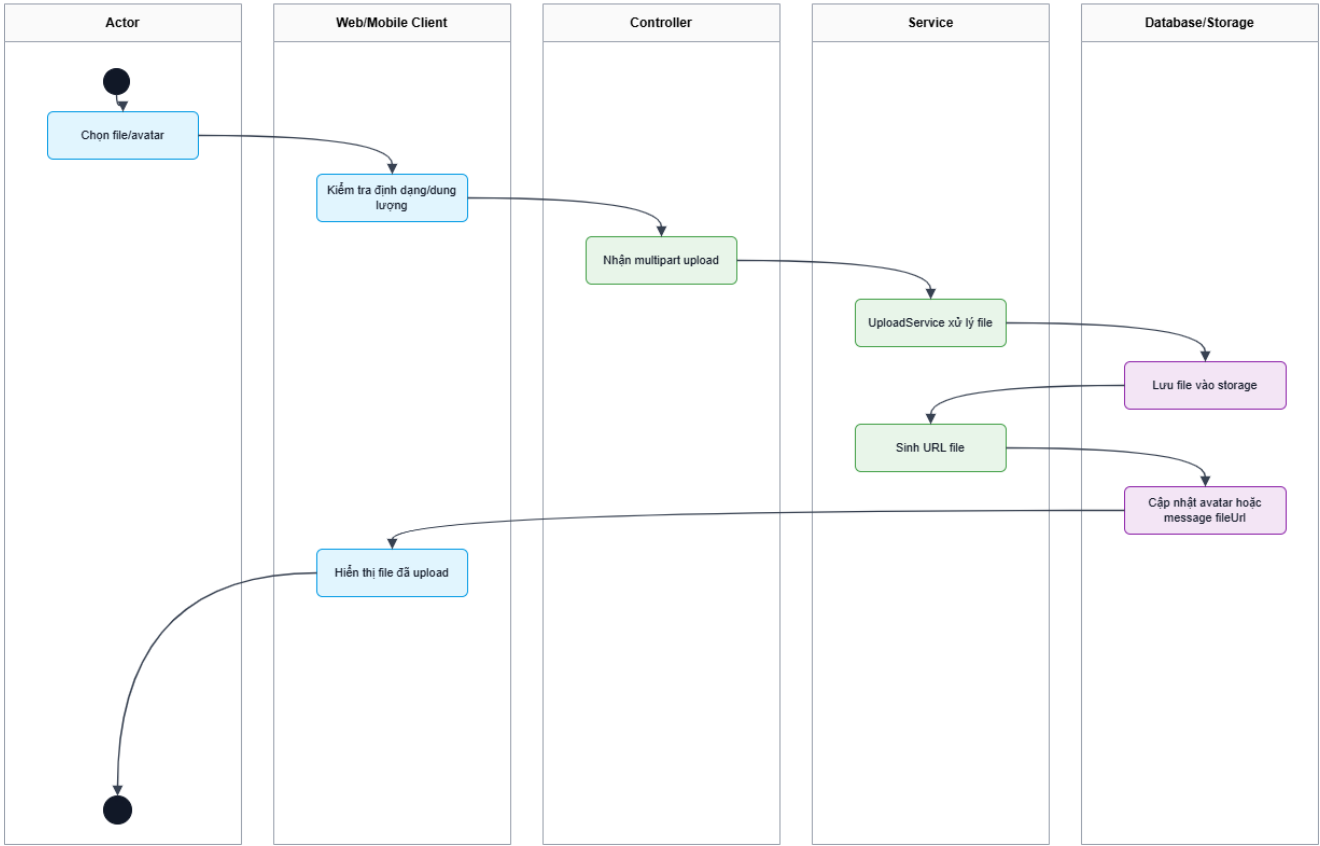
Mã	Luồng thay thế
A1	Nếu upload avatar/cover, backend cập nhật trường avatar/coverImage của User.
A2	Nếu upload trong chat, fileUrl được gắn vào Message.

Mã	Luồng ngoại lệ
E1	File rỗng hoặc sai định dạng: trả lỗi.
E2	Upload lỗi: thông báo thất bại và cho thử lại.

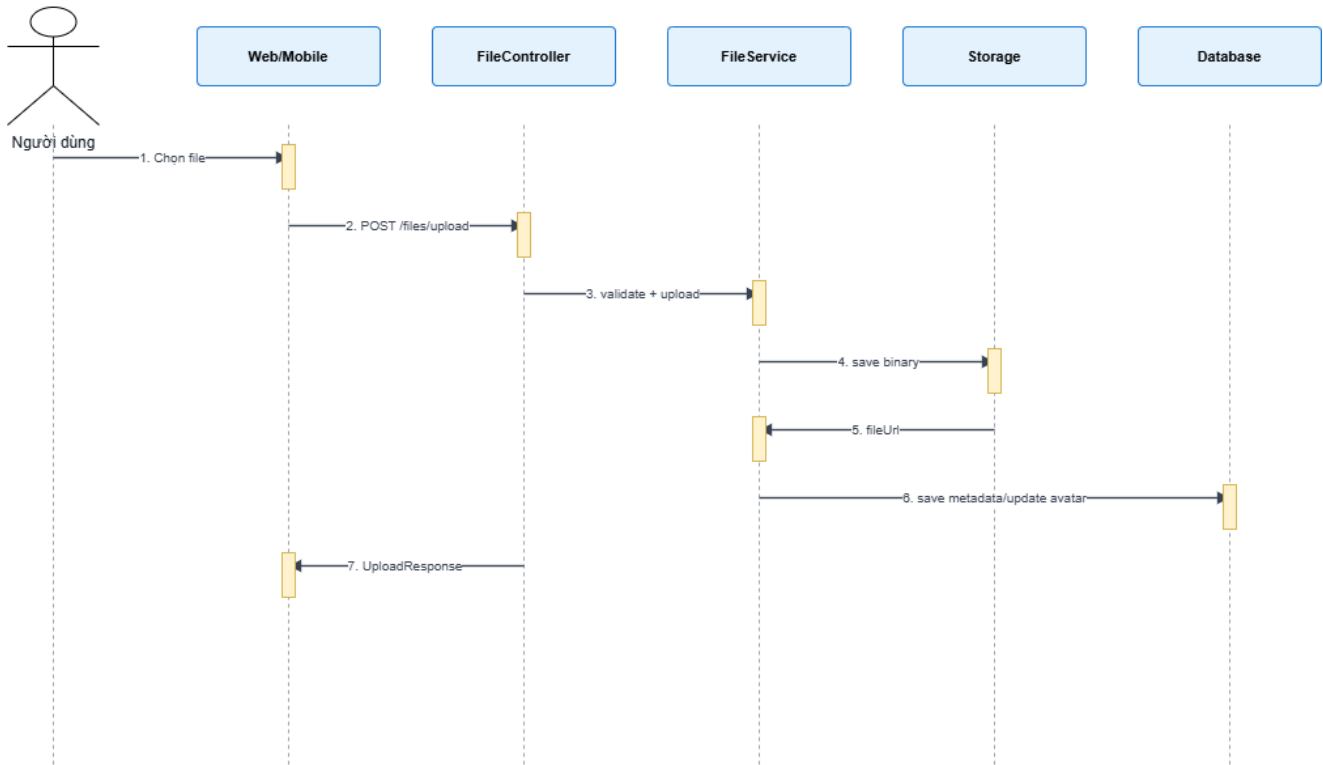
Ghi chú nghiệp vụ:

- Liên quan FileController /api/file/upload, UserController /upload-avatar, /upload-cover.

Activity Diagram - UC06: Upload avatar/cover/file



Sequence Diagram - UC06: Upload avatar/cover/file



3.6.07 UC07: Tìm kiếm và kết bạn

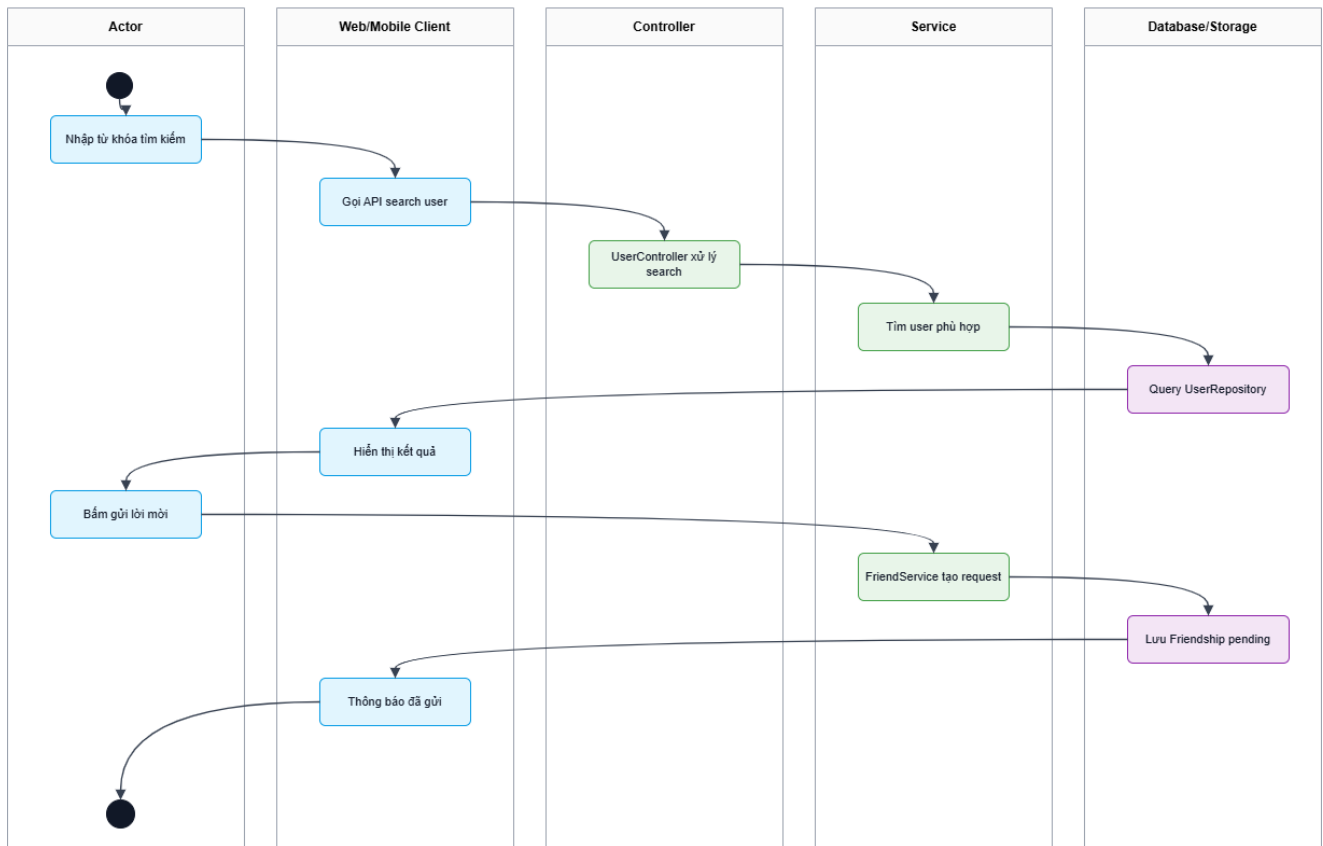
Mã UC	UC07
Tên use case	Tìm kiếm và kết bạn
Mô tả	Người dùng tìm người khác và gửi lời mời kết bạn.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; người cần tìm tồn tại.
Hậu điều kiện	Lời mời kết bạn được tạo với trạng thái chờ xử lý.

ST T	Người dùng/Client	Hệ thống
1	Nhập từ khóa tìm kiếm.	FriendshipController /search nhận keyword.
2	Xem danh sách kết quả.	FriendshipService lọc user phù hợp.
3	Bấm gửi lời mời.	Backend nhận FriendRequestDTO.
4		Kiểm tra chưa là bạn và chưa có request trùng.
5		Lưu Friendship status PENDING.
6	Nhận thông báo đã gửi lời mời.	Trả response thành công.

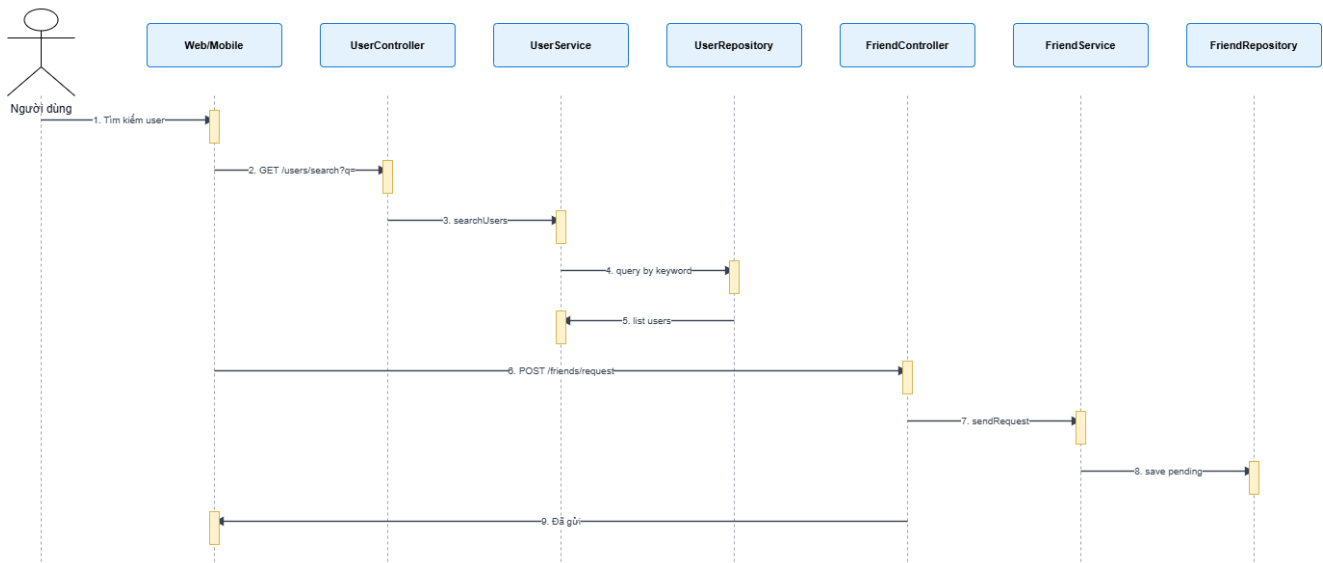
Mã	Luồng thay thế
A1	Nếu đã có lời mời pending, hệ thống không tạo thêm mà báo đã gửi.

Mã	Luồng ngoại lệ
E1	Tự gửi lời mời cho chính mình: báo lỗi.
E2	Người nhận đã chặn người gửi: không cho gửi request.

Activity Diagram - UC07: Tìm kiếm và kết bạn



Sequence Diagram - UC07: Tìm kiếm và kết bạn



3.6.08 UC08: Chấp nhận/từ chối lời mời kết bạn

Mã UC	UC08
Tên use case	Chấp nhận/từ chối lời mời kết bạn
Mô tả	Người nhận lời mời quyết định chấp nhận hoặc từ chối.

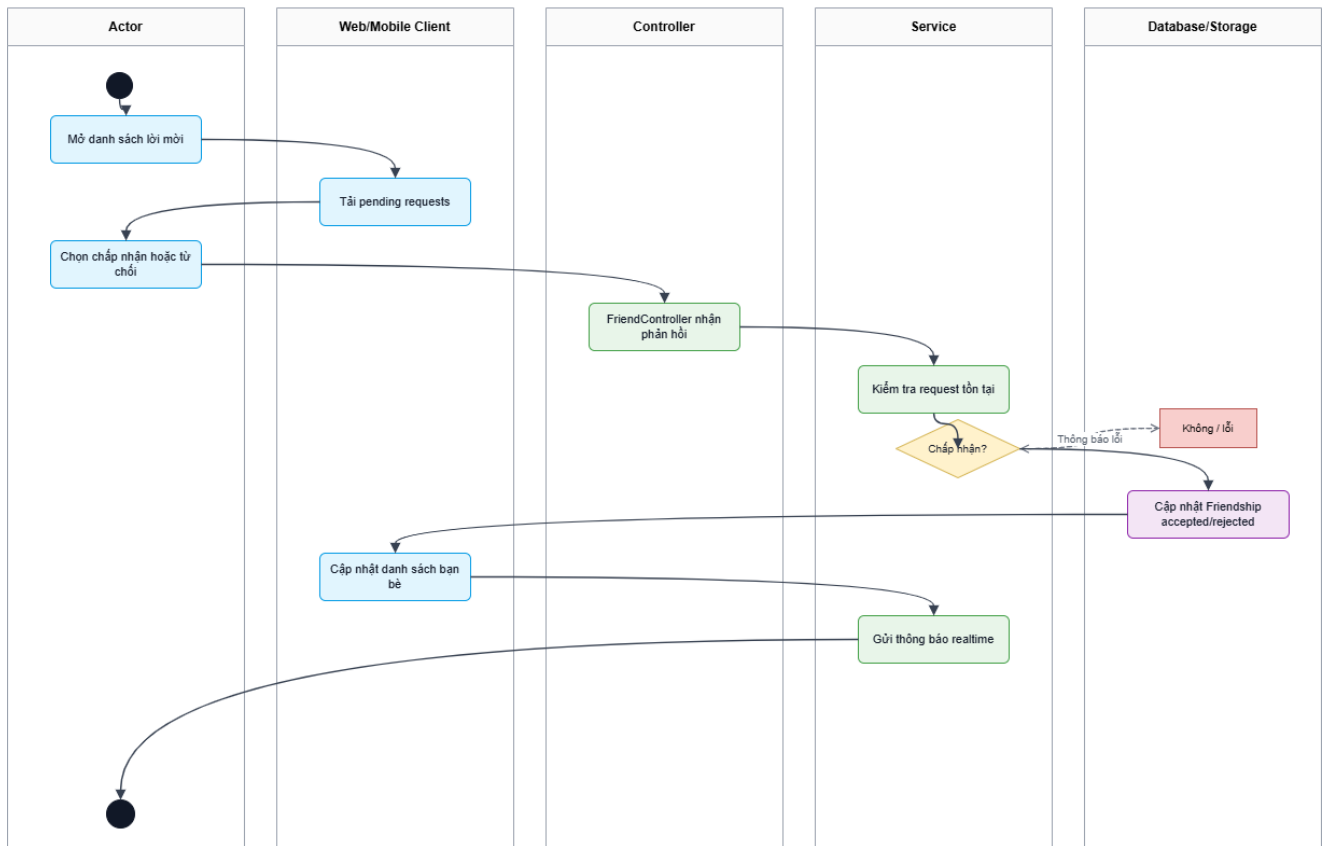
Actor chính	Người dùng
Tiền điều kiện	Có lời mời kết bạn pending gửi đến người dùng.
Hậu điều kiện	Friendship chuyển sang ACCEPTED hoặc bị REJECTED.

ST T	Người dùng/Client	Hệ thống
1	Mở danh sách lời mời.	FriendshipController /requests trả danh sách pending.
2	Chọn accept hoặc reject.	Backend nhận id lời mời.
3		Kiểm tra quyền của người nhận.
4		Cập nhật trạng thái lời mời.
5	Danh sách bạn bè cập nhật.	Trả response mới.

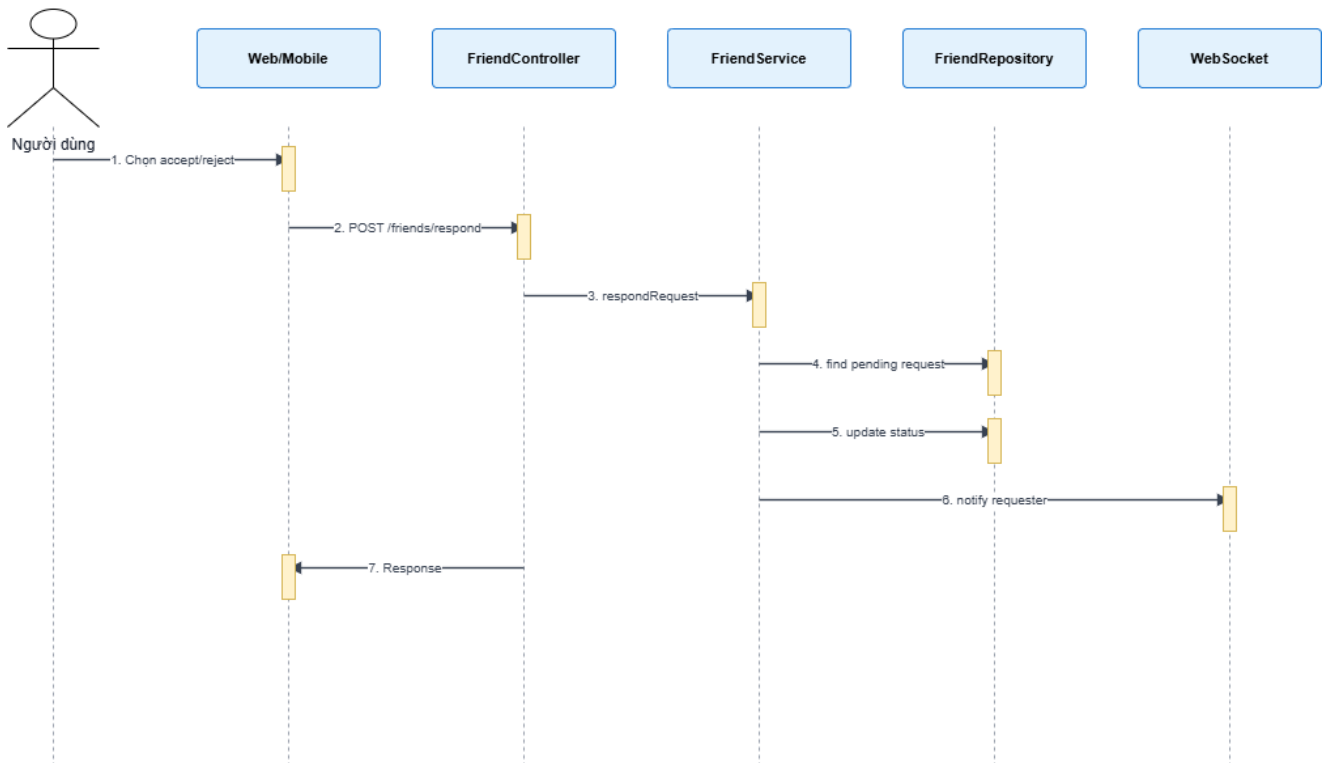
Mã	Luồng thay thế
A1	Nếu accept thành công, hai bên xuất hiện trong danh sách bạn bè.

Mã	Luồng ngoại lệ
E1	Request không tồn tại: trả lỗi.
E2	Người dùng không phải receiver: từ chối thao tác.

Activity Diagram - UC08: Chấp nhận/từ chối lời mời kết bạn



Sequence Diagram - UC08: Chấp nhận/từ chối lời mời kết bạn



3.6.09 UC09: Chặn và bỏ chặn người dùng

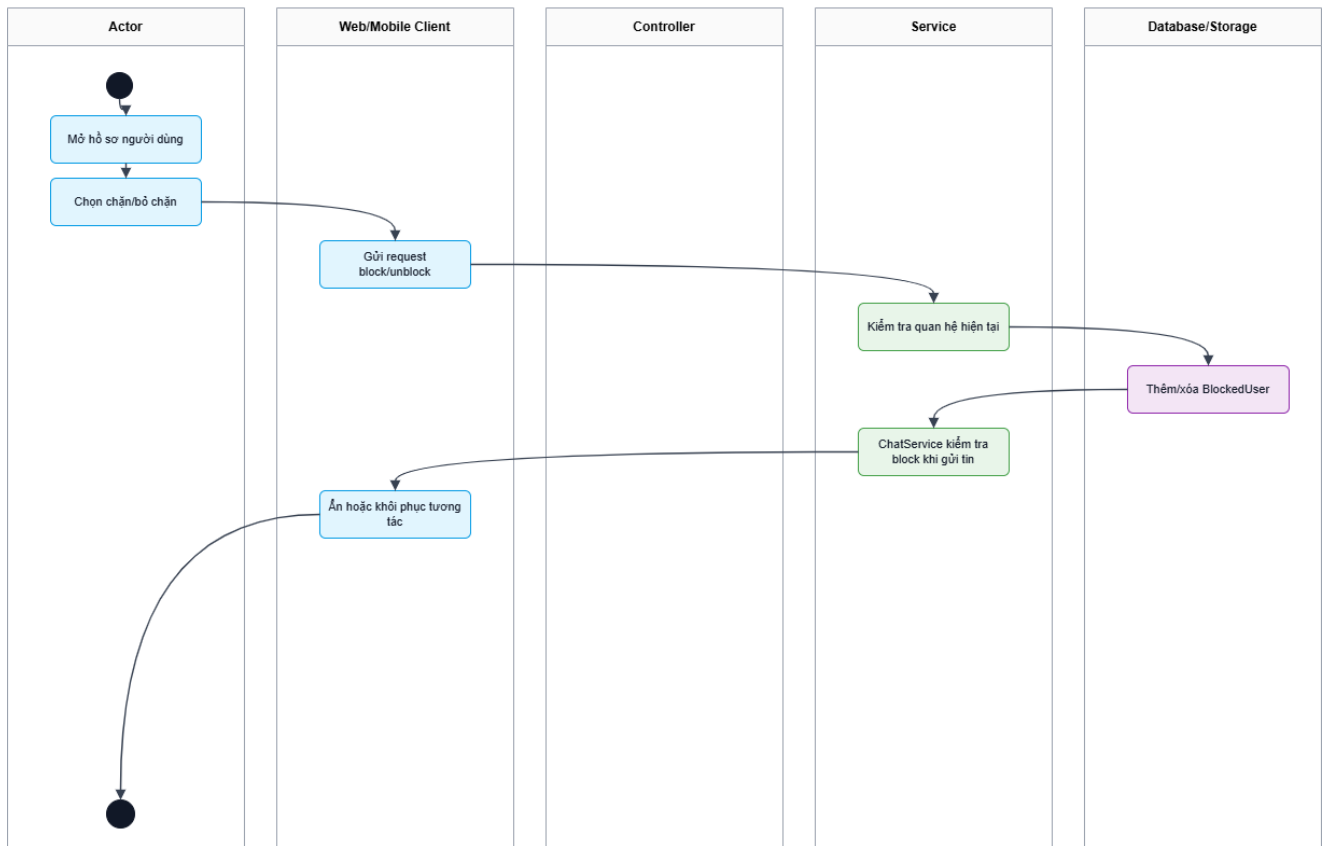
Mã UC	UC09
Tên use case	Chặn và bỏ chặn người dùng
Mô tả	Người dùng chặn hoặc bỏ chặn một user khác.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; target user tồn tại.
Hậu điều kiện	Trạng thái block được cập nhật, ảnh hưởng đến việc gửi tin nhắn.

ST T	Người dùng/Client	Hệ thống
1	Mở trang/profile người dùng hoặc cuộc trò chuyện.	Client hiển thị tùy chọn block.
2	Bấm Chặn.	BlockController nhận targetId.
3		BlockedService lưu quan hệ block.
4		Hệ thống kiểm tra trạng thái chặn khi gửi tin.
5	Bấm Bỏ chặn nếu cần.	Backend xóa bản ghi block.

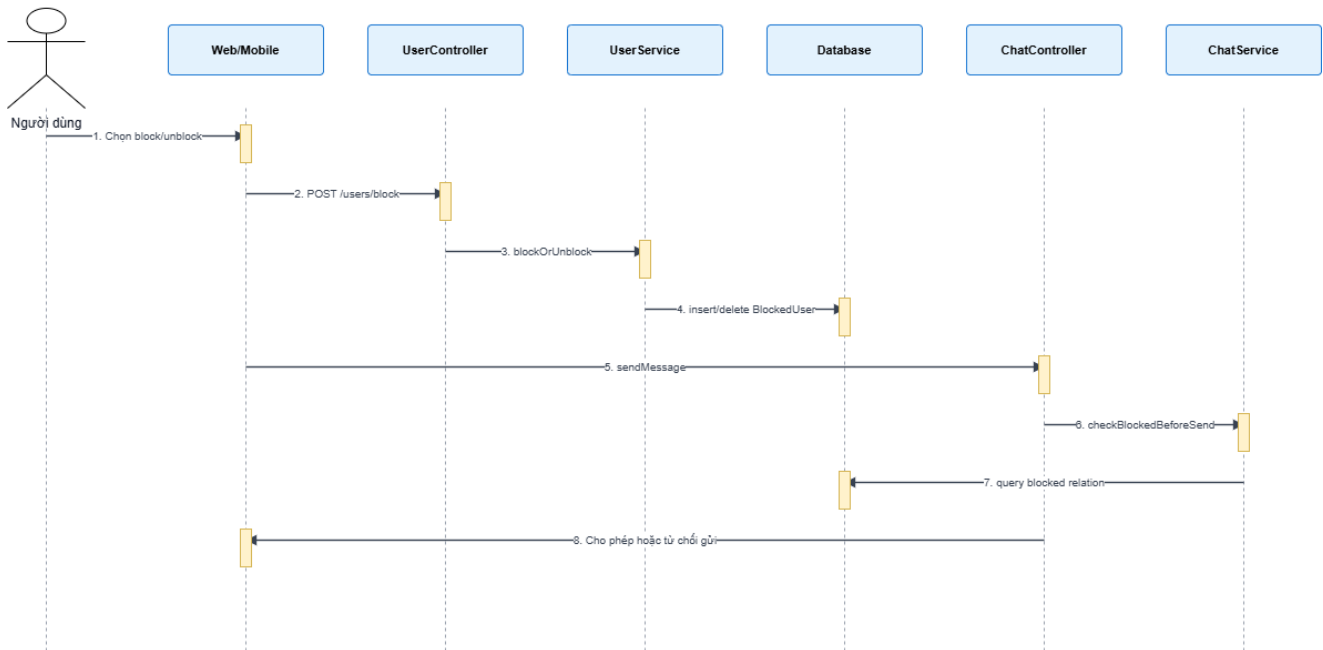
Mã	Luồng thay thế
A1	Nếu đã block trước đó, thao tác block lại không tạo bản ghi trùng.

Mã	Luồng ngoại lệ
E1	Chặn chính mình: báo lỗi.
E2	Target user không tồn tại: trả lỗi.

Activity Diagram - UC09: Chặn và bỏ chặn người dùng



Sequence Diagram - UC09: Chặn và bỏ chặn người dùng



3.6.10 UC10: Nhắn tin cá nhân realtime

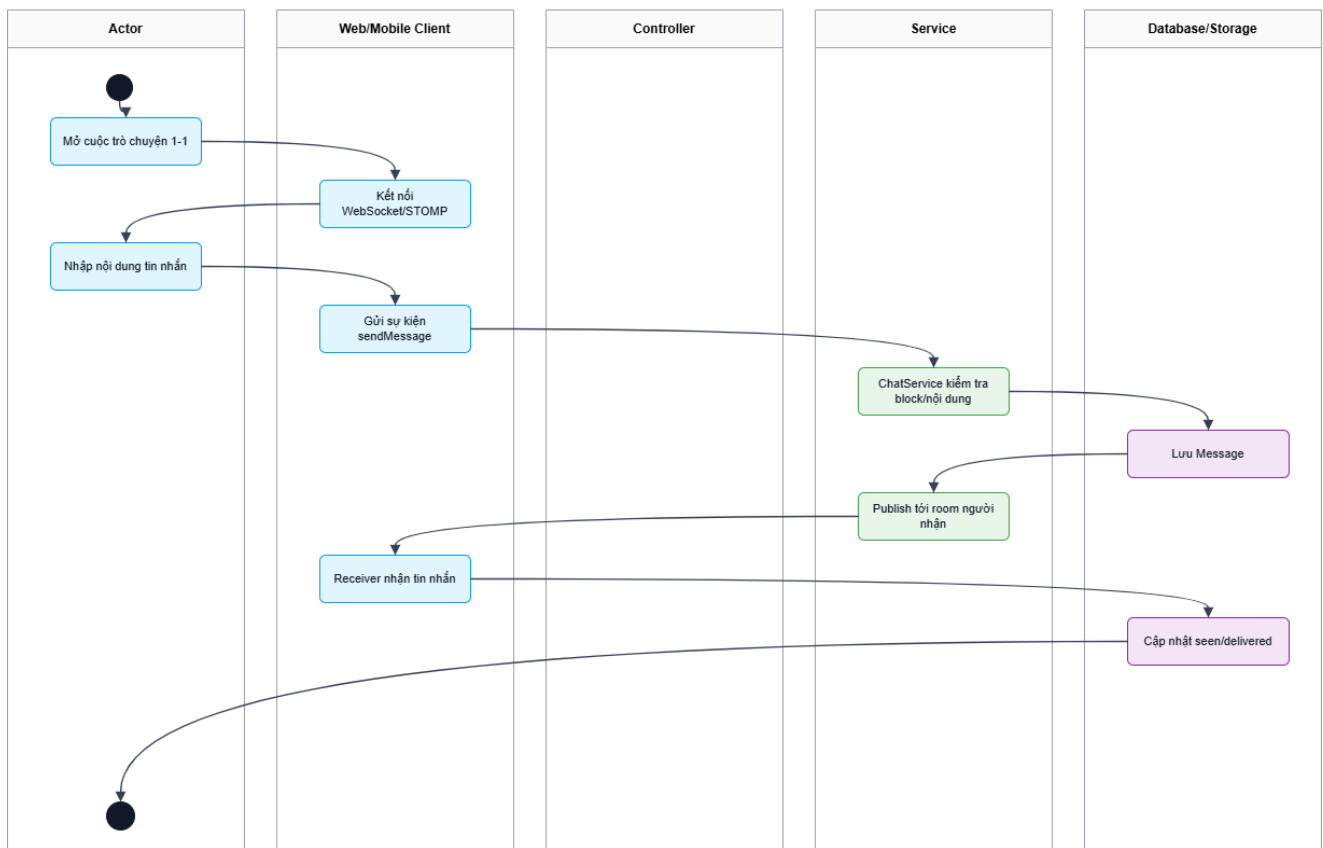
Mã UC	UC10
Tên use case	Nhắn tin cá nhân realtime
Mô tả	Hai người dùng gửi tin nhắn cá nhân qua WebSocket/STOMP.
Actor chính	Người dùng
Tiền điều kiện	Hai người dùng đăng nhập, không bị chặn nhau.
Hậu điều kiện	Tin nhắn được lưu và phát realtime đến room.

ST T	Người dùng/Client	Hệ thống
1	Mở cuộc trò chuyện với bạn bè.	Client xác định roomId.
2	Nhập nội dung tin nhắn.	Client gửi message đến /app/chat.send.
3		ChatController nhận SendMessageDTO.
4		ChatService kiểm tra block, trạng thái hội thoại và lưu tin nhắn.
5		MessageRepository lưu message.
6	Người nhận đang online nhận tin.	Server publish message đến topic/room.
7	Lịch sử tin nhắn cập nhật.	Client render message mới.

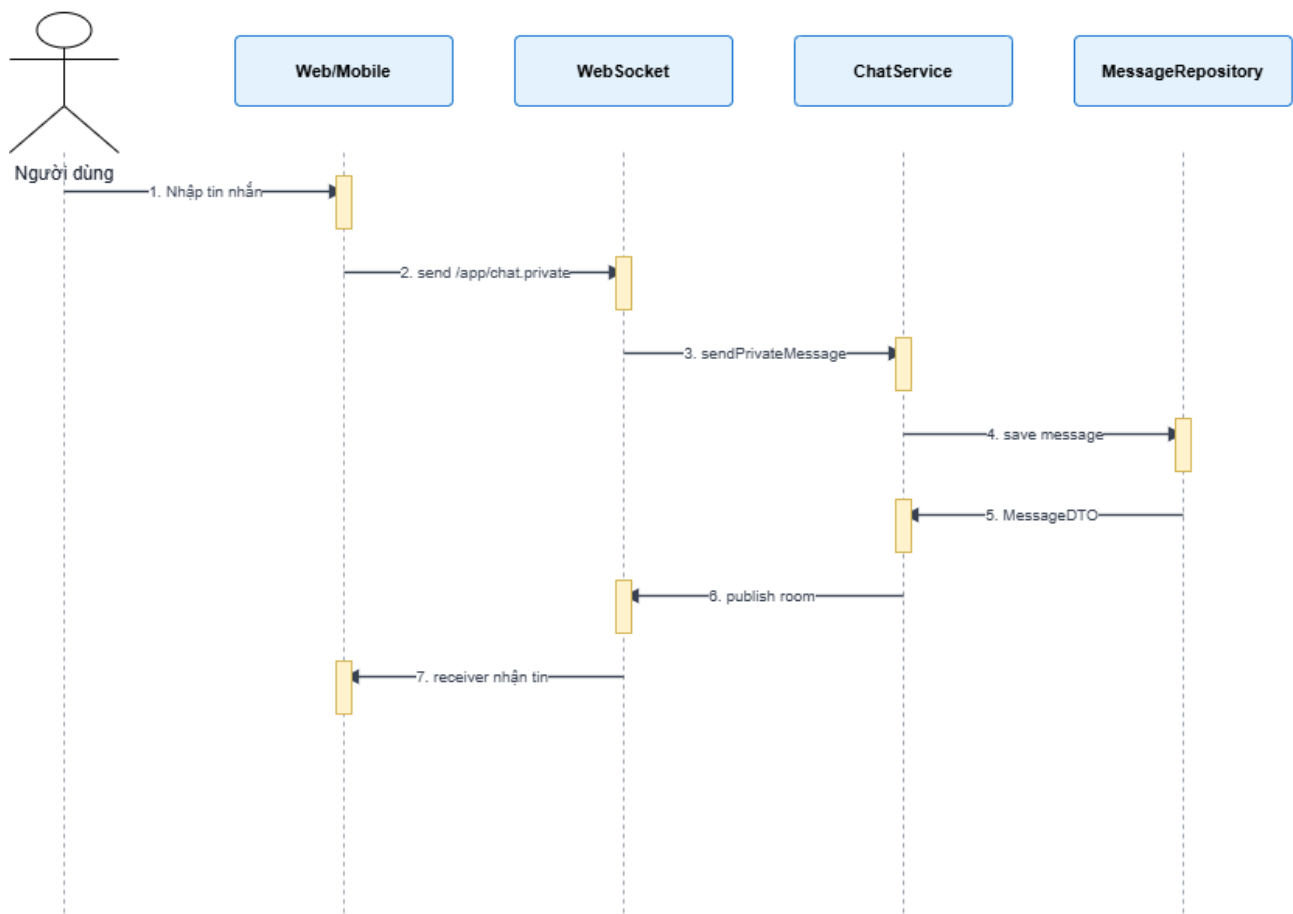
Mã	Luồng thay thế
A1	Nếu người nhận offline, tin nhắn vẫn lưu và hiển thị khi người nhận mở lại.
A2	Nếu tin nhắn có fileUrl, client hiển thị theo type file/image.

Mã	Luồng ngoại lệ
E1	Hai người dùng bị block: không cho gửi.
E2	Nếu nội dung rỗng hoặc dữ liệu không hợp lệ: hệ thống từ chối gửi và yêu cầu nhập lại.

Activity Diagram - UC10: Nhắn tin cá nhân realtime



Sequence Diagram - UC10: Nhắn tin cá nhân realtime



3.6.11 UC11: Thu hồi, xóa và reaction tin nhắn

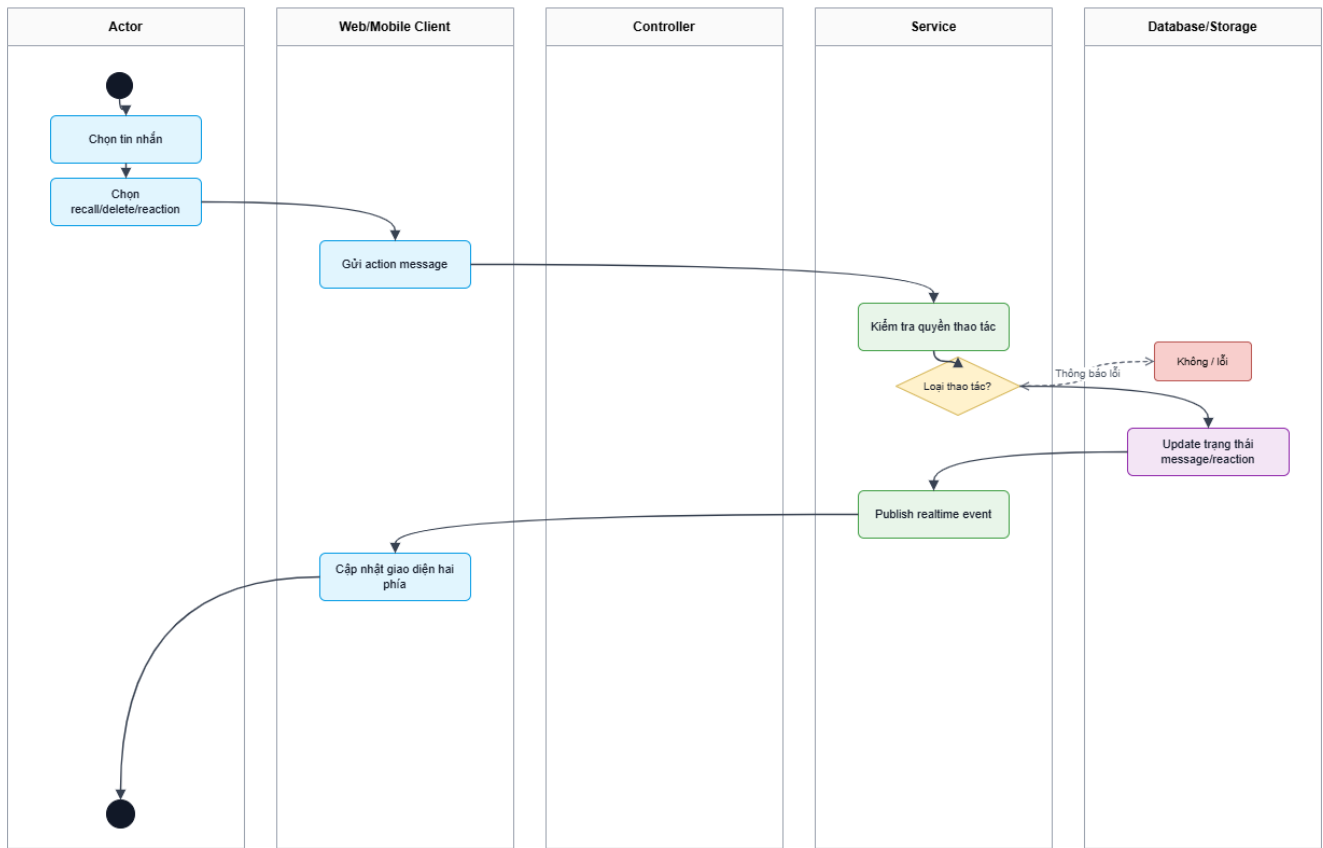
Mã UC	UC11
Tên use case	Thu hồi, xóa và reaction tin nhắn
Mô tả	Người dùng quản lý tin nhắn đã gửi hoặc tin nhắn trong cuộc trò chuyện.
Actor chính	Người dùng
Tiền điều kiện	Tin nhắn tồn tại và user có quyền thao tác.
Hậu điều kiện	Tin nhắn được cập nhật trạng thái recall/delete/reaction.

ST T	Người dùng/Client	Hệ thống
1	Chọn một tin nhắn.	Client hiển thị menu thao tác.
2	Bấm thu hồi/xóa/reaction.	ChatController nhận request.
3		ChatService kiểm tra quyền user.
4		Cập nhật isRecalled, isDeleted hoặc reactions.
5	Giao diện cập nhật.	Trả Message mới hoặc trạng thái thành công.

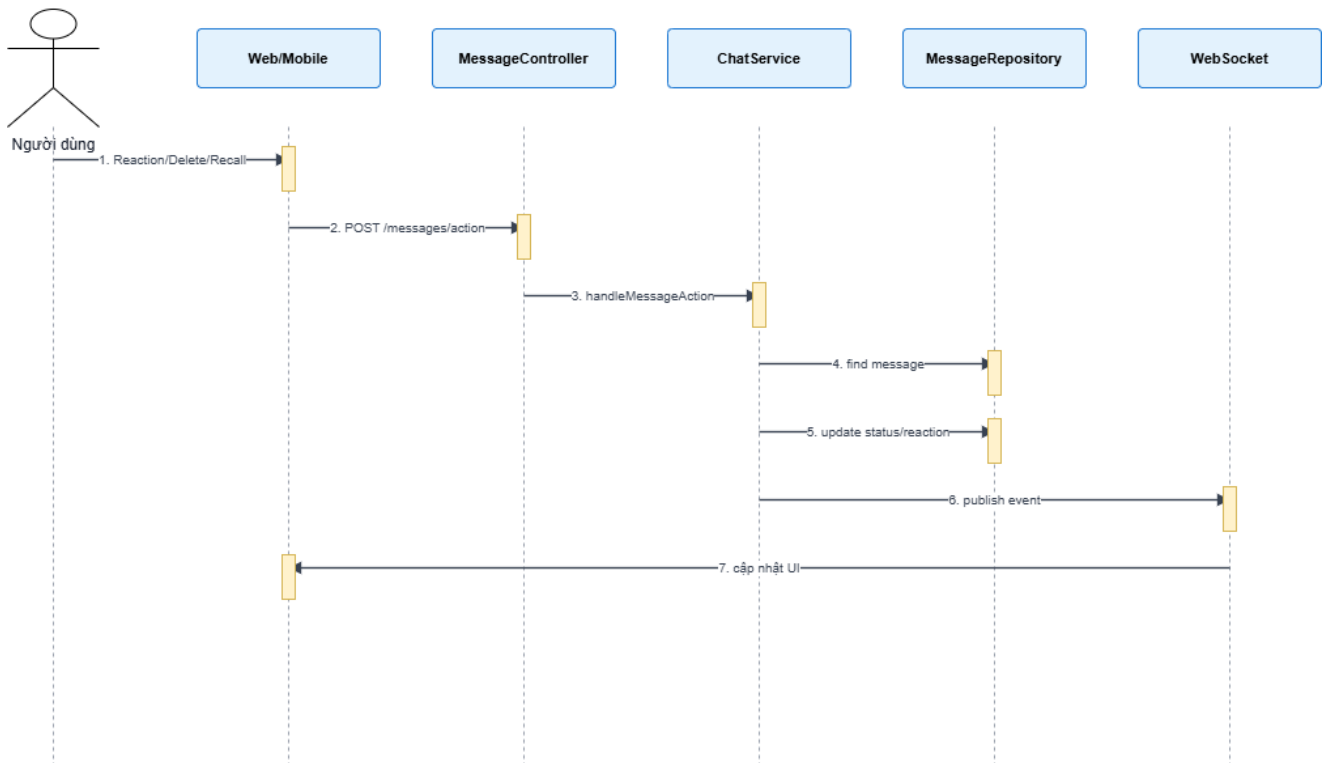
Mã	Luồng thay thế
A1	Xóa tin nhắn chỉ áp dụng cho user hiện tại nếu là delete for me.
A2	Reaction có thể thêm hoặc bỏ emoji nếu user bấm lại.

Mã	Luồng ngoại lệ
E1	Tin nhắn không tồn tại: trả lỗi.
E2	User không phải người gửi nhưng cố recall: từ chối.

Activity Diagram - UC11: Thu hồi, xóa và reaction tin nhắn



Sequence Diagram - UC11: Thu hồi, xóa và reaction tin nhắn



3.6.12 UC12: Tạo nhóm chat

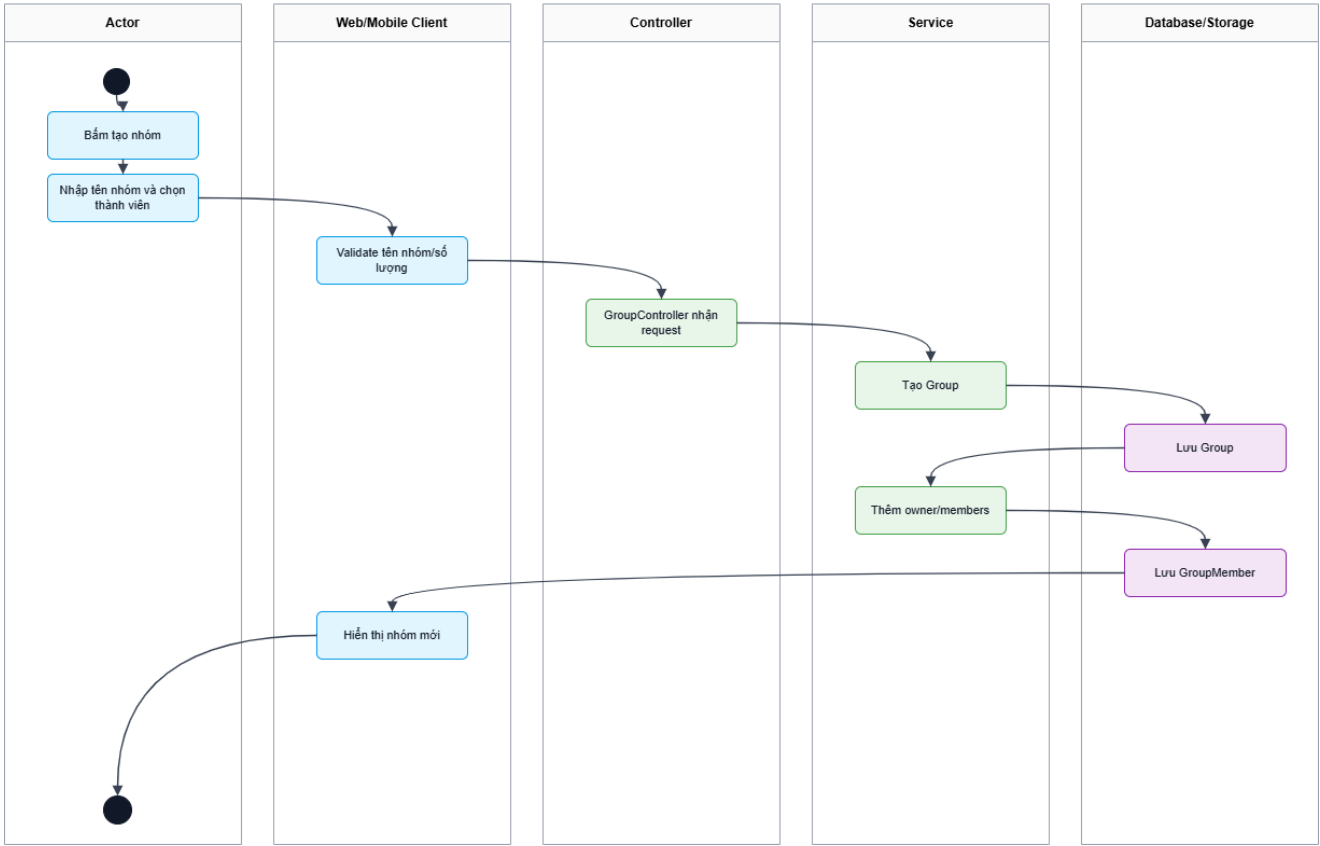
Mã UC	UC12
Tên use case	Tạo nhóm chat
Mô tả	Người dùng tạo nhóm chat và thêm thành viên ban đầu.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; danh sách thành viên hợp lệ.
Hậu điều kiện	Group và GroupMember được tạo, người tạo là OWNER/ADMIN.

ST T	Người dùng/Client	Hệ thống
1	Mở form tạo nhóm.	Client hiển thị danh sách bạn bè.
2	Nhập tên nhóm và chọn thành viên.	GroupController nhận CreateGroupDTO.
3	Bấm tạo nhóm.	GroupService tạo Group.
4		Tạo GroupMember cho creator và các thành viên.
5	Nhóm mới xuất hiện.	Trả thông tin group.

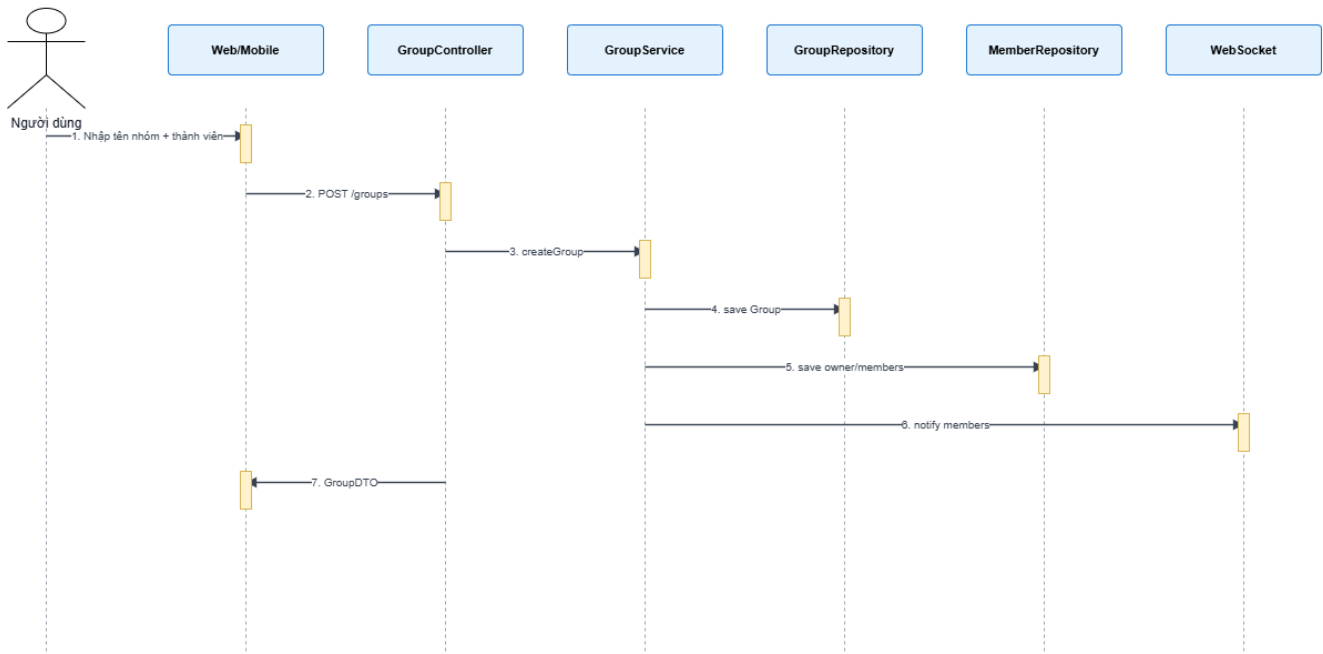
Mã	Luồng thay thế
A1	Nhóm có thể tạo chỉ với tên và người tạo, thêm thành viên sau.

Mã	Luồng ngoại lệ
E1	Tên nhóm rỗng: báo lỗi.
E2	Thành viên không tồn tại: không tạo nhóm.

Activity Diagram - UC12: Tạo nhóm chat



Sequence Diagram - UC12: Tạo nhóm chat



3.6.13 UC13: Quản lý thành viên nhóm

Đặc tả Use case

Mã UC	UC13
Tên use case	Quản lý thành viên nhóm
Mô tả	Quản trị nhóm quản lý danh sách thành viên trong nhóm chat, bao gồm thêm thành viên, xóa thành viên, thay đổi vai trò và kiểm soát quyền tham gia nhóm.
Actor chính	Quản trị nhóm (Owner/Admin)
Tiền điều kiện	Người dùng đã đăng nhập; nhóm chat tồn tại; actor có quyền Owner hoặc Admin trong nhóm.
Hậu điều kiện	Danh sách thành viên hoặc vai trò trong nhóm được cập nhật; các thành viên liên quan nhận thông báo realtime.

ST T	Người dùng/Client	Hệ thống
1	Mở màn hình thông tin nhóm hoặc quản lý thành viên.	Hệ thống tải thông tin nhóm, danh sách thành viên và vai trò hiện tại.
2	Chọn chức năng thêm thành viên, xóa thành viên hoặc đổi vai trò.	GroupController nhận yêu cầu và kiểm tra quyền của người thao tác.
3	Tìm kiếm/chọn người dùng cần thêm hoặc chọn thành viên cần xử lý.	GroupService kiểm tra người dùng có tồn tại, chưa bị chặn và đáp ứng điều kiện tham gia nhóm.
4	Xác nhận thao tác.	Hệ thống cập nhật bảng GroupMember/MemberRole trong cơ sở dữ liệu.
5	Xem kết quả cập nhật trên giao diện nhóm.	Hệ thống phát sự kiện realtime qua WebSocket để cập nhật danh sách thành viên cho các client đang mở nhóm.

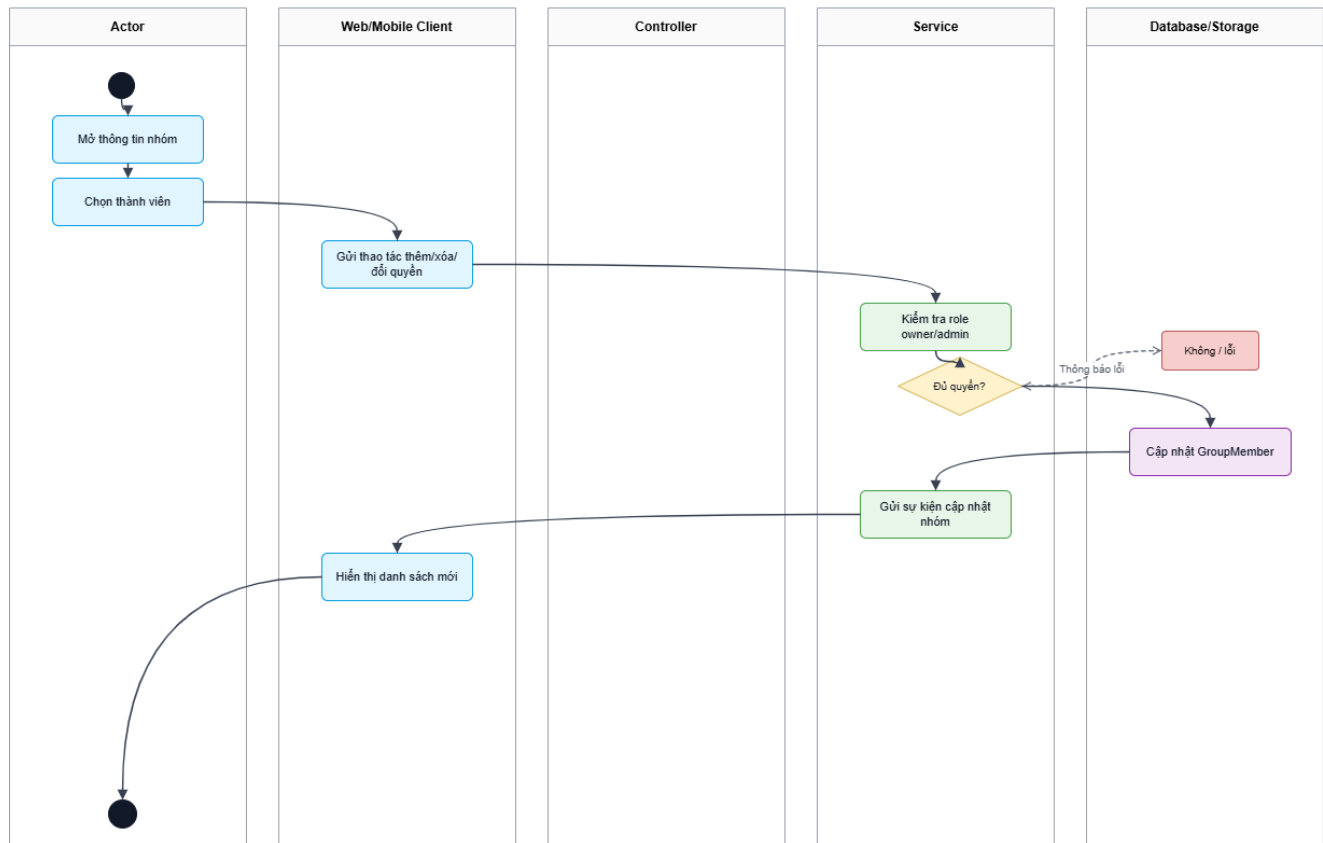
Mã	Luồng thay thế
A1	Owner có thể chuyển quyền sở hữu nhóm cho một thành viên khác. Sau khi chuyển quyền, owner cũ trở thành admin hoặc member tùy cấu hình.
A2	Admin chỉ có quyền quản lý member thông thường, không được xóa owner hoặc hạ quyền owner.
A3	Thành viên có thể tự rời nhóm. Khi rời nhóm, hệ thống cập nhật lại số lượng thành viên và gửi thông báo trong nhóm.
A4	Nếu nhóm bật chế độ duyệt thành viên, yêu cầu thêm thành viên có thể chuyển sang trạng thái chờ duyệt thay vì thêm ngay.

Mã	Luồng ngoại lệ
E1	Người thao tác không có quyền quản trị nhóm: hệ thống từ chối và hiển thị thông báo.
E2	Người dùng được thêm không tồn tại hoặc đã bị chặn: không cập nhật nhóm.
E3	Thành viên đã có trong nhóm: hệ thống báo trùng thành viên.
E4	Không thể xóa hoặc hạ quyền owner khi chưa chuyển quyền sở hữu.
E5	Lỗi kết nối hoặc lỗi cơ sở dữ liệu: hệ thống rollback thao tác và hiển thị lỗi.

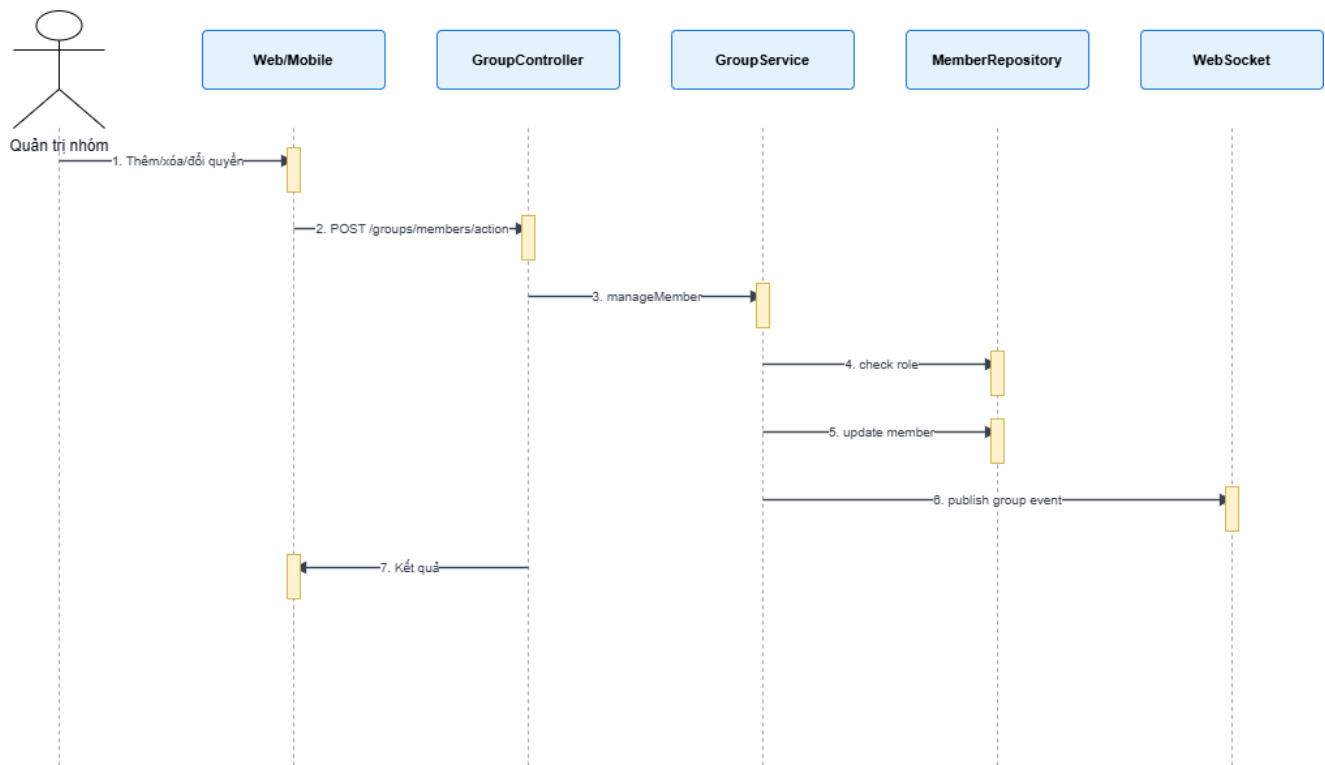
Ghi chú nghiệp vụ:

Use case này cần làm rõ quyền của Owner/Admin để tránh người quản trị cấp thấp xóa hoặc thay đổi quyền của người có vai trò cao hơn. Khi triển khai giao diện, nên có hộp xác nhận trước các thao tác xóa thành viên, đổi quyền hoặc chuyển quyền sở hữu

Activity Diagram - UC13: Quản lý thành viên nhóm



Sequence Diagram - UC13: Quản lý thành viên nhóm



3.6.14 UC14: Nhắn tin nhóm

Đặc tả Use case

Mã UC	UC14
Tên use case	Nhắn tin nhóm
Mô tả	Thành viên trong nhóm gửi và nhận tin nhắn realtime trong phòng chat nhóm. Tin nhắn có thể là văn bản, hình ảnh, file đính kèm hoặc phản hồi tin nhắn trước đó.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; là thành viên của nhóm; kết nối mạng và WebSocket hoạt động.
Hậu điều kiện	Tin nhắn được lưu vào cơ sở dữ liệu, hiển thị trong nhóm và gửi realtime đến các thành viên liên quan.

ST T	Người dùng/Client	Hệ thống
1	Mở cuộc trò chuyện nhóm.	Hệ thống tải thông tin nhóm, danh sách thành viên và lịch sử tin nhắn gần nhất.
2	Nhập nội dung tin nhắn hoặc chọn file/hình ảnh cần gửi.	Client kiểm tra dữ liệu đầu vào, dung lượng file và định dạng cơ bản.

3	Bấm gửi tin nhắn.	ChatController nhận request, kiểm tra token và quyền thành viên trong nhóm.
4	Chờ tin nhắn xuất hiện trên khung chat.	ChatService lưu tin nhắn vào MessageRepository/MongoDB và tạo metadata cho file nếu có.
5	Các thành viên khác nhận tin nhắn realtime.	WebSocketGateway phát sự kiện message:new đến room của nhóm, đồng thời cập nhật unread count/notification.
6	Người dùng xem trạng thái gửi/đã nhận/đã đọc nếu có.	Hệ thống cập nhật trạng thái read receipt và đồng bộ trên các thiết bị.

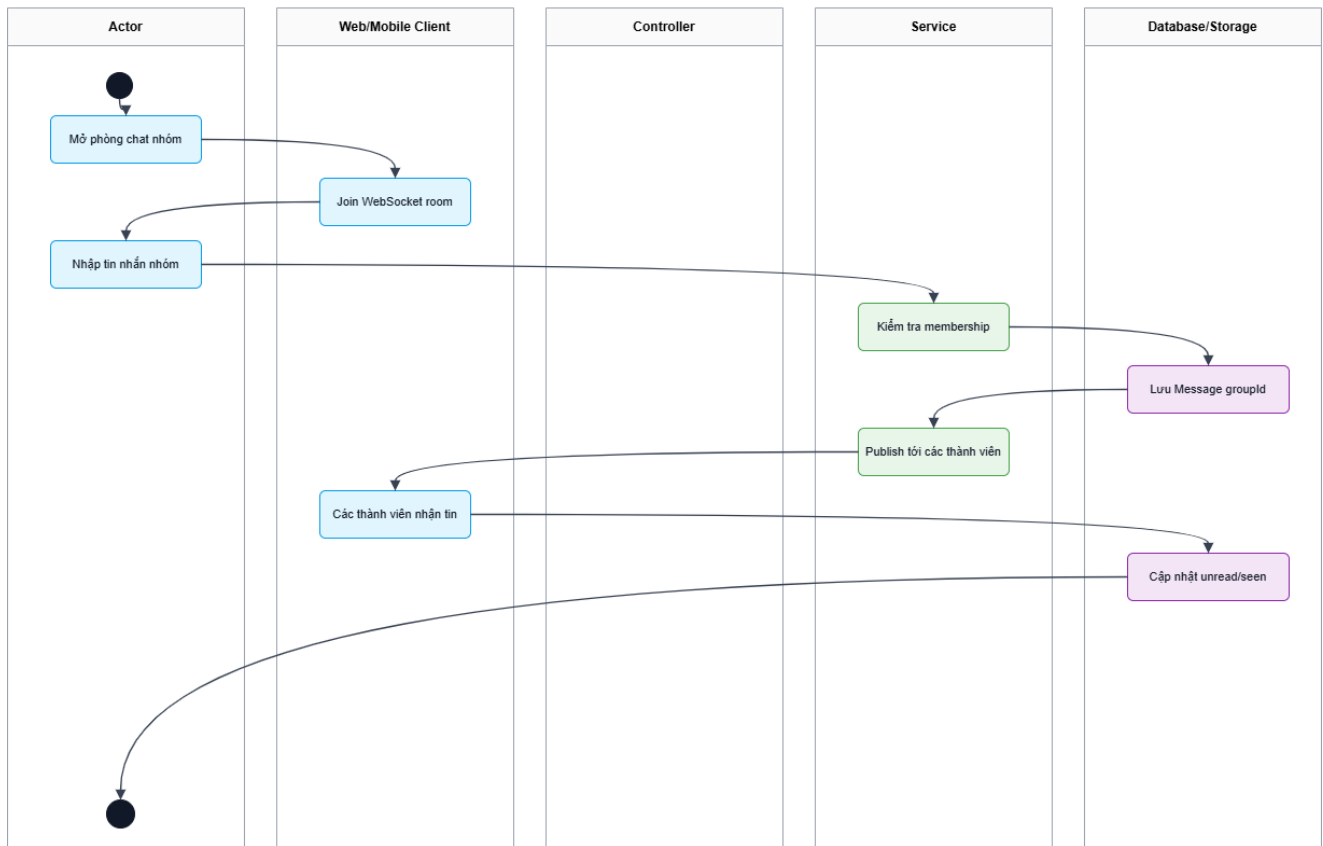
Mã	Luồng thay thế
A1	Người dùng trả lời một tin nhắn cũ: hệ thống lưu thêm replyToMessageId để hiển thị dạng trả lời.
A2	Người dùng gửi file/hình ảnh: hệ thống upload file lên storage trước, sau đó gửi tin nhắn chứa đường dẫn file.
A3	Người dùng thu hồi, xóa hoặc chỉnh sửa tin nhắn: hệ thống kiểm tra quyền và cập nhật trạng thái tin nhắn.
A4	Người dùng thả cảm xúc vào tin nhắn: hệ thống lưu reaction và phát sự kiện cập nhật realtime.

Mã	Luồng ngoại lệ
E1	Người dùng không còn là thành viên nhóm: hệ thống không cho gửi tin nhắn.
E2	Nội dung rỗng và không có file đính kèm: client yêu cầu nhập nội dung.
E3	File quá dung lượng hoặc sai định dạng: hệ thống từ chối upload/gửi.
E4	Kết nối WebSocket bị mất: tin nhắn có thể được gửi qua HTTP và đồng bộ lại khi kết nối khôi phục.
E5	Nhóm bị khóa quyền gửi: chỉ admin/owner được gửi tin nhắn.

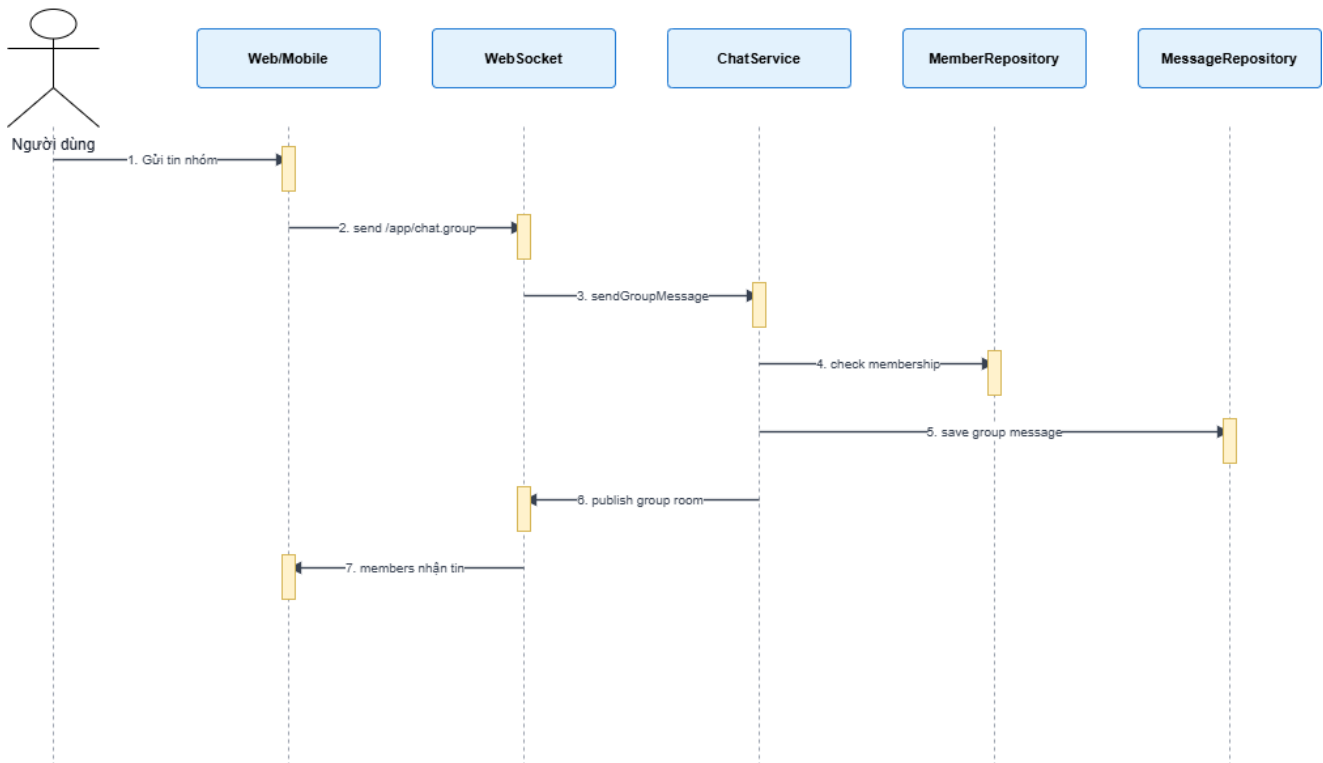
Ghi chú nghiệp vụ:

Tin nhắn nhóm cần ưu tiên tính realtime, đồng bộ trạng thái giữa nhiều thiết bị và đảm bảo người không còn trong nhóm không đọc hoặc gửi được tin nhắn mới. Khi báo cáo, nên mô tả rõ room WebSocket theo groupId.

Activity Diagram - UC14: Nhắn tin nhóm



Sequence Diagram - UC14: Nhắn tin nhóm



3.6.15 UC15: AI gợi ý tin nhắn và trợ lý Gemini

Đặc tả Use case

Mã UC	UC15
Tên use case	AI gợi ý tin nhắn và trợ lý Gemini
Mô tả	Người dùng sử dụng AI để tạo gợi ý viết lại tin nhắn hoặc hỏi trợ lý AI trong khung chat.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; nội dung câu hỏi/tin nhắn không rỗng; backend cấu hình Gemini API key.
Hậu điều kiện	Hệ thống trả về danh sách gợi ý hoặc câu trả lời từ Gemini; người dùng có thể áp dụng gợi ý vào ô nhập tin nhắn.

ST T	Người dùng/Client	Hệ thống
1	Nhập tin nhắn hoặc câu hỏi trong khung chat.	Client hiển thị nút AI gợi ý hoặc mở AiAssistantBox.
2	Bấm nút AI gợi ý cách diễn đạt.	ChatController nhận request POST /api/chat/ai/suggestions.
3	Chờ kết quả.	AIService tạo prompt và gọi Gemini API để sinh danh sách gợi ý.
4	Xem các câu gợi ý.	Backend trả về JSON gồm danh sách suggestions; client hiển thị trong popup.
5	Chọn một gợi ý để áp dụng.	Client đưa câu gợi ý vào ô nhập tin nhắn để người dùng chỉnh tiếp hoặc gửi.
6	Mở AI Chatbox và nhập câu hỏi.	ChatController nhận /api/chat/ai/assistant, gửi context và câu hỏi cho AIService, trả answer về client.

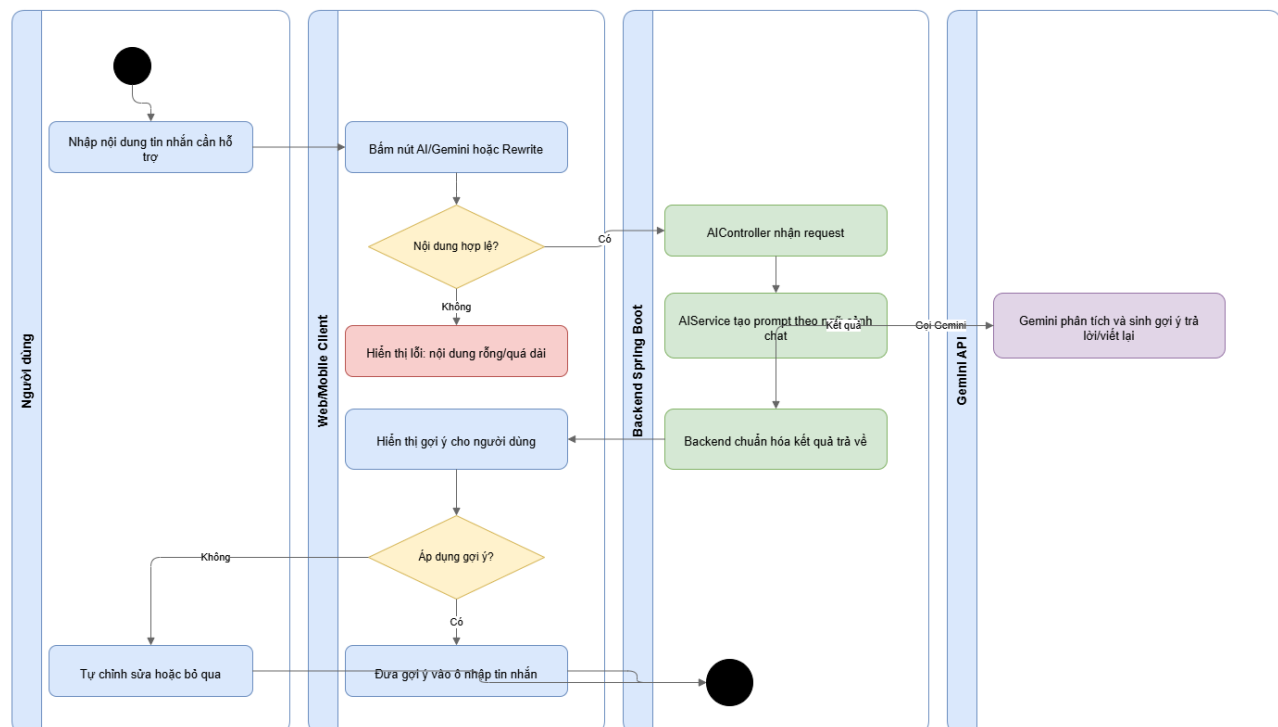
Mã	Luồng thay thế
A1	Nếu Gemini API tạm thời không phản hồi, frontend có thể hiển thị gợi ý fallback đơn giản để người dùng vẫn tiếp tục nhập tin.
A2	Nếu người dùng không chọn gợi ý, nội dung trong ô nhập được giữ nguyên.
A3	Nếu người dùng hỏi về dữ liệu hội thoại, client gửi thêm context để AI trả lời sát dữ liệu hiện có.
A4	Nếu context không đủ, AI trả lời rằng hiện tại app chưa có dữ liệu đó thay vì suy đoán.

Mã	Luồng ngoại lệ
E1	Nội dung rỗng: client yêu cầu nhập tin nhắn hoặc câu hỏi trước khi gọi AI.
E2	Gemini API lỗi hoặc timeout: hiển thị thông báo backend AI chưa sẵn sàng/thử lại sau.
E3	API key Gemini sai hoặc thiếu: backend trả thông báo lỗi cấu hình.
E4	Response AI không đúng định dạng danh sách: hệ thống dùng fallback suggestions.
E5	Người dùng chưa đăng nhập hoặc token hết hạn: hệ thống từ chối request.

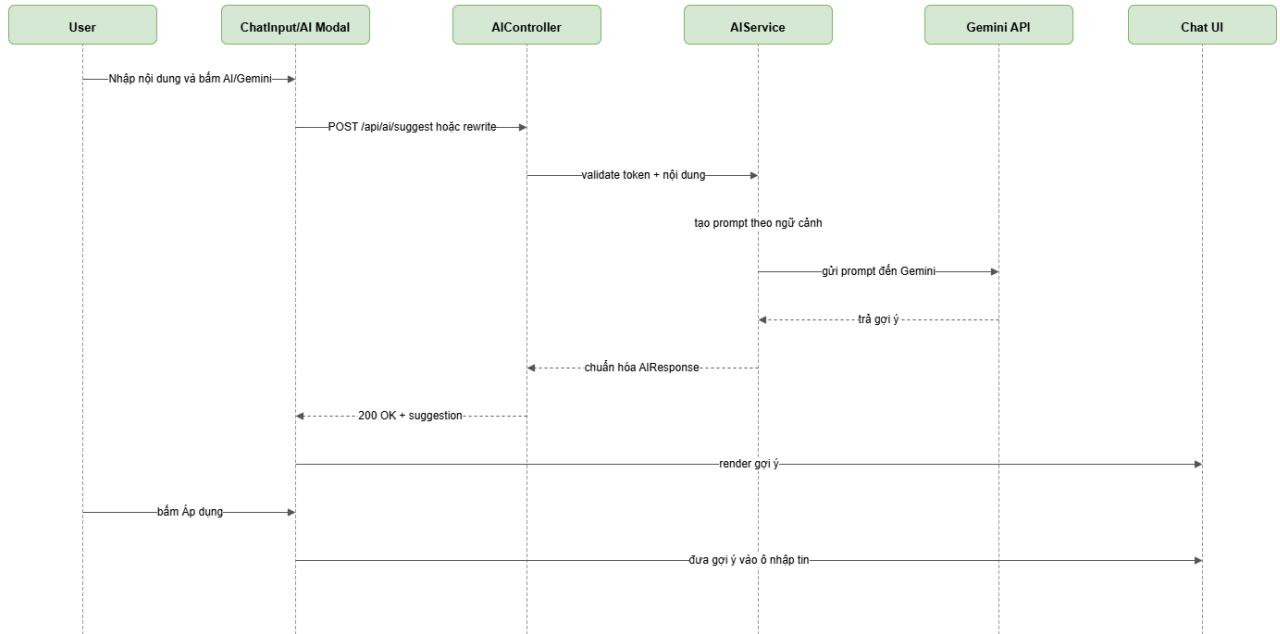
Ghi chú nghiệp vụ:

Chức năng AI chỉ đóng vai trò hỗ trợ người dùng soạn tin nhắn, không nên tự động gửi thay người dùng. Để tránh hiểu nhầm, giao diện cần có nút “Áp dụng” và cho phép người dùng chỉnh sửa trước khi gửi.

ACTIVITY DIAGRAM - UC15: AI GỢI Ý TIN NHẮN VÀ TRỢ LÝ GEMINI



SEQUENCE DIAGRAM - UC15: AI GỢI Ý TIN NHÃN VÀ TRỢ LÝ GEMINI



3.6.16 UC16: Gọi audio/video demo

Đặc tả Use case

Mã UC	UC16
Tên use case	Gọi audio/video demo 1-1
Mô tả	Người dùng thực hiện cuộc gọi audio hoặc video demo trong cuộc trò chuyện cá nhân. Hệ thống gửi tín hiệu mời gọi qua WebSocket; gọi nhóm chưa hoàn thiện và được đưa vào hướng phát triển.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; có cuộc trò chuyện hợp lệ; trình duyệt/thiết bị cho phép sử dụng camera hoặc micro.
Hậu điều kiện	Cuộc gọi demo được thiết lập hoặc kết thúc ở mức giao diện/signaling; chưa lưu lịch sử cuộc gọi hoàn chỉnh.

ST T	Người dùng/Client	Hệ thống
1	Mở cuộc trò chuyện và bấm nút gọi audio/video.	Client yêu cầu quyền truy cập micro/camera từ thiết bị.
2	Cho phép quyền micro/camera.	CallSocketController gửi tín hiệu mời gọi qua /app/call.signal.
3	Người nhận nhận popup cuộc gọi.	Hệ thống phát sự kiện call:incoming qua WebSocket đến client người nhận.

4	Người nhận bấm chấp nhận.	Hệ thống trao đổi thông tin signaling như offer, answer và ICE candidate nếu có WebRTC.
5	Hai bên bắt đầu cuộc gọi.	Client hiển thị màn hình cuộc gọi, thời lượng và trạng thái kết nối.
6	Một bên bấm kết thúc cuộc gọi.	Hệ thống phát sự kiện kết thúc cuộc gọi cho các client liên quan; chưa lưu lịch sử cuộc gọi hoàn chỉnh.

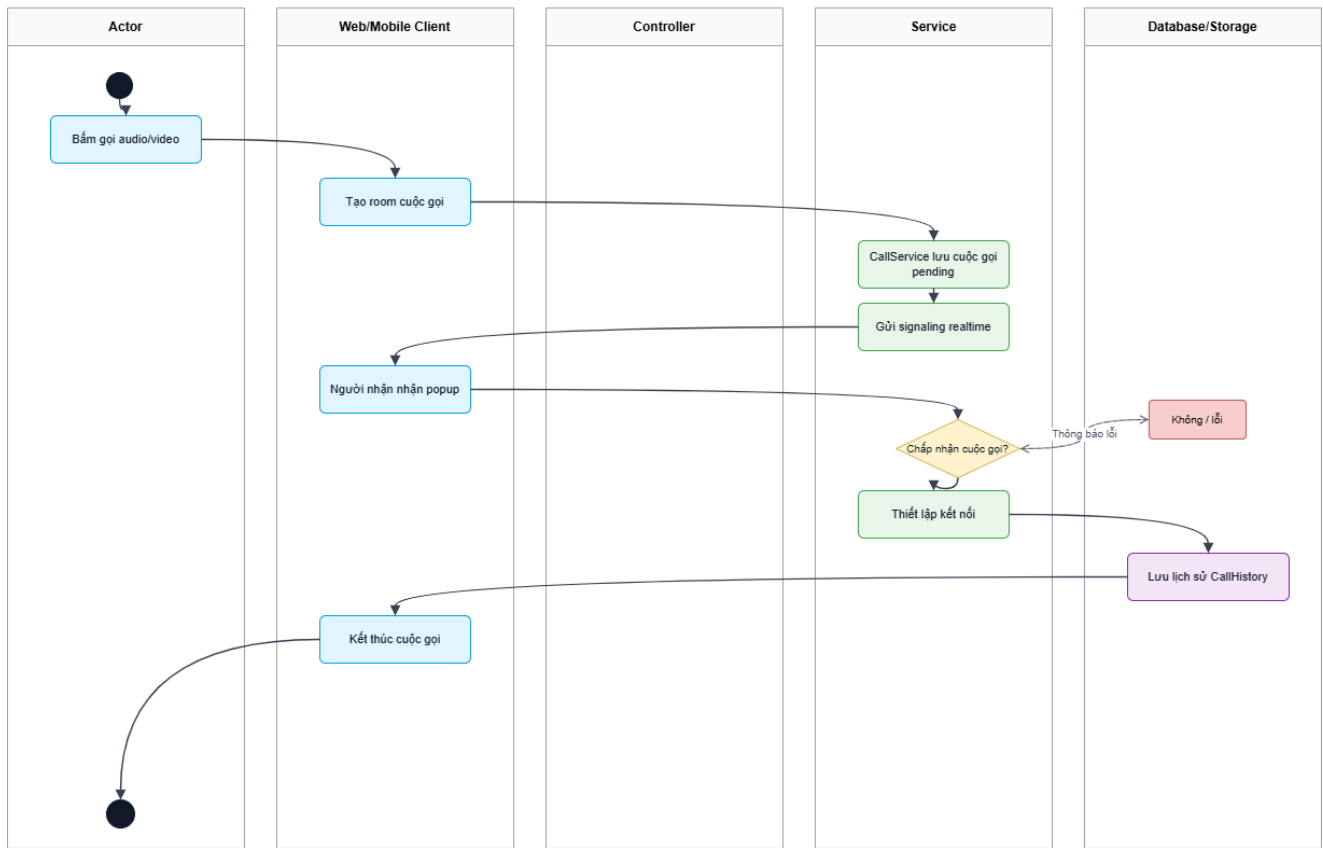
Mã	Luồng thay thế
A1	Người nhận từ chối cuộc gọi: hệ thống lưu trạng thái rejected và thông báo cho người gọi.
A2	Người nhận không phản hồi: sau thời gian chờ, cuộc gọi chuyển sang trạng thái missed.
A3	Nếu chỉ gọi audio, client chỉ yêu cầu quyền micro và không bật camera.
A4	Trong nhóm chat, hệ thống có thể tạo phòng gọi nhóm demo và cho nhiều thành viên tham gia.

Mã	Luồng ngoại lệ
E1	Người dùng từ chối quyền camera/micro: hệ thống không bắt đầu cuộc gọi.
E2	Người nhận offline: hệ thống báo không thể kết nối hoặc lưu cuộc gọi nhờ.
E3	Thiết bị không có camera/micro: chức năng tương ứng bị vô hiệu hóa.
E4	Kết nối mạng yếu hoặc signaling lỗi: cuộc gọi không thiết lập thành công.
E5	Người dùng không có quyền trong cuộc trò chuyện: hệ thống từ chối tạo cuộc gọi.

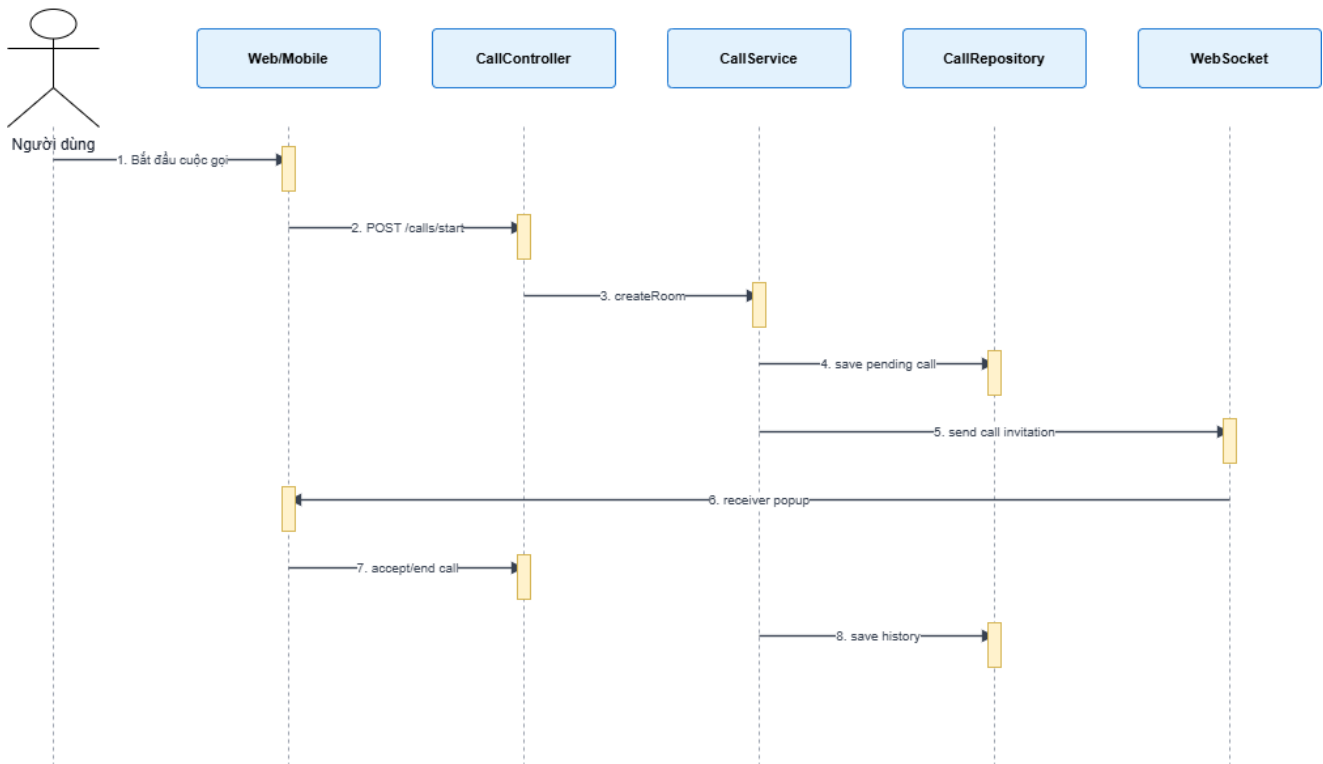
Ghi chú nghiệp vụ:

Nếu project hiện chỉ triển khai mức demo, phần báo cáo nên trình bày rõ phạm vi là xử lý giao diện và signaling cơ bản, không khẳng định đã hoàn thiện media server production.

Activity Diagram - UC16: Gọi audio/video demo



Sequence Diagram - UC16: Gọi audio/video demo



3.6.17 UC17: Xem trạng thái online

Đặc tả Use case

Mã UC	UC17
Tên use case	Xem trạng thái online
Mô tả	Người dùng xem trạng thái online/offline, last seen và typing của bạn bè hoặc thành viên trong cuộc trò chuyện thông qua kết nối realtime.
Actor chính	Người dùng
Tiền điều kiện	Người dùng đã đăng nhập; client kết nối được WebSocket; hệ thống quản lý online dựa trên WebSocket session.
Hậu điều kiện	Giao diện hiển thị trạng thái online, offline, đang nhập hoặc thời gian hoạt động gần nhất của người dùng liên quan.

ST T	Người dùng/Client	Hệ thống
1	Đăng nhập và mở danh sách bạn bè hoặc cuộc trò chuyện.	Client thiết lập kết nối WebSocket với server.
2	Yêu cầu lấy trạng thái người dùng liên quan.	SocketEventListener lấy trạng thái từ danh sách session WebSocket đang kết nối.
3	Xem biểu tượng online/offline trên giao diện.	Hệ thống trả danh sách trạng thái và lastSeen cho client.
4	Bạn bè đăng nhập hoặc thoát ứng dụng.	WebSocketGateway phát sự kiện presence:update đến các client được phép xem.
5	Người dùng nhập tin nhắn.	Client gửi sự kiện typing:start/typing:stop; hệ thống phát lại trong phạm vi cuộc trò chuyện.
6	Giao diện tự cập nhật trạng thái mới.	Client cập nhật badge online, dòng last seen hoặc trạng thái đang nhập.

Mã	Luồng thay thế
A1	Nếu người dùng tắt hiển thị trạng thái, hệ thống chỉ hiển thị trạng thái hạn chế hoặc không hiển thị last seen.
A2	Khi người dùng không thao tác trong thời gian dài, trạng thái có thể chuyển từ online sang away/idle nếu hệ thống hỗ trợ.
A3	Nếu đang xem nhóm chat, client có thể chỉ hiển thị số lượng thành viên online thay vì hiện tất cả từng người.

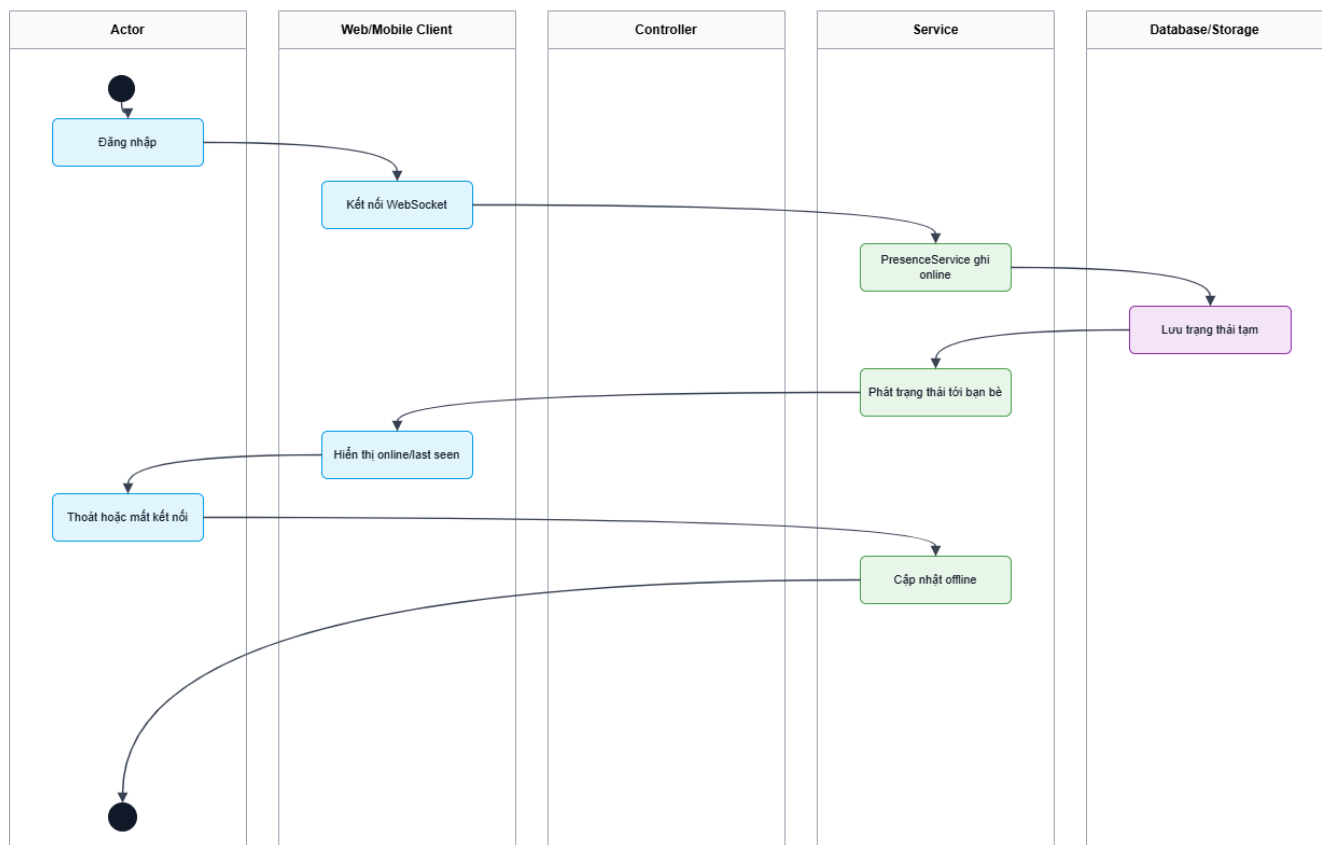
Mã	Luồng ngoại lệ
----	----------------

E1	WebSocket mất kết nối: giao diện giữ trạng thái gần nhất và thử reconnect.
E2	WebSocket session không phản hồi hoặc client mất kết nối: hệ thống cập nhật trạng thái offline tạm thời.
E3	Người xem không có quan hệ bạn bè hoặc không cùng nhóm: hệ thống không trả trạng thái chi tiết.
E4	Client gửi typing quá nhiều: hệ thống giới hạn tần suất để tránh spam sự kiện.

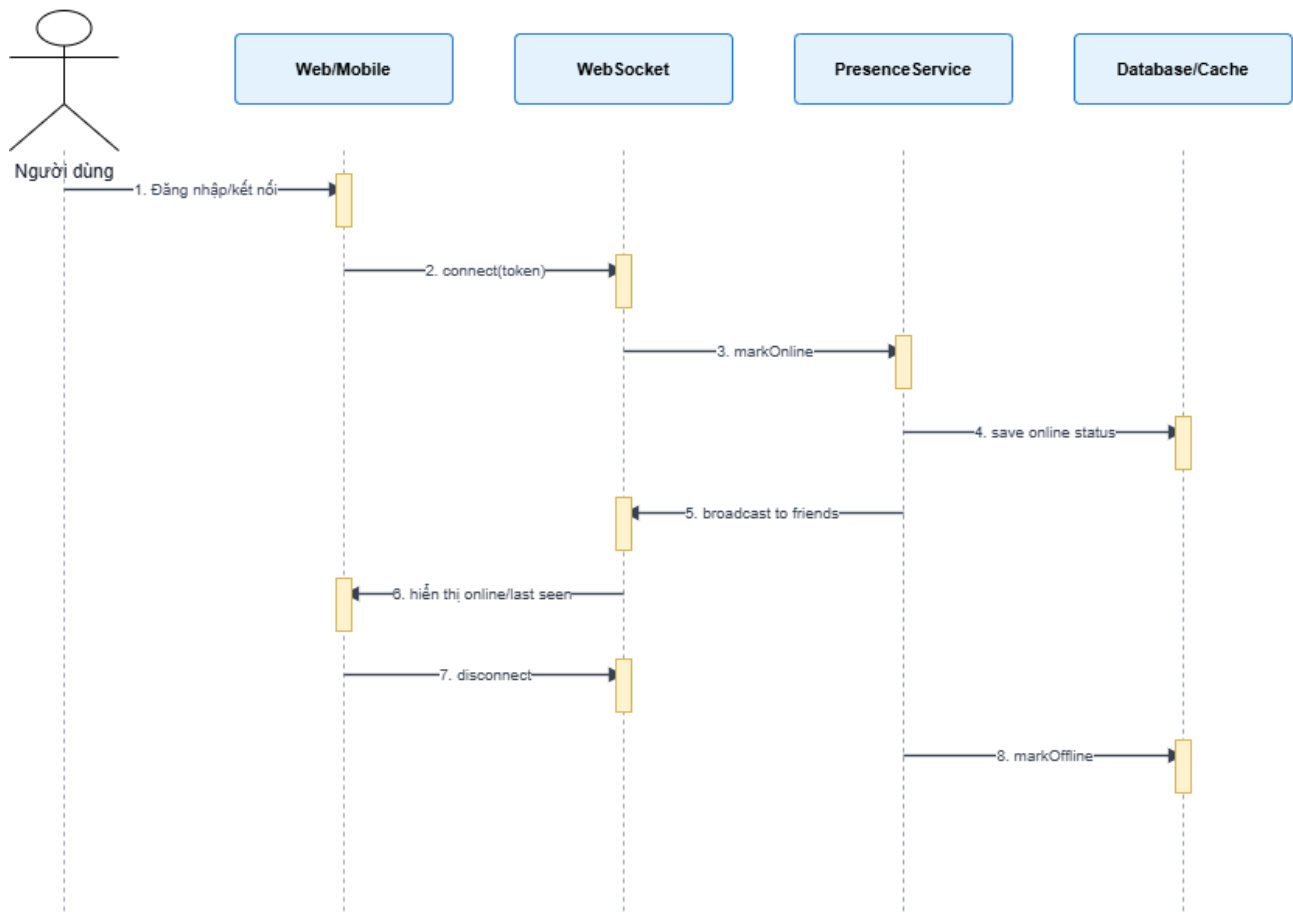
Ghi chú nghiệp vụ:

Presence là dữ liệu thay đổi liên tục nên nên lưu ở bộ nhớ phiên WebSocket thay vì chỉ lưu database. Khi viết báo cáo, cần tách rõ trạng thái realtime (online/typing) và trạng thái lưu lâu hơn (lastSeen).

Activity Diagram - UC17: Xem trạng thái online



Sequence Diagram - UC17: Xem trạng thái online



3.6.18 UC18: Quản trị người dùng

Đặc tả Use case

Mã UC	UC18
Tên use case	Quản trị người dùng
Mô tả	Admin hệ thống đăng nhập vào trang /admin, xem danh sách người dùng và xóa user thông qua API quản trị cơ bản.
Actor chính	Admin hệ thống
Tiền điều kiện	Admin đã đăng nhập; token có role ADMIN; frontend route /admin được bảo vệ bằng ProtectedRoute.
Hậu điều kiện	Danh sách người dùng được hiển thị hoặc user được xóa khỏi hệ thống nếu thao tác hợp lệ.

ST T	Người dùng/Client	Hệ thống
---------	-------------------	----------

1	Admin đăng nhập bằng tài khoản admin.	AuthController xác thực và trả JWT có role ADMIN.
2	Admin được điều hướng vào /admin.	ProtectedRoute kiểm tra roleRequired="ADMIN".
3	Trang admin yêu cầu danh sách user.	AdminUserController xử lý GET /api/admin/users.
4	Admin xem danh sách người dùng.	UserService trả danh sách UserResponse.
5	Admin chọn xóa một user.	AdminUserController xử lý DELETE /api/admin/users/{id} và gọi UserService.deleteUser.
6	Giao diện làm mới danh sách.	Backend trả thông báo thành công; frontend có thể gọi lại API danh sách.

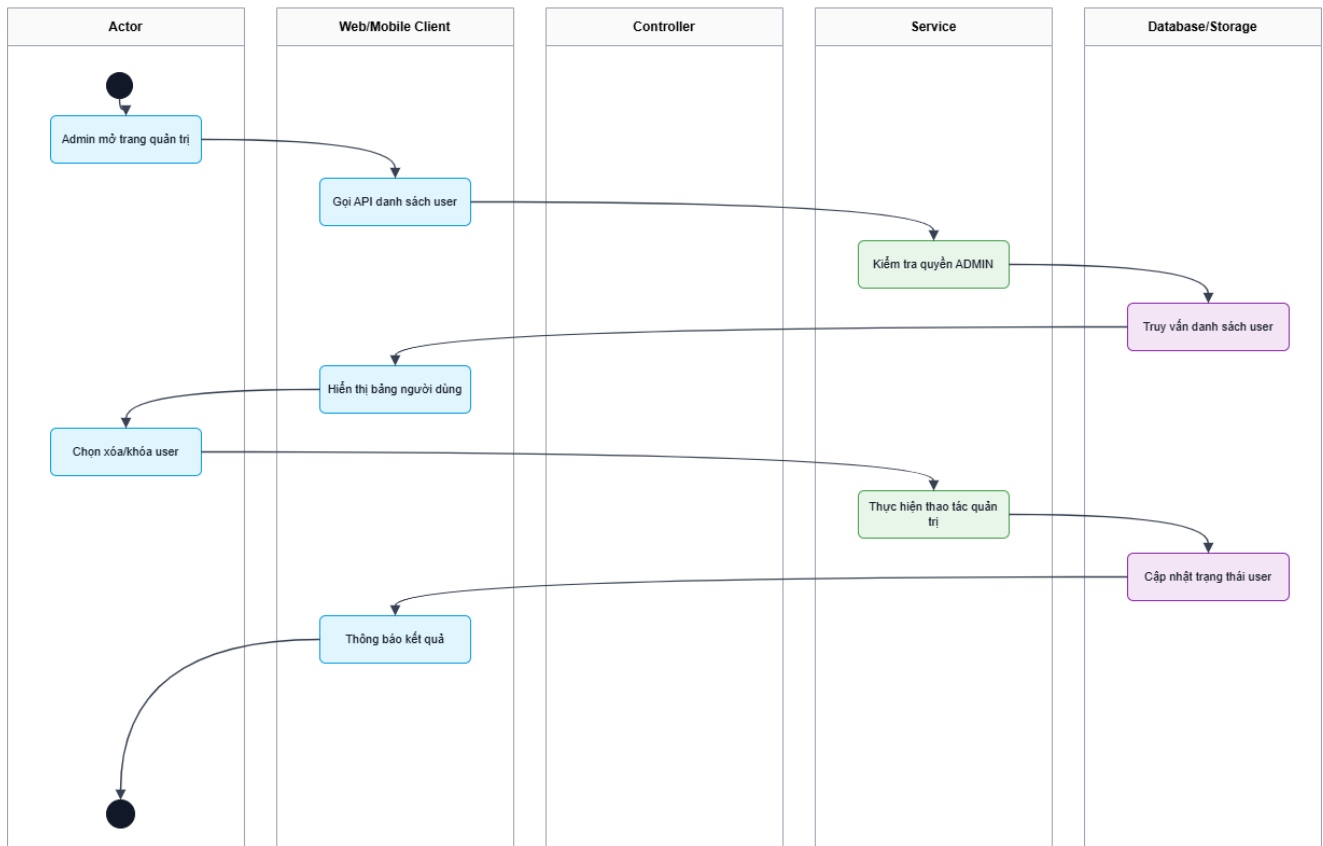
Mã	Luồng thay thế
A1	Admin chỉ xem danh sách mà không thay đổi dữ liệu: hệ thống chỉ thực hiện truy vấn GET /api/admin/users.
A2	Nếu frontend admin chưa hoàn thiện UI, chức năng vẫn có thể kiểm tra qua API backend.
A3	Nếu không phải ADMIN, ProtectedRoute chuyển người dùng về trang phù hợp hoặc chặn truy cập.

Mã	Luồng ngoại lệ
E1	Người thao tác không có quyền ADMIN: hệ thống từ chối truy cập trang/API quản trị.
E2	Không tìm thấy người dùng cần xóa: hệ thống báo lỗi và không cập nhật dữ liệu.
E3	Token hết hạn hoặc sai: request bị từ chối.
E4	Xóa user đang có dữ liệu liên quan có thể gây lỗi ràng buộc; cần xử lý kiểm tra trước khi xóa.
E5	Lỗi database: hệ thống trả thông báo thất bại.

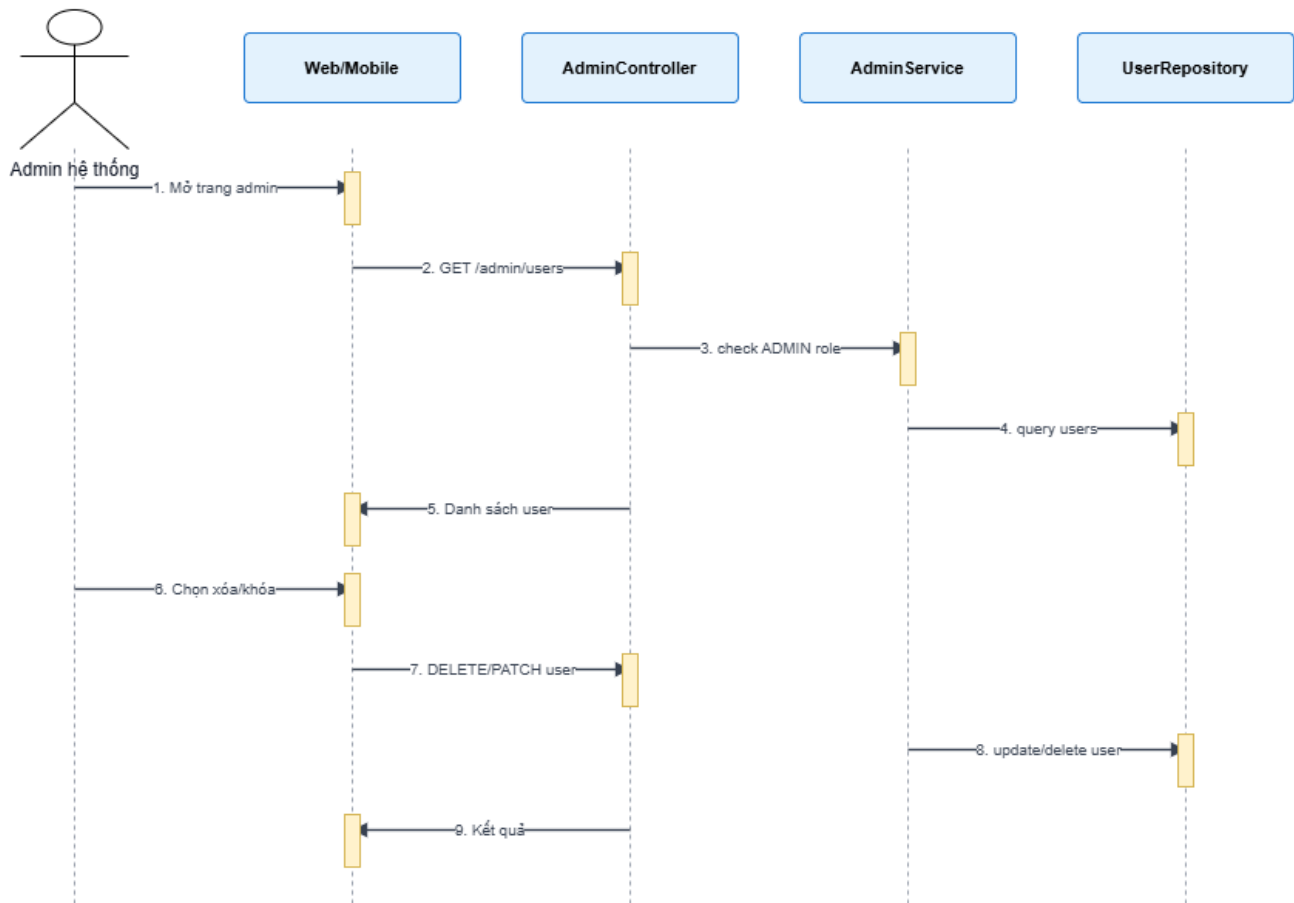
Ghi chú nghiệp vụ:

Nếu project chưa có giao diện admin hoàn chỉnh, có thể trình bày UC18 như chức năng dự kiến/mở rộng hoặc mô tả ở mức API quản trị. Không nên mô tả quá mức những màn hình chưa có nếu không có ảnh minh chứng.

Activity Diagram - UC18: Quản trị người dùng



Sequence Diagram - UC18: Quản trị người dùng



3.6.19 UC19: Tạo và quản lý bình chọn trong nhóm

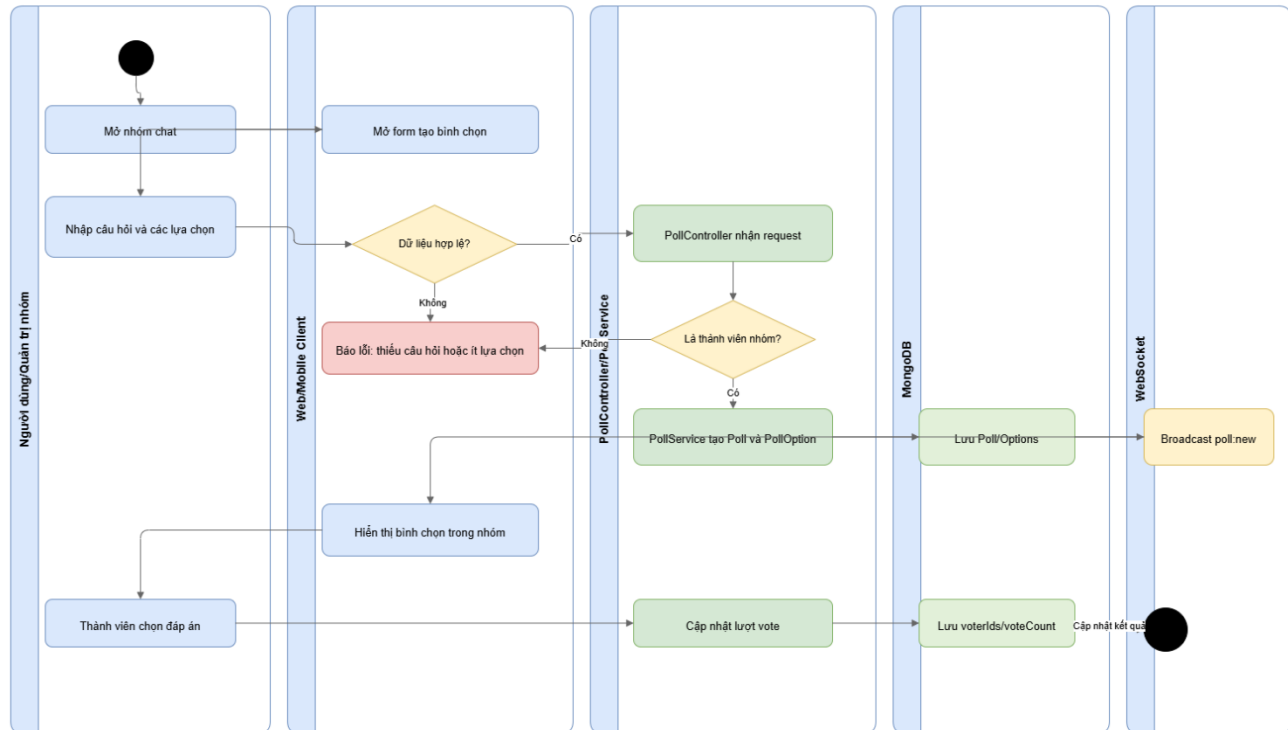
Mã UC	UC19	
Tên use case	Tạo và quản lý bình chọn trong nhóm	
Mô tả	Người dùng tạo poll trong nhóm/hội thoại, thêm lựa chọn, vote và đóng bình chọn.	
Actor chính	Người dùng, Quản trị nhóm	
Tiền điều kiện	Người dùng đã đăng nhập; đang ở room hợp lệ; PollController và PollService hoạt động.	
Hậu điều kiện	Poll được lưu trong MongoDB, hiển thị trong room và kết quả vote được cập nhật cho các thành viên.	
STT	Người dùng/Client	Hệ thống

1	Mở cuộc trò chuyện nhóm và chọn tạo bình chọn.	Client mở CreatePollModal.
2	Nhập câu hỏi, các lựa chọn và cấu hình bình chọn.	PollController nhận CreatePollRequest.
3	Bấm tạo bình chọn.	PollService tạo Poll và PollOption trong MongoDB.
4	Poll xuất hiện trong khung chat.	Message có thể gắn pollId để hiển thị PollMessageCard.
5	Thành viên chọn đáp án và bấm vote.	PollService cập nhật voters trong PollOption.
6	Admin/người tạo đóng bình chọn nếu cần.	PUT /api/chat/polls/{pollId}/close cập nhật trạng thái closed.

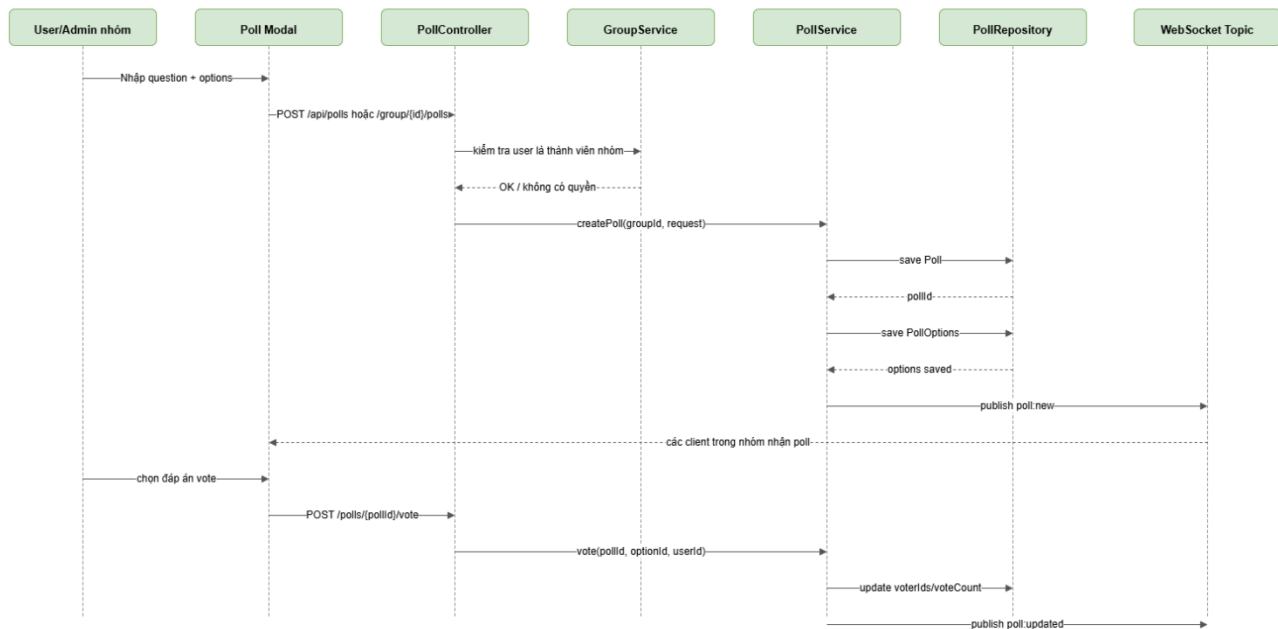
Mã	Luồng thay thế
A1	Poll cho phép chọn nhiều đáp án nếu multipleChoice được bật.
A2	Nếu allowAddOption bật, thành viên có thể thêm lựa chọn mới.
A3	Nếu poll ẩn kết quả đến khi vote, client chỉ hiển thị kết quả sau khi người dùng đã bình chọn.
Mã	Luồng ngoại lệ
E1	Câu hỏi hoặc danh sách lựa chọn không hợp lệ: không tạo poll.
E2	Poll đã đóng: không cho vote hoặc thêm lựa chọn.
E3	Người dùng không thuộc room: hệ thống từ chối thao tác.

Ghi chú nghiệp vụ: UC19 liên quan PollController, PollService, PollRepository, CreatePollModal và PollMessageCard. Ảnh giao diện cần chụp: modal tạo bình chọn, poll trong khung chat và kết quả sau khi vote.

ACTIVITY DIAGRAM - UC19: TẠO VÀ QUẢN LÝ BÌNH CHỌN TRONG NHÓM



SEQUENCE DIAGRAM - UC19: TẠO VÀ QUẢN LÝ BÌNH CHỌN TRONG NHÓM

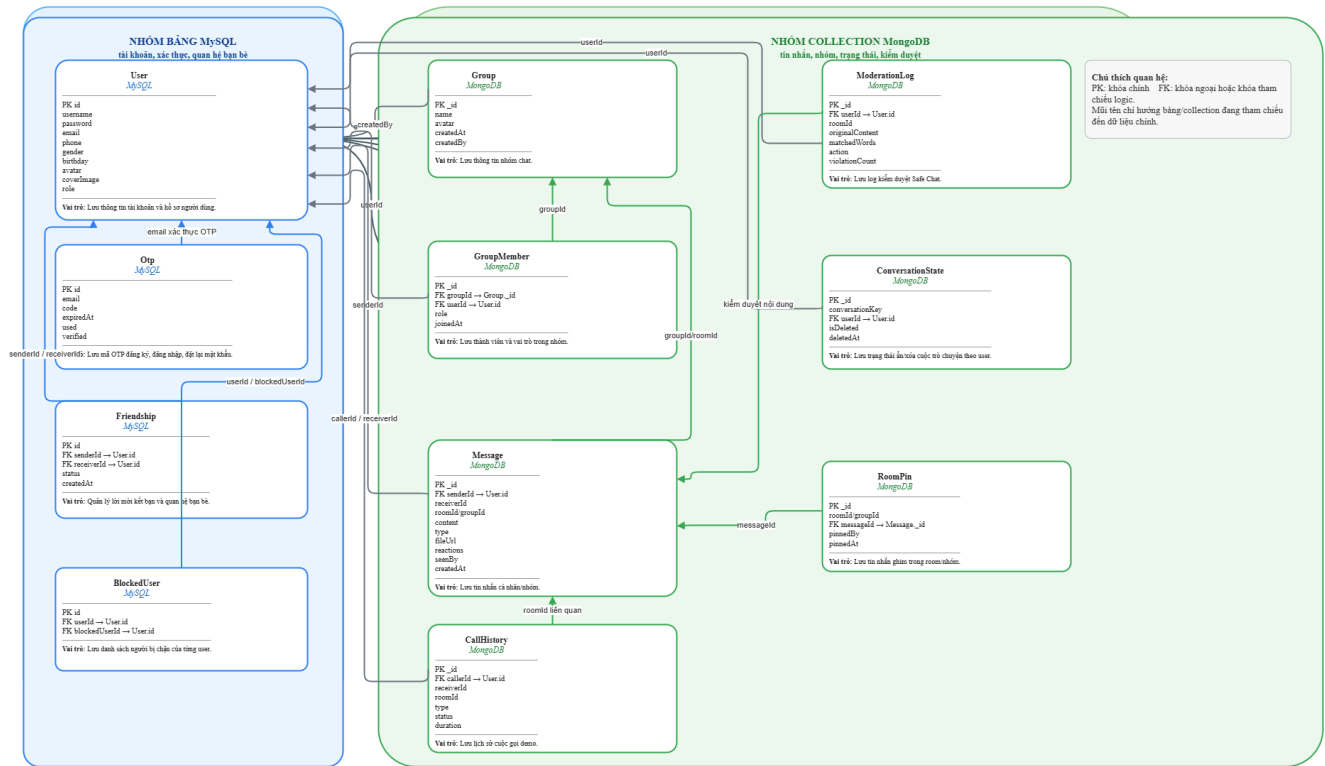


3.7 Thiết kế dữ liệu

Entity/Collection	Kiểu lưu trữ	Trường chính	Vai trò
User	MySQL	id, username, password, email, phone, gender,	Lưu tài khoản, hồ sơ và phân quyền.

		birthday, avatar, coverImage, role	
Otp	MySQL	id, email, code, createdAt, expiredAt, used, verified	Lưu mã OTP đăng ký/quên mật khẩu.
Friendship	MySQL	id, senderId, receiverId, status, createdAt, updatedAt	Quản lý lời mời và quan hệ bạn bè.
BlockedUser	MySQL	id, userId, blockedUserId	Quản lý quan hệ chặn người dùng.
Conversation Background	MySQL	id, roomId, background, updatedBy, updatedAt	Lưu nền cuộc trò chuyện theo room.
Message	MongoDB	id, senderId, receiverId, roomId, content, type, fileUrl, isDeleted, isRecalled, pollId, pinned	Lưu tin nhắn cá nhân/nhóm, file, poll và trạng thái ghim.
MessageReaction	MongoDB	id, messageId, roomId, userId, emoji, createdAt	Lưu reaction emoji của tin nhắn.
Group	MongoDB	id, name, avatar, createdBy, createdAt	Lưu thông tin nhóm chat.
GroupMember	MongoDB	id, groupId, userId, role, removed, removedAt, removedBy	Lưu thành viên, vai trò và trạng thái rời/xóa khỏi nhóm.
Conversation State	MongoDB	id, conversationKey, userId, isDeleted, deletedAt	Lưu trạng thái ẩn/xóa cuộc trò chuyện theo user.
Conversation Nickname	MongoDB	id, conversationKey, ownerUserId, targetUserId, nickname, updatedAt	Lưu biệt danh hội thoại do từng user đặt.
RoomPin	MongoDB	id, roomId, messageId, pinnedBy, pinnedAt	Lưu tin nhắn ghim trong room.
Poll	MongoDB	id, roomId, question, createdBy, options, multipleChoice, allowAddOption, anonymous, closed	Lưu bình chọn trong nhóm/hội thoại.

ERD - THIẾT KẾ DỮ LIỆU HỆ THỐNG CHATAPP



3.8 Thiết kế API backend

Nhóm API	Endpoint tiêu biểu	Chức năng
Auth	POST /api/auth/register, /login, /forgot-password, /reset-password	Đăng ký, đăng nhập, quên mật khẩu.
OTP	POST /api/auth/send-otp, /verify-otp	Gửi và xác thực OTP.
User	GET /api/user/me, /{id}; PUT /api/user/update; POST /upload-avatar, /upload-cover, /change-password	Hồ sơ cá nhân, avatar, cover, đổi mật khẩu.
Friend	POST /api/friends/request, /accept/{id}, /reject/{id}; GET /api/friends, /requests, /search; DELETE /api/friends/{friendId}	Kết bạn, tìm kiếm, hủy bạn bè.
Block	POST /api/block/{targetId}; DELETE /api/block/{targetId}; GET /check/{targetId}, /status/{targetId}	Chặn, bỏ chặn, kiểm tra trạng thái chặn.
Chat	GET /api/chat/messages/{roomId}; DELETE /message/{id}; PUT /message/recall/{id}; POST/DELETE /message/{id}/reaction; POST /read/{roomId}; GET /unread/{roomId}	Lịch sử tin nhắn, xóa/thu hồi/reaction, read/unread.

Chat cá nhân hóa	PUT/GET /api/chat/background/{roomId}; POST /nickname/update; GET /nickname; PUT /message/{id}/pin, /unpin; GET /pinned/{roomId}	Đổi nền hội thoại, biệt danh và ghim tin nhắn.
AI	POST /api/chat/ai/suggestions, /api/chat/ai/assistant	Gợi ý viết lại tin nhắn và trợ lý Gemini.
WebSocket	/app/chat.send, /app/call.signal; subscribe /topic/chat/{roomId}, /topic/users/status, /topic/users/list	Gửi tin nhắn, call signal, online status realtime.
Group	POST /api/chat/group/create, /add-member; GET /my-groups, /members; PUT /name, /avatar, /update-role; DELETE /remove-member, /leave, /delete	Tạo nhóm và quản lý nhóm.
Poll	POST /api/chat/polls; GET /api/chat/polls/{pollId}; POST /{pollId}/vote, /{pollId}/options; PUT /{pollId}/close	Tạo poll, xem poll, vote, thêm option, đóng poll.
File	POST /api/file/upload	Upload file dùng trong chat, lưu local /uploads.
Conversation	DELETE /api/conversations/{partnerId}, PUT /restore/{partnerId}	Ẩn/xóa hoặc khôi phục hội thoại.
Admin	GET /api/admin/users, DELETE /api/admin/users/{id}	Quản trị người dùng.

CHƯƠNG 4: HIỆN THỰC HỆ THỐNG

4.1 Môi trường phát triển

Thành phần	Công nghệ/Công cụ	Vai trò
Backend	Java 17, Spring Boot 3.5.13, Maven	Xây dựng API, WebSocket, security và service nghiệp vụ.
Cơ sở dữ liệu	MySQL, MongoDB	Lưu tài khoản, quan hệ, tin nhắn, nhóm và log.
Web client	React 19, Vite, React Router, Axios, @stomp/stompjs, sockjs-client, emoji-picker-react	Giao diện web và kết nối API/realtime.
Mobile client	React Native 0.81.5, Expo 54, expo-router, AsyncStorage, Axios, STOMP	Ứng dụng mobile đa nền tảng.
Upload/Email	MultipartFile, local /uploads, S3Service, Spring Mail	Upload file chat, avatar/cover và gửi OTP.
Thiết kế sơ đồ	draw.io/Mermaid	Vẽ use case, activity, sequence, ERD, architecture.
AI	Gemini API qua AIService	Gợi ý viết lại tin nhắn và trợ lý AI trong chat.

4.2 Cấu trúc thư mục project

Thư mục/File	Mô tả
ChatApp/	Backend Spring Boot.
ChatApp/src/main/java/com/chatapp/config	Cấu hình security, JWT filter, CORS, WebSocket, OpenAPI, S3.
ChatApp/src/main/java/com/chatapp/controller	Các REST controller và websocket mapping.
ChatApp/src/main/java/com/chatapp/service	Xử lý nghiệp vụ chính.
ChatApp/src/main/java/com/chatapp/repository	Repository thao tác MySQL/MongoDB.
ChatApp/src/main/java/com/chatapp/entity	Entity/Document của hệ thống.
ChatApp/src/main/java/com/chatapp/dto	DTO request/response.
frontendAppChat/src/features/auth	Trang và API đăng nhập, đăng ký, quên mật khẩu.
frontendAppChat/src/features/chat	Màn hình chat, chat box, input, call modal, socket.

frontendAppChat/src/features/friend	Tìm bạn, danh sách bạn bè, lời mời.
frontendAppChat/src/features/user	Profile, edit profile, đổi mật khẩu.
AppMobile/chatapp-mobile/app	Các route mobile bằng expo-router.
AppMobile/chatapp-mobile/src/shared/api	Axios dùng chung cho mobile.
AppMobile/chatapp-mobile/src/shared/socket	Kết nối socket trên mobile.
ChatApp/src/main/java/com/chatapp/controller/PollController.java	Controller xử lý tạo/vote/đóng bình chọn.
ChatApp/src/main/java/com/chatapp/entity/Poll.java	Document MongoDB lưu câu hỏi, lựa chọn và cấu hình bình chọn.
ChatApp/src/main/java/com/chatapp/entity/ConversationBackground.java	Entity MySQL lưu nền hội thoại.
ChatApp/src/main/java/com/chatapp/entity/ConversationNickname.java	Document MongoDB lưu biệt danh trong hội thoại.
frontendAppChat/src/features/chat/components/CreatePollModal.jsx	Giao diện tạo bình chọn trên web.

4.3 Hiện thực backend

4.3.1 Lớp cấu hình

Backend có các lớp cấu hình quan trọng như SecurityConfig, JwtFilter, WebConfig, WebSocketConfig, OpenApiConfig, PasswordConfig và S3Config. Các lớp này quyết định cách request đi vào hệ thống, endpoint nào cần bảo vệ, client nào được phép truy cập và WebSocket hoạt động tại đường dẫn nào.

4.3.2 Controller

Controller là nơi nhận request từ client. Ví dụ AuthController xử lý đăng ký/đăng nhập, UserController xử lý hồ sơ, FriendshipController xử lý bạn bè, ChatController xử lý tin nhắn, GroupController xử lý nhóm, FileController xử lý upload và CallController xử lý tín hiệu cuộc gọi.

4.3.3 Service

Service chứa logic nghiệp vụ. ChatService xử lý gửi tin nhắn, kiểm tra block, reaction, ghim tin, nền hội thoại và biệt danh; GroupService kiểm tra quyền quản trị nhóm; FriendshipService xử lý lời mời; PollService xử lý bình chọn; AIService gọi Gemini để gợi ý/assistant; JwtService sinh và kiểm tra token.

4.3.4 Repository

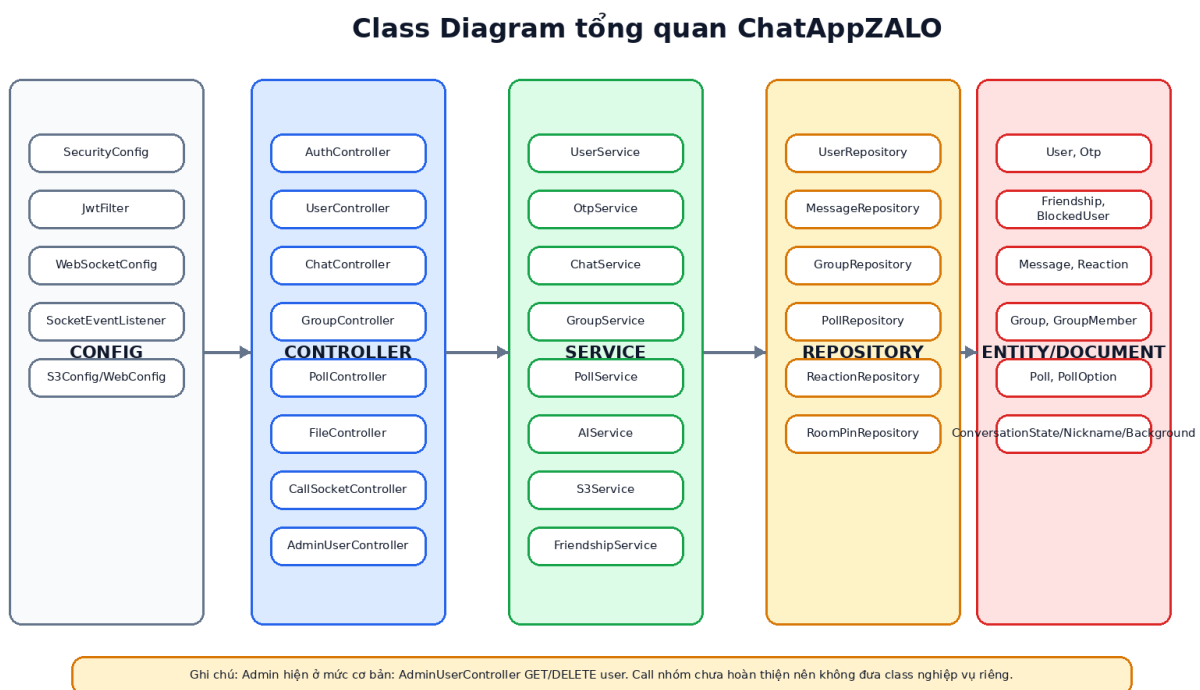
Repository làm việc với dữ liệu. Project có các repository như UserRepository, OtpRepository, FriendshipRepository, BlockedUserRepository, MessageRepository, GroupRepository, GroupMemberRepository, MessageReactionRepository, PollRepository, ConversationStateRepository, ConversationBackgroundRepository, ConversationNicknameRepository và RoomPinRepository.

4.3.5 DTO và response

DTO giúp tách dữ liệu request/response khỏi entity. Các DTO như LoginRequest, RegisterRequest, SendMessageDTO, CreateGroupDTO, UserResponse, FriendResponseDTO giúp API rõ ràng hơn và tránh gửi dư dữ liệu nhạy cảm.

4.3.6 Xử lý lỗi

GlobalExceptionHandler giúp gom xử lý lỗi chung, tránh việc controller phải viết lặp lại try-catch. Khi có lỗi validate, lỗi quyền hoặc lỗi dữ liệu, backend nên trả response dễ hiểu để client hiển thị cho người dùng.



Hình 4.1 Luồng request backend đã hiện thực

GHI CHÚ THAY SƠ ĐỒ/HÌNH: Có thể dùng 06_ClassDiagram_ChatAppZALO_v9.drawio hoặc 01_Architecture_ChatAppZALO_v9.drawio để minh họa controller/service/repository đúng với code.

4.4 Hiện thực các controller chính

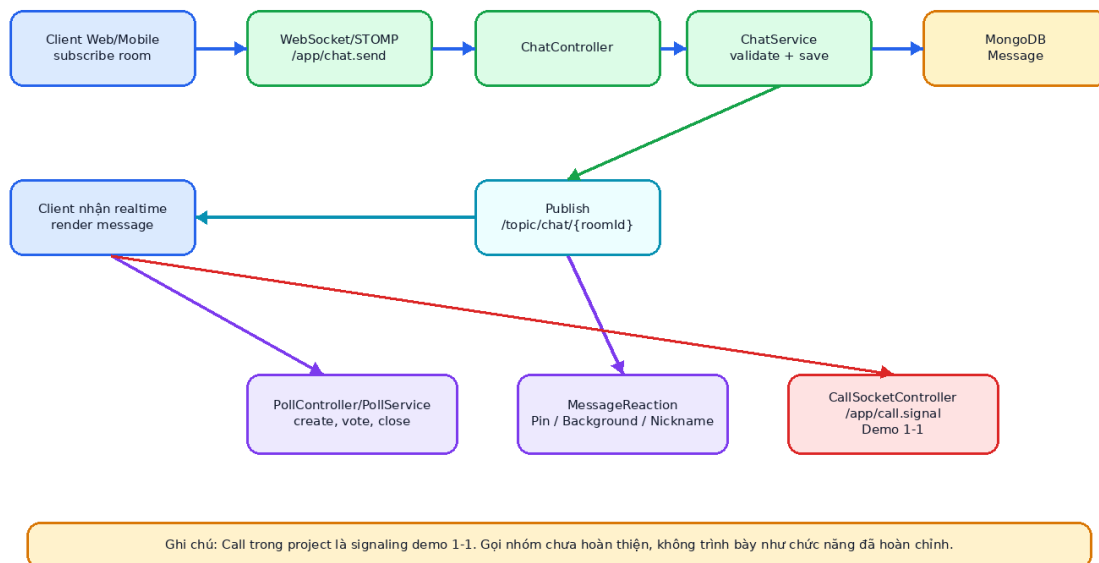
Controller	Endpoint/Mapping chính	Nhiệm vụ
AuthController	/api/auth/register, /login, /forgot-password, /reset-password	Quản lý đăng ký, đăng nhập và đặt lại mật khẩu.
OtpController	/api/auth/send-otp, /verify-otp	Gửi và xác thực OTP.
UserController	/api/user/me, /update, /upload-avatar, /upload-cover, /change-password, /{id}	Quản lý hồ sơ cá nhân.
FriendshipController	/api/friends/request, /accept, /reject, /search, DELETE /{friendId}	Kết bạn và danh sách bạn bè.
BlockController	/api/block/{targetId}, /check, /status	Chặn và kiểm tra trạng thái chặn.
ChatController	/api/chat/messages, /message/recall, /reaction, /background, /nickname, /pinned, /ai/suggestions, /ai/assistant, @MessageMapping chat.send	Tin nhắn, reaction, thu hồi, nền, biệt danh, ghim tin và AI Gemini.
GroupController	/api/chat/group/create, /members, /add-member, /remove-member, /name, /avatar, /update-role, /leave, /delete	Tạo và quản lý nhóm.
PollController	/api/chat/polls, /{pollId}, /vote, /options, /close	Tạo bình chọn, vote, thêm lựa chọn và đóng bình chọn.
FileController	/api/file/upload	Upload file chat vào thư mục uploads.
CallSocketController	@MessageMapping /call.signal	Gửi tín hiệu cuộc gọi audio/video demo qua WebSocket.
UserStatusController / SocketEventListener	/topic/users/status, /topic/users/list	Cập nhật trạng thái online/offline dựa trên WebSocket session.
AdminUserController	/api/admin/users	Trang quản trị user.

4.5 Hiện thực WebSocket và realtime

Khi client mở ứng dụng chat, client tạo kết nối WebSocket đến backend và subscribe vào room tương ứng. Khi người dùng gửi tin nhắn, client gửi payload đến destination của server. Backend kiểm tra dữ liệu, lưu tin nhắn rồi publish message mới đến các client đang subscribe room đó.

WebSocket cũng được sử dụng cho tín hiệu gọi audio/video. Trong phạm vi đồ án, phần gọi ở mức demo nhằm thể hiện quá trình gửi signal giữa hai client, chưa đi sâu vào tối ưu media server hoặc triển khai production phức tạp.

Luồng realtime ChatAppZALO: chat, poll, reaction và call signa



Hình 4.2 Luồng realtime trong chức năng chat

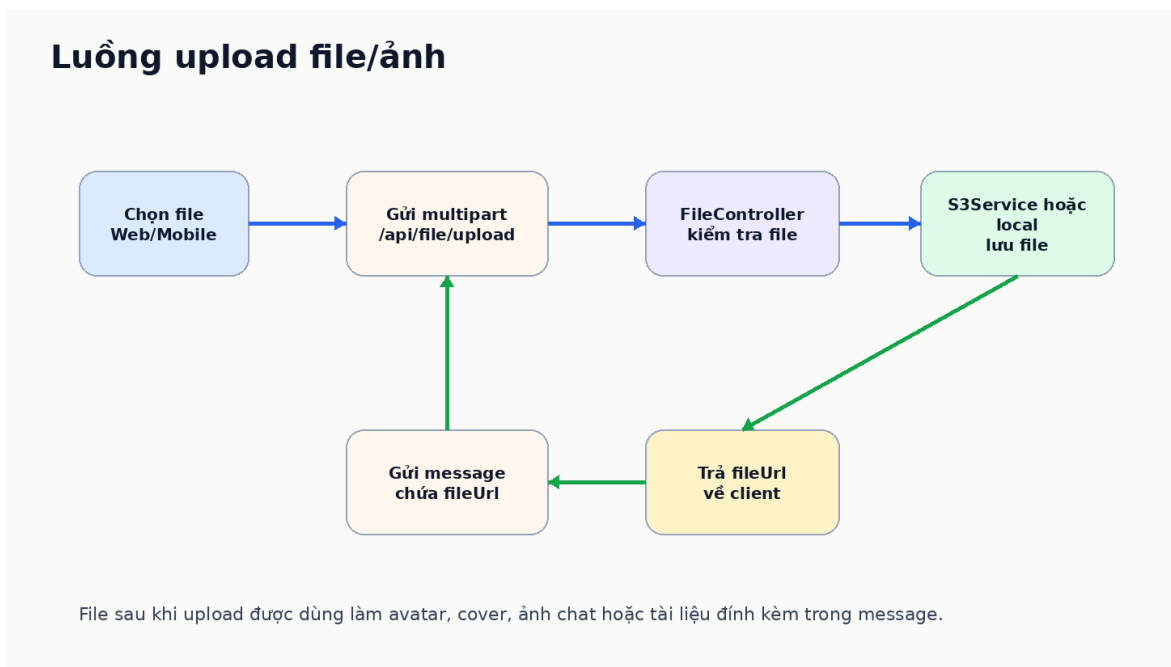
GHI CHÚ THAY SƠ ĐỒ/HÌNH: Dán sơ đồ mới

05_Realtime_Message_Call_Poll_ChatAppZALO_v9.drawio nếu thay hình cũ. Lưu ý call nhóm chưa hoàn thiện, chỉ trình bày call signal demo 1-1.

4.6 Hiện thực upload file

Chức năng upload được dùng ở nhiều vị trí: gửi file trong chat, đổi avatar, đổi cover và đổi avatar nhóm. Backend nhận file theo dạng multipart/form-data, lưu file và trả đường dẫn. Client dùng đường dẫn này để hiển thị lại ảnh hoặc tạo tin nhắn đính kèm.

Khi mở rộng production, storage có thể chuyển sang Amazon S3 hoặc dịch vụ tương đương để tách file khỏi server backend. Trong project đã có S3Config và S3Service để hỗ trợ hướng mở rộng này.



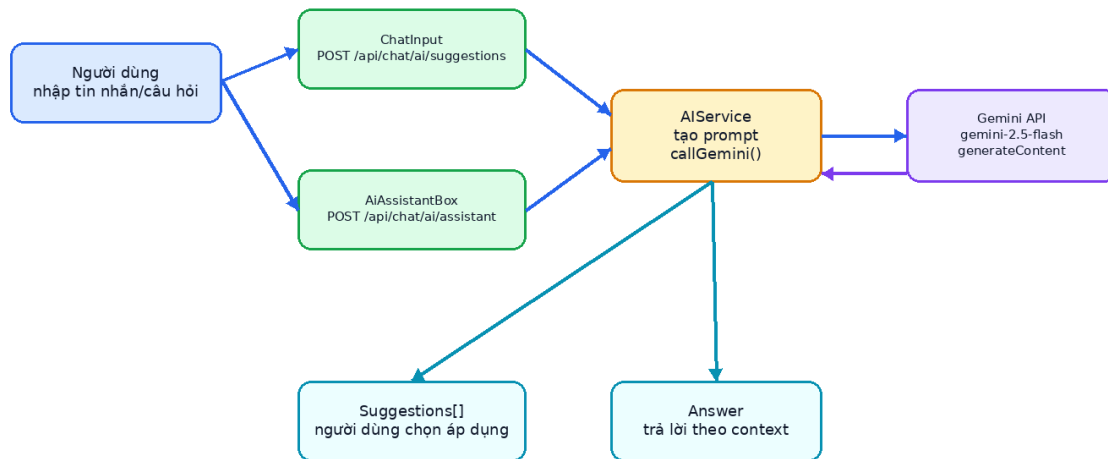
Hình 4.3 Luồng upload file trong hệ thống

4.7 Hiện thực AI Gemini

Gemini AI trong đề tài được dùng cho hai chức năng chính: gợi ý viết lại tin nhắn và trợ lý AI trong khung chat. Backend có AIService gọi Gemini API; frontend có ChatInput để hiển thị gợi ý và AiAssistantBox để người dùng hỏi AI. Chức năng này hỗ trợ người dùng trước khi gửi tin, không tự động chặn nội dung và không thay thế quyết định của người dùng.

Trong backend, lớp liên quan trực tiếp là AIService. AIService gọi Gemini API để tạo gợi ý viết lại tin nhắn và trả lời trong hộp trợ lý AI. Ở frontend, ChatInput và AiAssistantBox hiển thị các gợi ý để người dùng chọn áp dụng trước khi gửi.

Quy trình Gemini AI Assistant

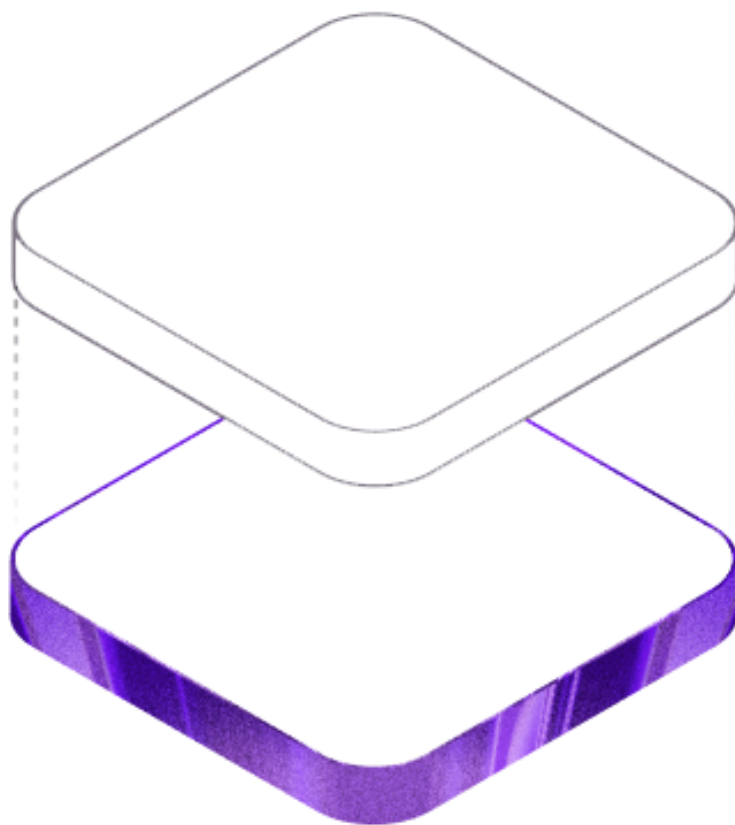


Ghi chú: Code hiện tại dùng AIService gọi Gemini API cho gợi ý viết lại tin nhắn và AI Chatbox. Không mô tả module kiểm tra nội dung độc lập.

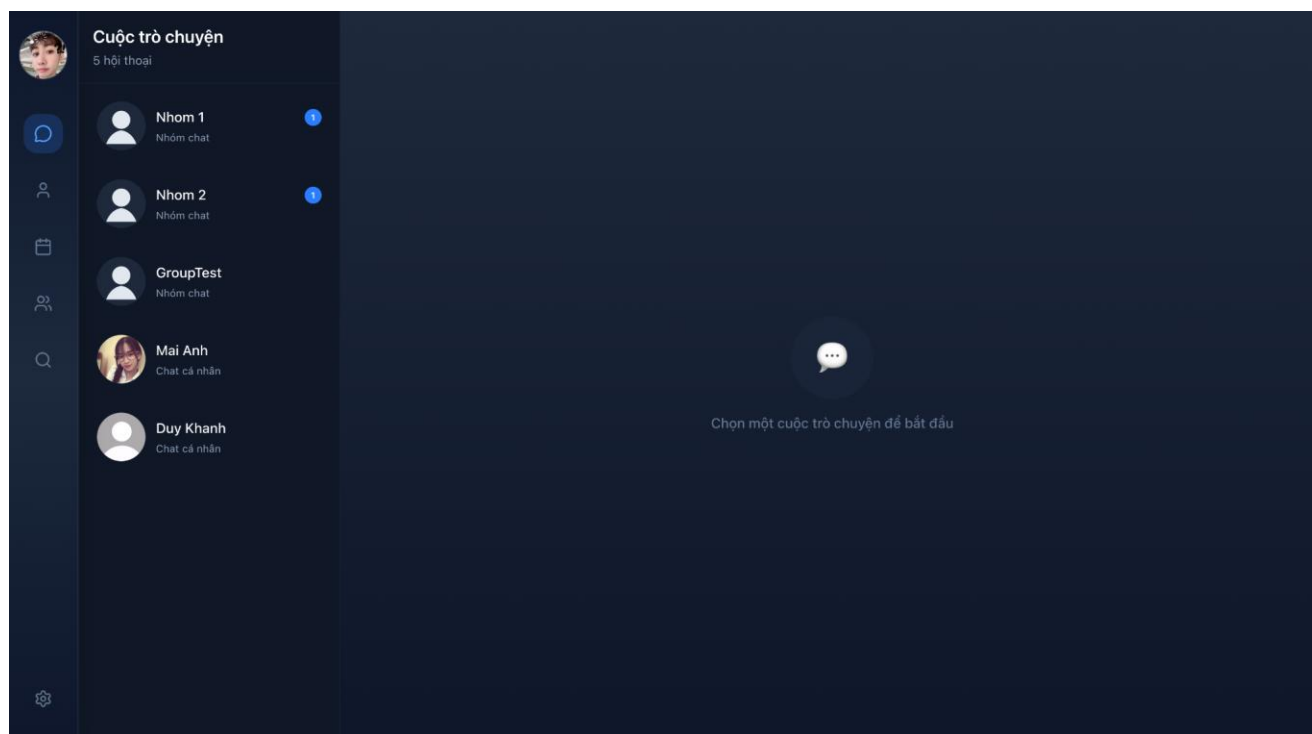
Hình 4.4 Quy trình Gemini AI Assistant trong project

GHI CHÚ THAY SƠ ĐỒ/HÌNH: Dán sơ đồ mới
04_Gemini_AI_Assistant_Flow_v9.drawio.

4.8 Hiện thực giao diện Web



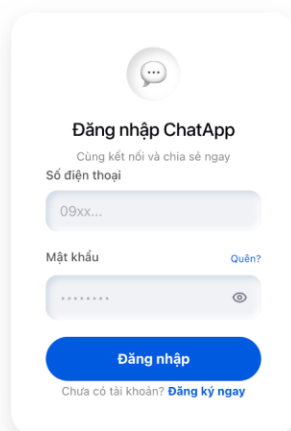
Hình 4.5 Ảnh minh họa/hero có trong project web client



Hình 4.6 Giao diện web minh họa theo bố cục ChatApp

4.8.1 Màn hình đăng nhập

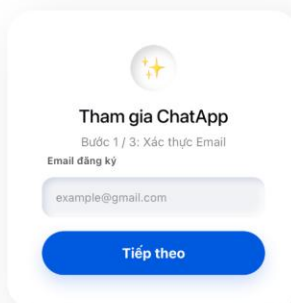
Chức năng để người dùng nhập tài khoản và mật khẩu. Sau khi đăng nhập thành công, token được lưu để gọi các API cần xác thực.



The login screen for ChatApp features a speech bubble icon at the top. Below it, the title "Đăng nhập ChatApp" is displayed, followed by the subtitle "Cùng kết nối và chia sẻ ngay". The form includes a label "Số điện thoại" above a text input field containing "09xx...". Below this is a label "Mật khẩu" above a password input field with masked characters "....." and a "Quên?" link. A blue "Đăng nhập" button is positioned below the password field. At the bottom, a link "Chưa có tài khoản? Đăng ký ngay" is provided.

4.8.2 Màn hình đăng ký

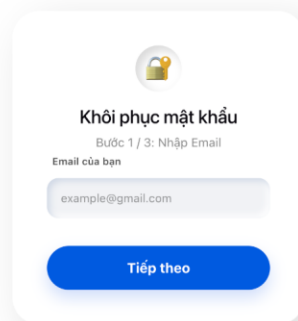
Cho phép nhập thông tin tài khoản mới và chuyển sang bước xác thực OTP.



The registration screen for ChatApp features a star icon at the top. Below it, the title "Tham gia ChatApp" is displayed, followed by the subtitle "Bước 1 / 3: Xác thực Email". The form includes a label "Email đăng ký" above a text input field containing "example@gmail.com". A blue "Tiếp theo" button is positioned below the email field.

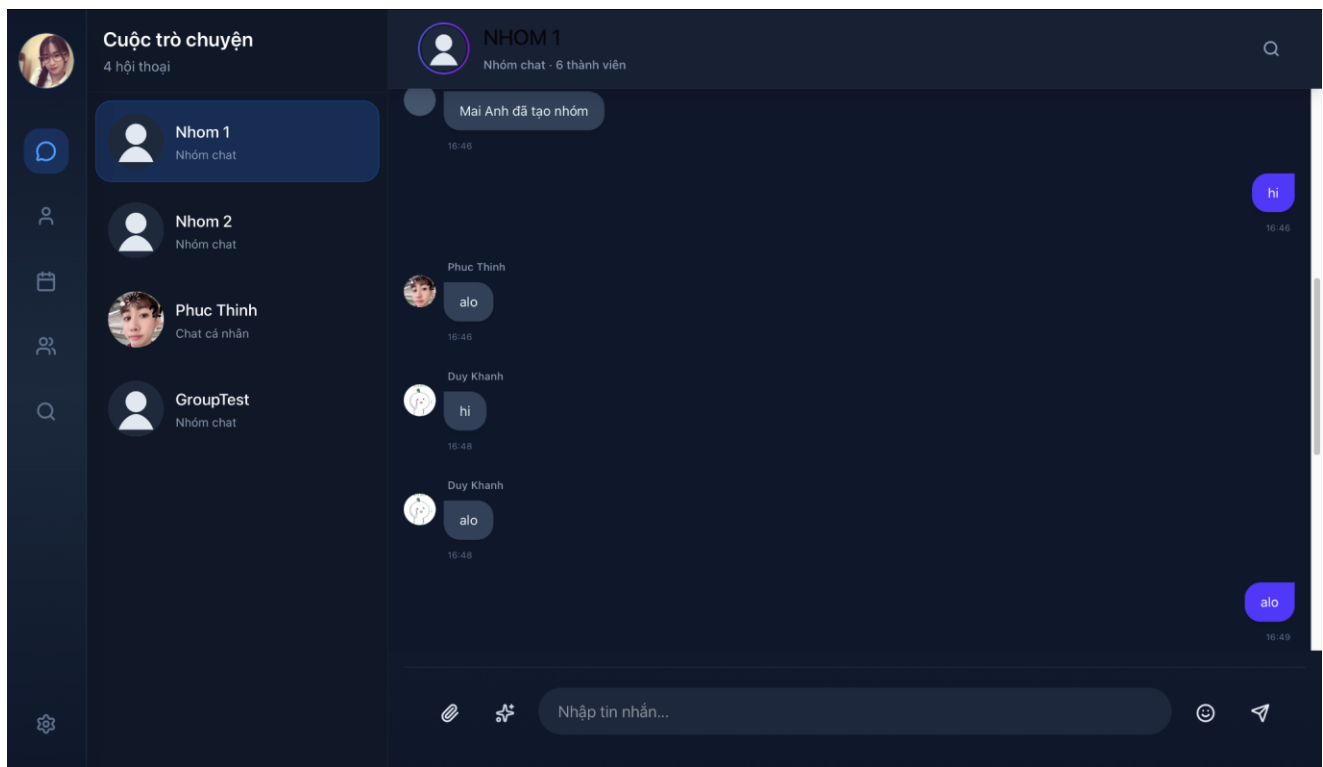
4.8.3 Màn hình quên mật khẩu

Cho phép gửi OTP và đặt mật khẩu mới khi người dùng quên mật khẩu.



4.8.4 Màn hình chat chính

Gồm sidebar, danh sách hội thoại, khung tin nhắn và ô nhập nội dung. Đây là màn hình quan trọng nhất của hệ thống.



4.8.5 Khung nhập tin nhắn

ChatInput hỗ trợ nhập text, gửi file, emoji, gợi ý AI bằng Gemini và gửi message qua socket.



4.8.6 Danh sách hội thoại

ChatList hiển thị các cuộc trò chuyện cá nhân hoặc nhóm, giúp người dùng chuyển room nhanh.

4.8.7 Tìm kiếm bạn bè

FriendSearch hỗ trợ tìm user và gửi lời mời kết bạn.



Cuộc trò chuyện

5 hội thoại



GroupTest

Nhóm chat



Nhom 2

Nhóm chat



Nhom 1

Nhóm chat

1



Mai Anh

Chat cá nhân

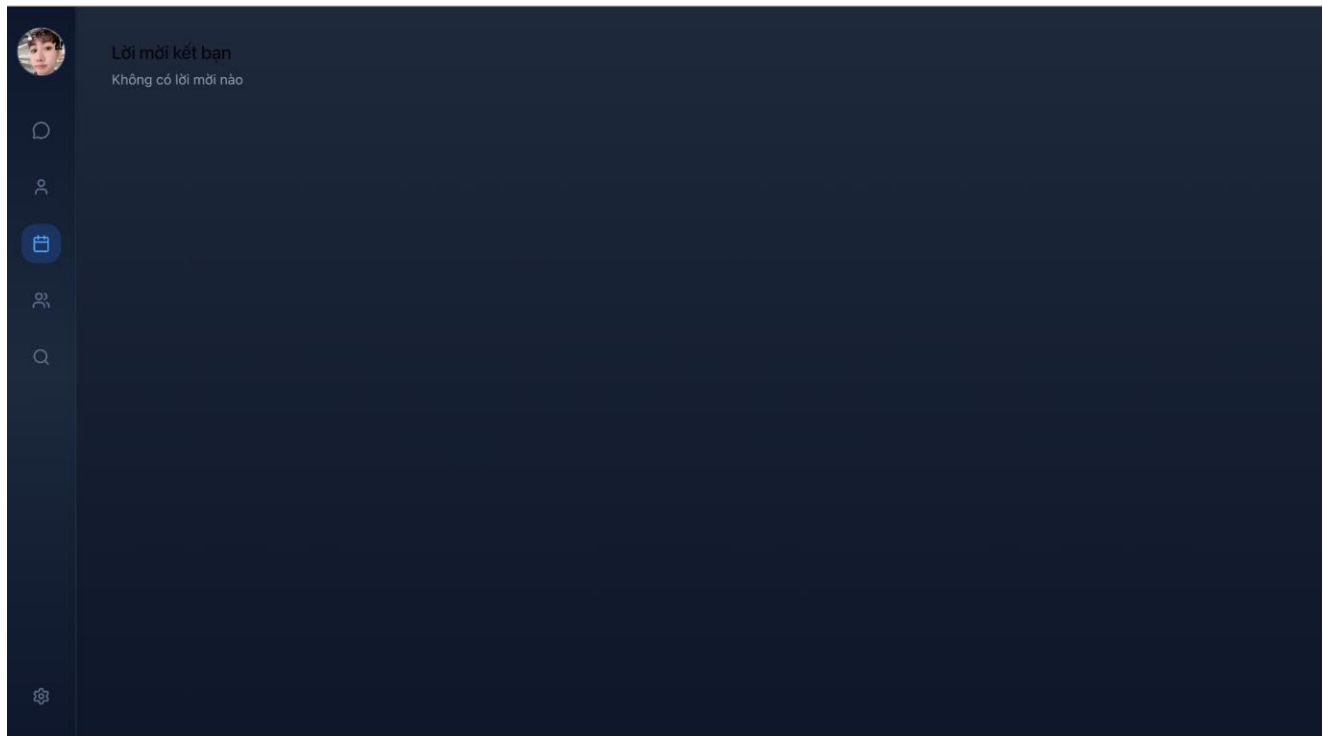


Duy Khanh

Chat cá nhân

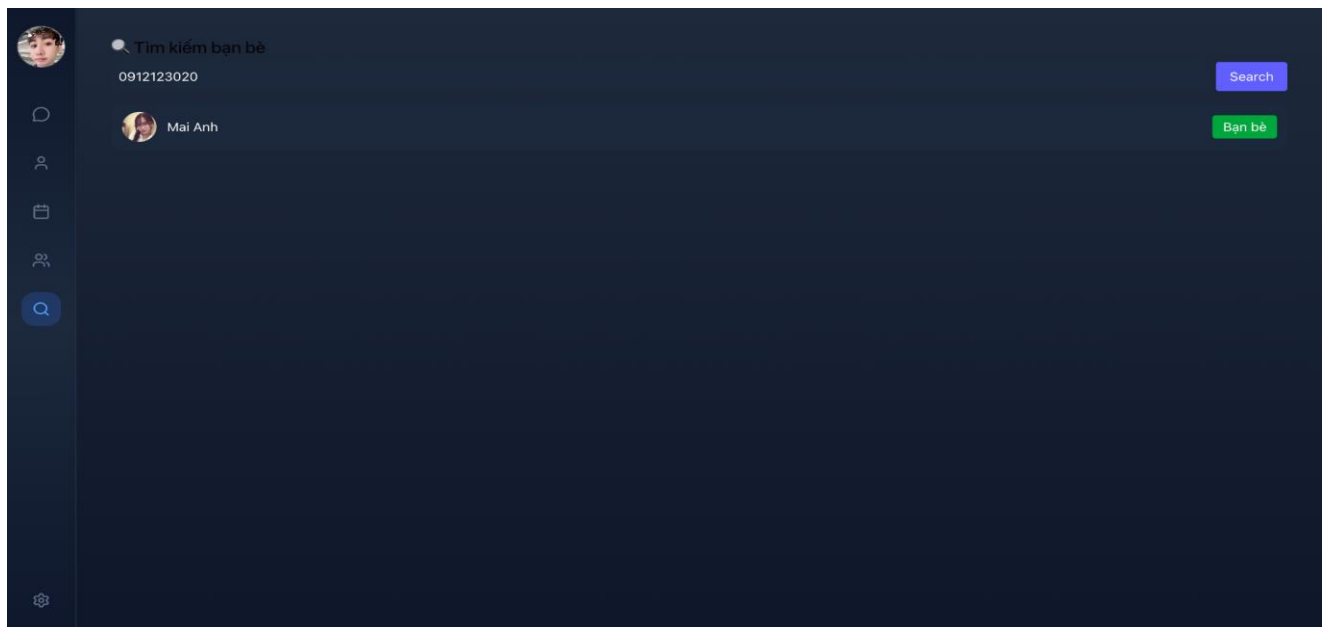
4.8.8 Lời mời kết bạn

FriendRequests hiển thị lời mời đến và cho phép accept/reject.



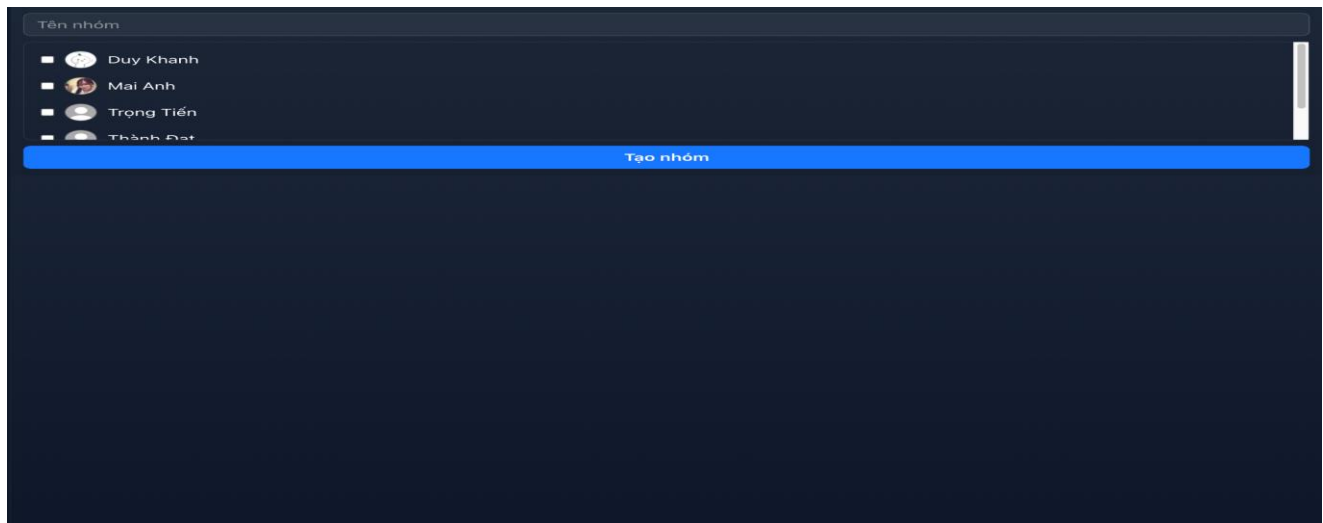
4.8.9 Danh sách bạn bè

FriendsList hiển thị bạn bè đã accept, có thể mở chat nhanh.



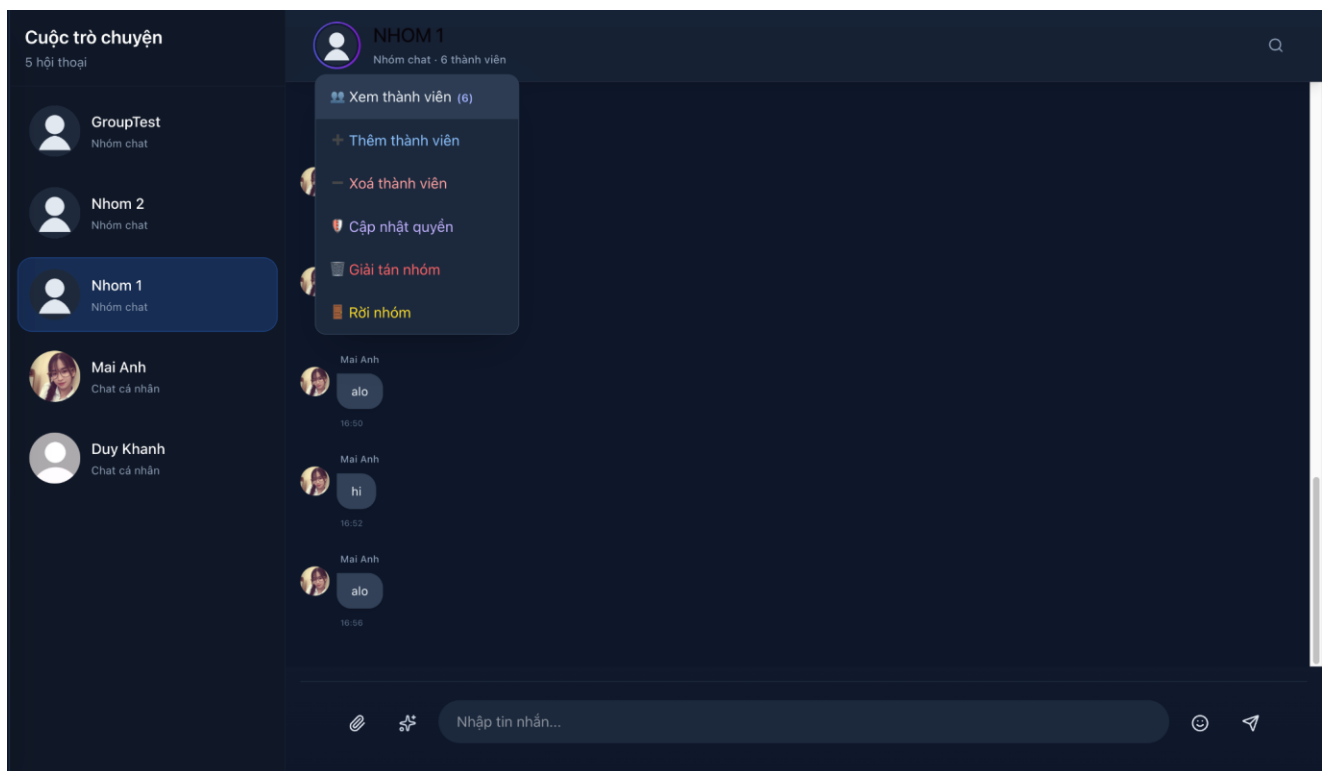
4.8.10 Tạo nhóm

CreateGroup cho phép nhập tên nhóm và chọn thành viên.



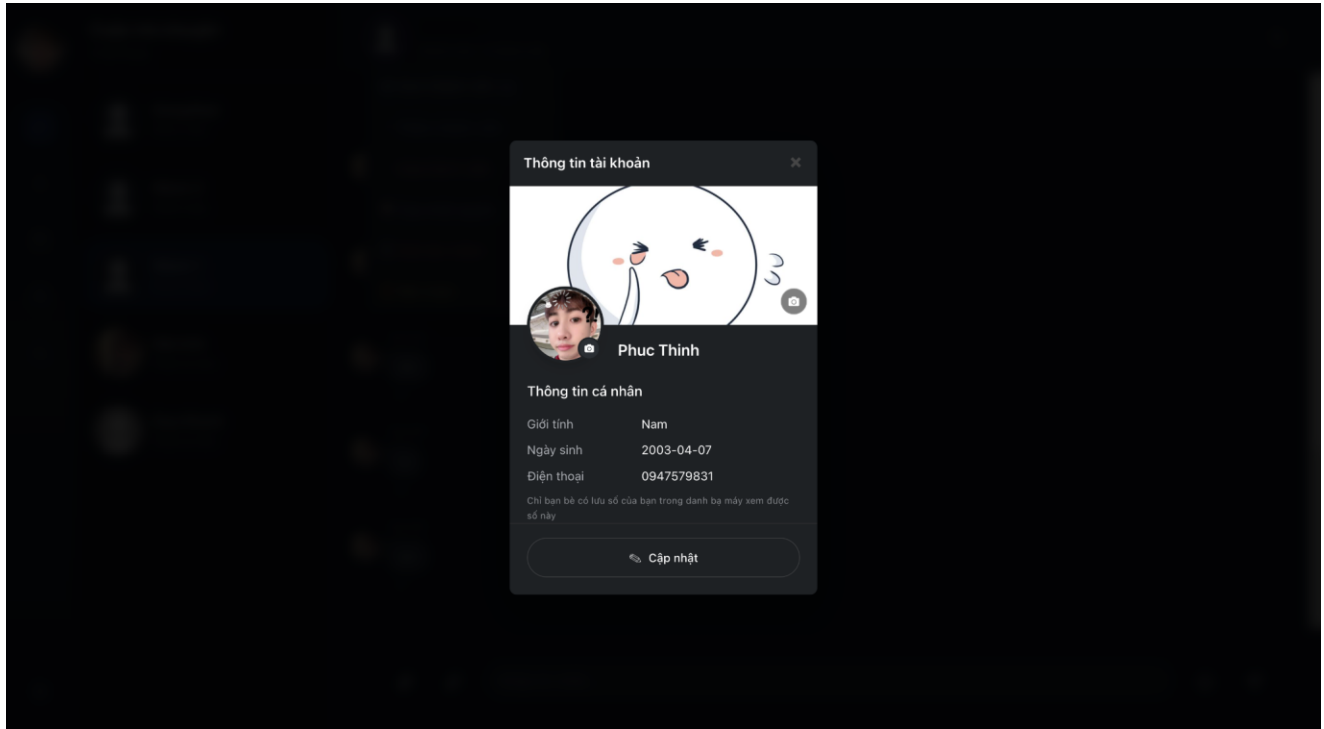
4.8.11 Quản lý nhóm

Giao diện thông tin nhóm cho phép xem thành viên, thêm/xóa thành viên hoặc đổi thông tin nhóm.



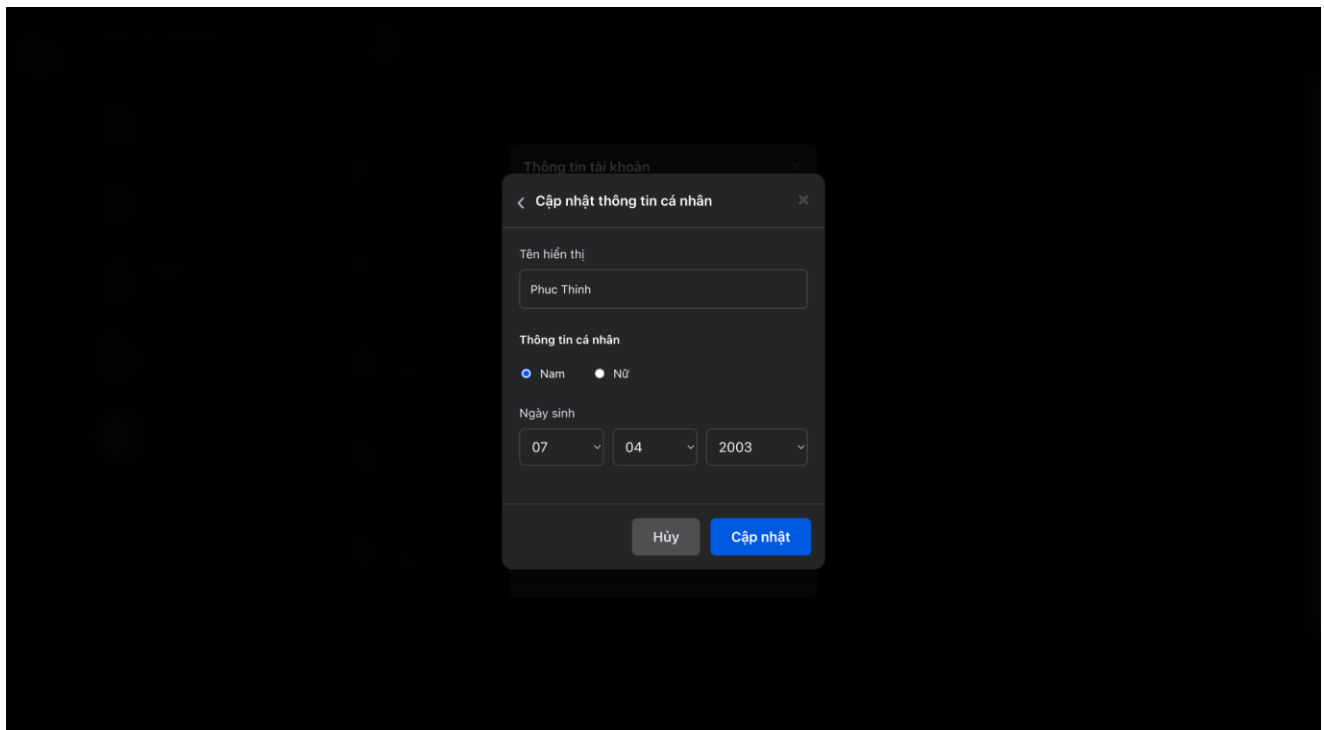
4.8.12 Hồ sơ cá nhân

ProfileModal hiển thị thông tin user, avatar, cover và các nút chỉnh sửa.



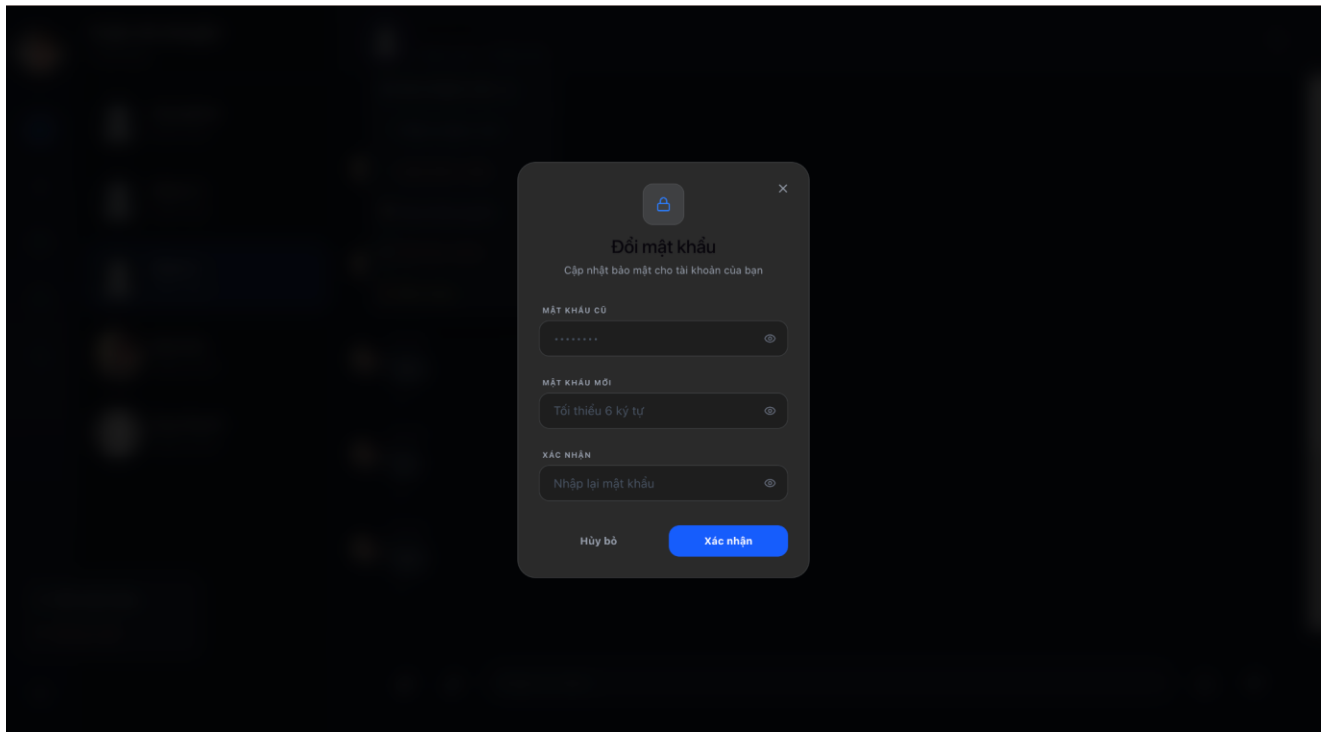
4.8.13 Chỉnh sửa hồ sơ

EditProfileModal cho phép đổi tên, thông tin cá nhân, avatar/cover.



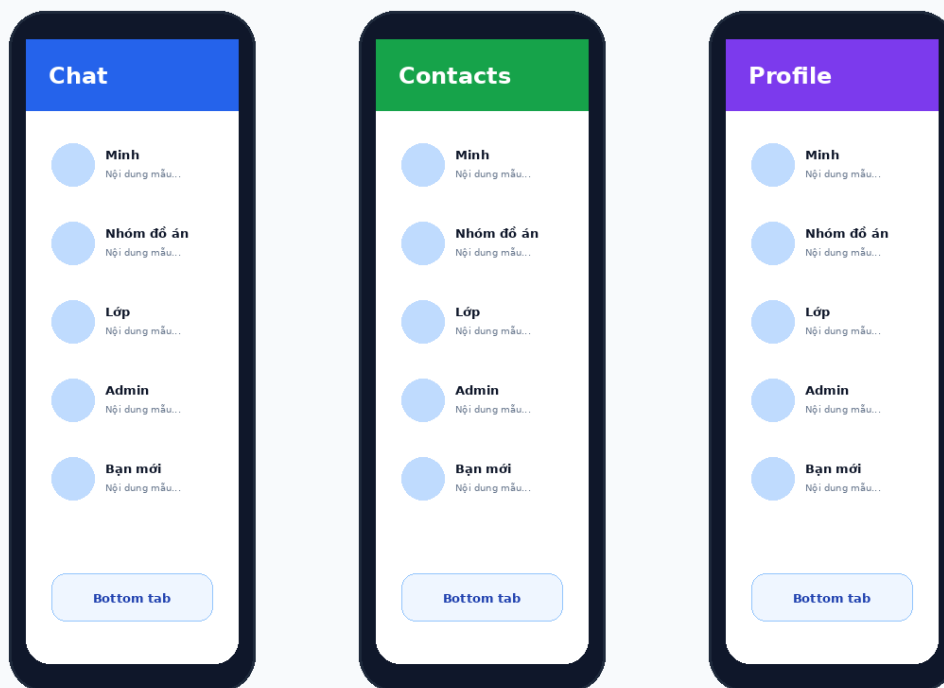
4.8.14 Đổi mật khẩu

ChangePasswordModal hỗ trợ đổi mật khẩu khi user đã đăng nhập.



4.9 Hiện thực giao diện Mobile

Giao diện mobile minh họa



Hình 4.7 Giao diện mobile minh họa theo bố cục ChatApp

4.9.1 Login mobile



Chào mừng trở lại

Đăng nhập để tiếp tục trò chuyện

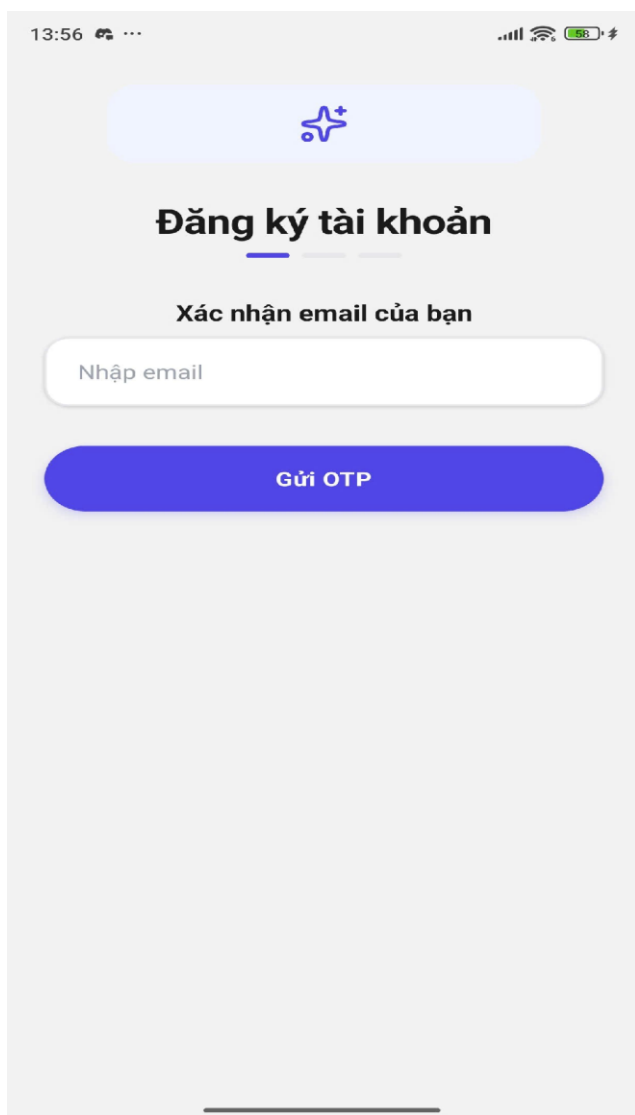


[Quên mật khẩu?](#)

Đăng nhập

Chưa có tài khoản? [Đăng ký ngay](#)

4.9.2 Register mobile



4.9.3 Forgot password mobile

Màn hình quên mật khẩu trong app/(auth)/forgotPassword.tsx.

4.9.4 Tab Chat

Màn hình app/(tabs)/chat.tsx hiển thị danh sách hội thoại.

4.9.5 Room chat

Màn hình app/chat/[roomId].tsx hiển thị tin nhắn trong một room.

4.9.6 Tab Contacts

Màn hình app/(tabs)/contacts.tsx hiển thị danh bạ/bạn bè.

4.9.7 Tab Discovery

Màn hình app/(tabs)/discovery.tsx có thể dùng cho khám phá/tìm kiếm.

4.9.8 Profile mobile

Màn hình app/(tabs)/profile.tsx và profile-detail/profileInfo.

4.9.9 Edit profile mobile

Màn hình editprofile.tsx dùng để chỉnh sửa hồ sơ.

4.9.10 Change password mobile

Màn hình changePassword.tsx hỗ trợ đổi mật khẩu.

CHƯƠNG 5: KIỂM THỬ HỆ THỐNG

5.1 Mục tiêu kiểm thử

Kiểm thử được thực hiện để đảm bảo các chức năng chính hoạt động đúng theo yêu cầu đã phân tích. Nhóm tập trung vào kiểm thử thủ công theo kịch bản vì đây là đồ án môn học, đồng thời ghi nhận các lỗi phát sinh để điều chỉnh giao diện và backend.

Các nhóm chức năng cần kiểm thử gồm tài khoản, OTP, hồ sơ, bạn bè, block, chat cá nhân, chat nhóm, bình chọn nhóm, upload file, AI Gemini, gọi demo, web client và mobile client. Mỗi test case có dữ liệu đầu vào, thao tác và kết quả mong đợi.

5.2 Kịch bản kiểm thử chức năng

Project Name	ChatAppZALO	Creator	Phạm Tấn Đạt, Phúc Thịnh
Project Code	Module01	Reviewer/Approver	Tôn Long Phước
Document Code	UT-CHATAPP-01	Issue Date	02/06/2026
Notes	Release kiểm thử bao gồm 19 use case chính của hệ thống ChatAppZALO.		

No	Function code	Passed	Failed	Untested	N	A	B	Total Test Cases
1	UT Lab 1	8			2	4	2	8
2	UT Lab 2	6			2	3	1	6
3	UT Lab 3	8			3	4	1	8
4	UT Lab 4	6			2	3	1	6
5	UT Lab 5	6			3	2	1	6
6	UT Lab 6	6			3	2	1	6
7	UT Lab 7	7			3	4		7

8	UT Lab 8	5			3	2		5
9	UT Lab 9	5			3	2		5
10	UT Lab10	8			6	1	1	8
11	UT Lab11	7			5	1	1	7
12	UT Lab12	6			3	2	1	6
13	UT Lab13	6			4	2		6
14	UT Lab14	7			5	2		7
15	UT Lab15	7			5	2		7
16	UT Lab16	6			5	1		6
17	UT Lab17	5			4	1		5
18	UT Lab18	6			4	2		6
Sub total		115			65	40	10	115
Test coverage		100.00	%					
Test successful coverage		100.00	%					
Normal case		56.52	%					
Abnormal case		34.78	%					
Boundary case		8.70	%					

5.3 Kiểm thử giao diện

Màn hình	Nội dung cần kiểm tra
Login/Register	Form dễ nhìn, báo lỗi rõ, không bị vỡ layout khi nhập thiếu dữ liệu.
Chat chính	Sidebar, chat list và chat box hiển thị cân đối; tin nhắn của mình và người khác phân biệt được.
Bạn bè	Tìm kiếm, lời mời, danh sách bạn bè có nút thao tác rõ ràng.

Nhóm	Form tạo nhóm và thông tin nhóm để thao tác.
Profile	Avatar, cover, thông tin cá nhân và nút chỉnh sửa hiển thị đúng.
AI Gemini	Popup gợi ý tin nhắn và hộp AI Chatbox hiển thị rõ, không che nội dung chính.
Mobile	Các tab chính hoạt động, màn hình vừa khung điện thoại, không bị tràn chữ.

5.4 Nhận xét sau kiểm thử

Qua kiểm thử, các luồng chức năng chính có thể trình diễn được. Hệ thống đã thể hiện được quá trình đăng ký, xác thực, đăng nhập, kết bạn, chat realtime, tạo nhóm, upload file và AI Gemini. Đây là các yêu cầu quan trọng nhất của đề tài.

Một số phần vẫn cần hoàn thiện thêm nếu triển khai thực tế, ví dụ kiểm thử tải nhiều người dùng online, tối ưu giao diện responsive, kiểm thử bảo mật sâu, xử lý refresh token và tối ưu tính năng gọi audio/video. Những điểm này được đưa vào hạn chế và hướng phát triển.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết quả đạt được

Nhóm đã xây dựng được hệ thống ChatAppZALO có đầy đủ backend, web client và mobile client. Các chức năng cốt lõi như đăng ký, OTP, đăng nhập, hồ sơ, bạn bè, block, chat cá nhân, nhóm chat, upload file, tin nhắn realtime, bình chọn nhóm, reaction, ghim tin và AI Gemini đã được phân tích và hiện thực ở mức đồ án.

Về mặt kỹ thuật, đề tài giúp nhóm áp dụng nhiều kiến thức quan trọng: thiết kế API, JWT authentication, phân lớp controller-service-repository, lưu trữ MySQL/MongoDB, xử lý file, WebSocket/STOMP, component React và route mobile bằng Expo.

Về mặt báo cáo, tài liệu đã bố trí sẵn các vị trí cần chèn sơ đồ và ảnh giao diện. Điều này giúp nhóm dễ hoàn thiện bản nộp cuối cùng bằng cách chụp ảnh phần mềm đang chạy và vẽ lại các sơ đồ UML bằng draw.io.

6.2 Hạn chế

Hạn chế	Mô tả
Giao diện	Một số màn hình còn cần chỉnh thêm spacing, responsive và hiệu ứng để chuyên nghiệp hơn.
Hiệu năng	Chưa kiểm thử với số lượng lớn người dùng online và số lượng tin nhắn lớn.
Bảo mật	Chưa hoàn thiện refresh token rotation, rate limit theo endpoint và quản lý thiết bị đăng nhập.
Realtime	Chức năng gọi audio/video mới ở mức demo 1-1; gọi nhóm chưa hoàn thiện trong project hiện tại.
AI	Gemini AI hiện hỗ trợ gợi ý viết lại tin nhắn và AI Chatbox; chưa có RAG hay tìm kiếm ngữ nghĩa trong lịch sử chat.
Triển khai	Chưa đóng gói production hoàn chỉnh bằng Docker Compose/Kubernetes, chưa cấu hình CI/CD đầy đủ.
Kiểm thử	Chủ yếu kiểm thử thủ công, chưa có bộ unit test/integration test đầy đủ.

6.3 Hướng phát triển

Hướng phát triển	Nội dung đề xuất
Hoàn thiện UI/UX	Thiết kế lại giao diện đồng bộ hơn, thêm dark mode, animation và responsive tốt hơn.
Nâng cấp realtime	Tối ưu room subscription, unread count, typing indicator và read receipt chi tiết hơn.
Bảo mật	Thêm refresh token, rate limit, quản lý thiết bị đăng nhập và xác thực 2 lớp.

AI nâng cao	Bổ sung tóm tắt hội thoại, dịch tự động, tìm kiếm tin nhắn theo ngữ nghĩa và context nhiều phòng chat.
Media	Hoàn thiện gọi audio/video 1-1, sau đó bổ sung gọi nhóm bằng WebRTC hoặc media server/SFU.
Triển khai cloud	Đóng gói Docker, deploy lên cloud, dùng S3/CloudFront cho file, cấu hình CI/CD.
Kiểm thử tự động	Viết unit test cho service, integration test cho controller và test giao diện cơ bản.

6.4 Kết luận chung

ChatApp là đề tài phù hợp để vận dụng tổng hợp kiến thức về phát triển phần mềm. Thông qua quá trình thực hiện, nhóm hiểu rõ hơn cách một hệ thống chat được thiết kế từ dữ liệu, API, realtime đến giao diện người dùng.

Mặc dù hệ thống vẫn còn điểm cần cải thiện, phiên bản hiện tại đã đủ để trình diễn các chức năng chính và làm nền tảng cho các hướng phát triển tiếp theo. Nếu có thêm thời gian, nhóm sẽ tiếp tục tối ưu giao diện, nâng cấp bảo mật, hoàn thiện gọi audio/video nhóm và mở rộng AI Gemini theo hướng chatbot hoặc tóm tắt hội thoại.

TÀI LIỆU THAM KHẢO

Các tài liệu Tiếng Anh

- [1]. Craig Walls - Spring in Action, Sixth Edition – Manning Publications, 2022
- [2]. Kathy Sierra, Bert Bates & Trisha Gee - Head First Java, Third Edition – O'Reilly Media, 2022
- [3]. Alex Banks & Eve Porcello - Learning React, Second Edition – O'Reilly Media, 2020
- [4]. Bonnie Eisenman - Learning React Native, Second Edition – O'Reilly Media, 2017
- [5]. Shannon Bradshaw, Eoin Brazil & Kristina Chodorow - MongoDB: The Definitive Guide, Third Edition – O'Reilly Media, 2019

Các tài liệu từ Internet

- [6]. <https://spring.io/projects/spring-boot>
- [7]. <https://docs.spring.io/spring-boot/>
- [8]. <https://react.dev/>
- [9]. <https://vite.dev/>
- [10]. <https://reactnative.dev/>
- [11]. <https://docs.expo.dev/>
- [12]. <https://dev.mysql.com/doc/>
- [13]. <https://www.mongodb.com/docs/>
- [14]. <https://stomp.github.io/>
- [15]. <https://docs.docker.com/>
- [16]. <https://ai.google.dev/gemini-api/docs>
- [17]. <https://ai.google.dev/gemini-api/docs>
- [18]. <https://ai.google.dev/gemini-api/docs>
- [19]. <https://ai.google.dev/gemini-api/docs>